



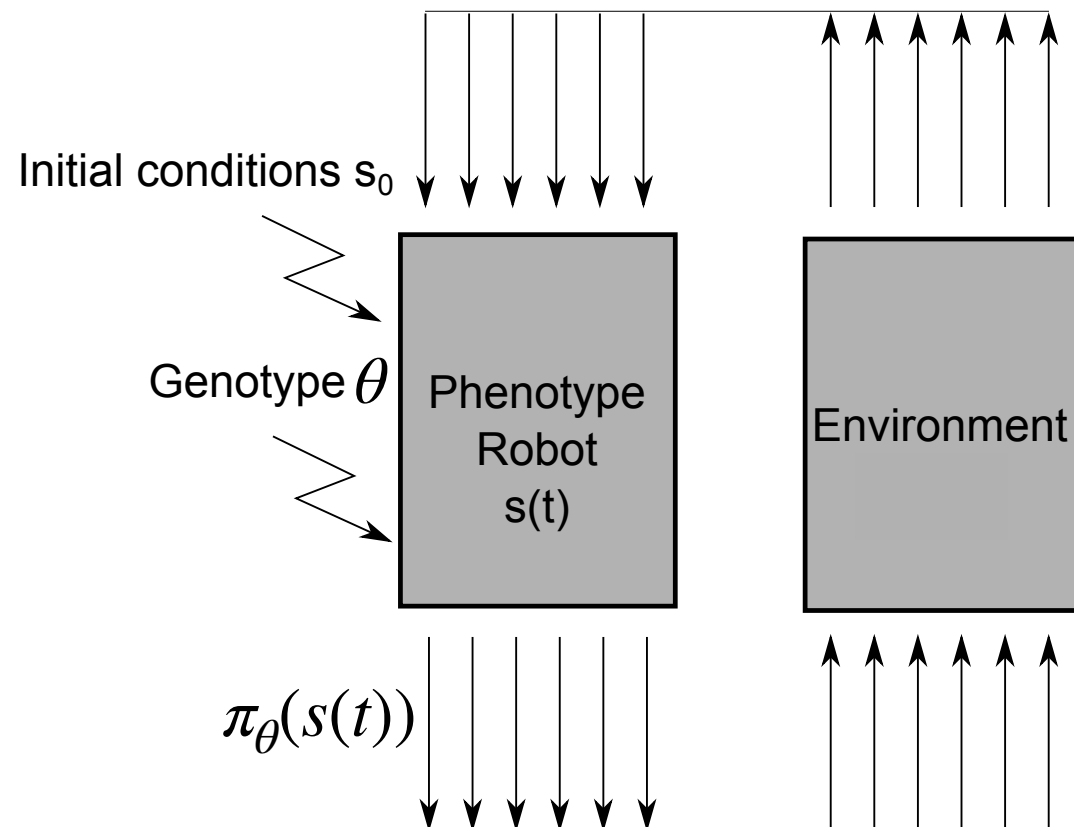
Multi-objective Optimization with Evolutionary Algorithms

Stéphane Doncieux
Professor



Reinforcement learning: direct policy search

- Parameterised policy: $\pi_{\theta} : S \rightarrow A$
- Find θ maximising the cumulated reward $f(\theta) \in \mathbb{R}$ during an episode



$$s(t+1) = G(s(t), \pi_{\theta}(s(t)))$$

where:

- $G(.)$ models the robot and its environment

$$f(\theta) = \sum_{t=0}^T R(s(t), a(t))$$

Beyond multi-objective problems resolution examples of use of multi-objective approaches

1. From single to multi-objective problem: **multi-objectivization**

- Addition of new « helper objectives » [Jensen 2004]

Beyond multi-objective problems resolution examples of use of multi-objective approaches

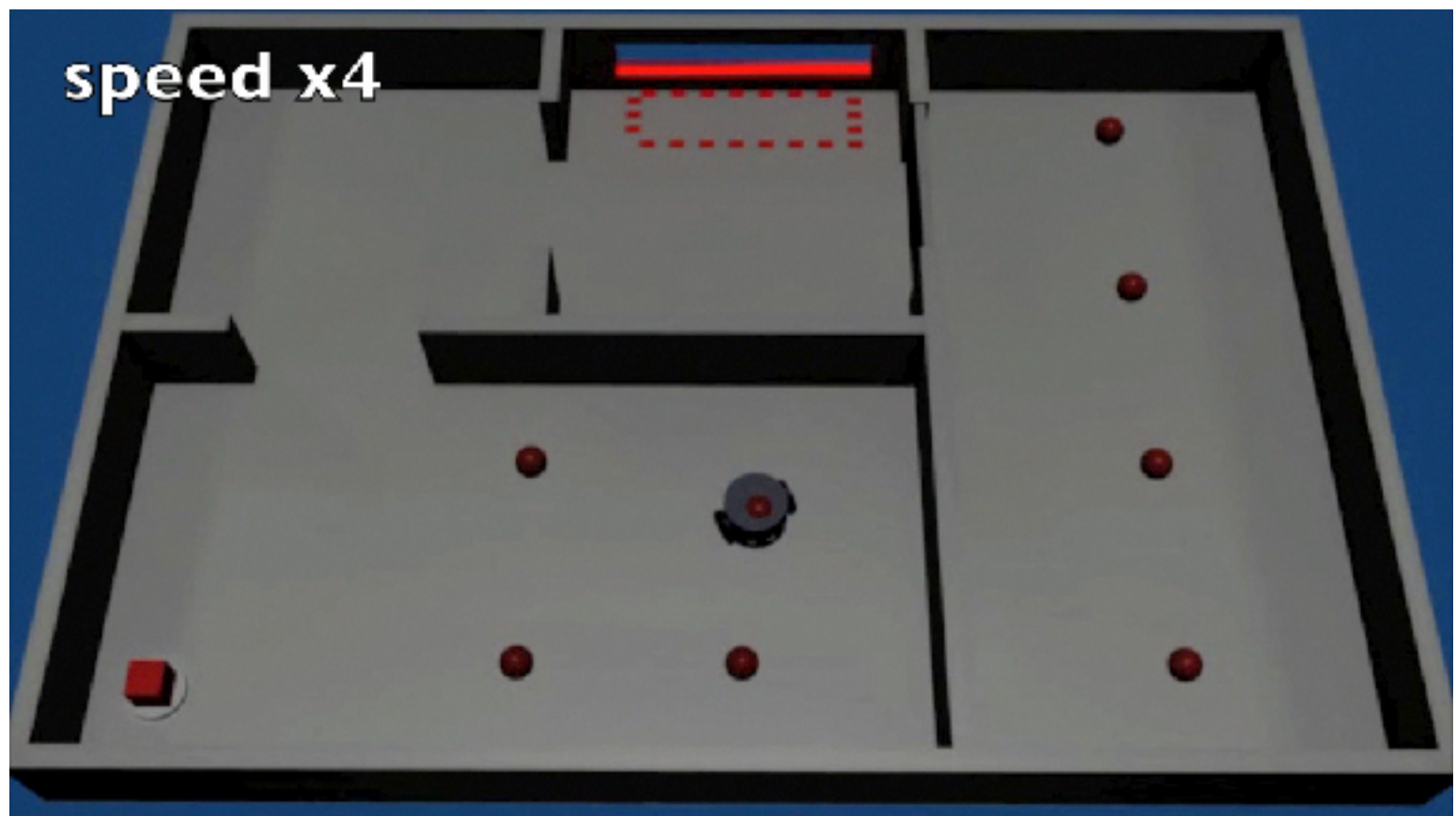
1. From single to multi-objective problem: **multi-objectivization**

- Addition of new « helper objectives » [Jensen 2004]
- Application to learning in robotics:

Fitness objectives:

1. nb_{ball}
2. Behavioral diversity
(Objective rewarding exploration)

Multi-objective EA:
NSGA-II



<https://github.com/doncieux/collectball>

Beyond multi-objective problems resolution examples of use of multi-objective approaches

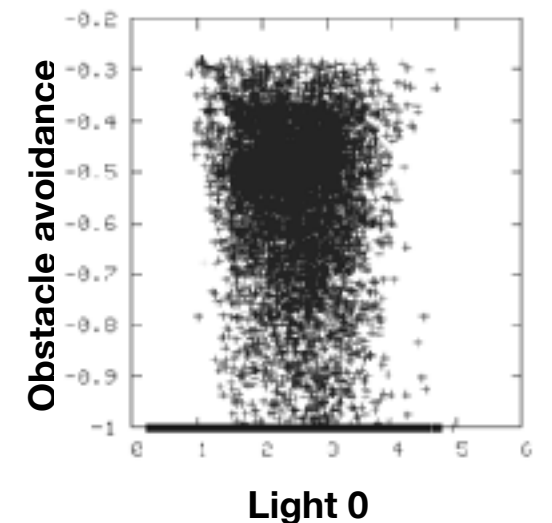
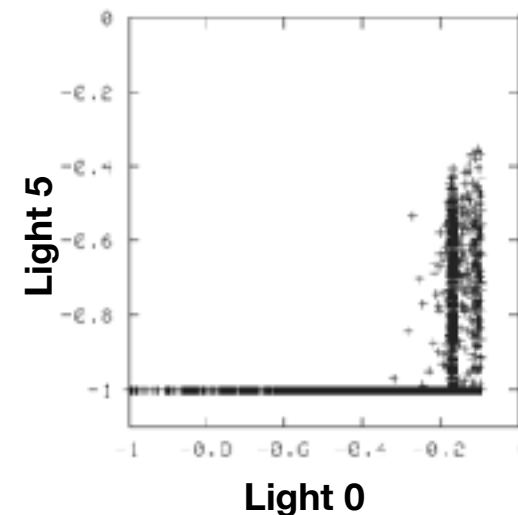
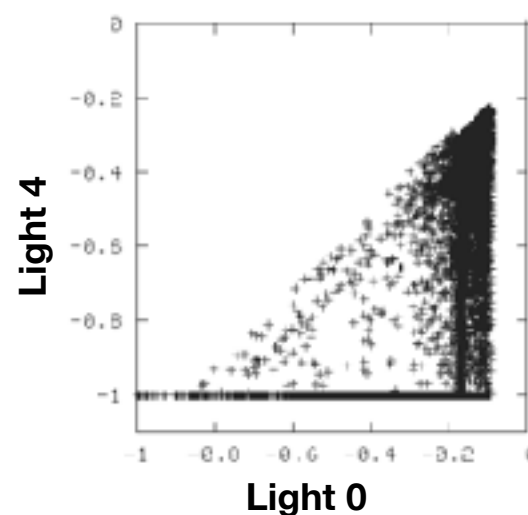
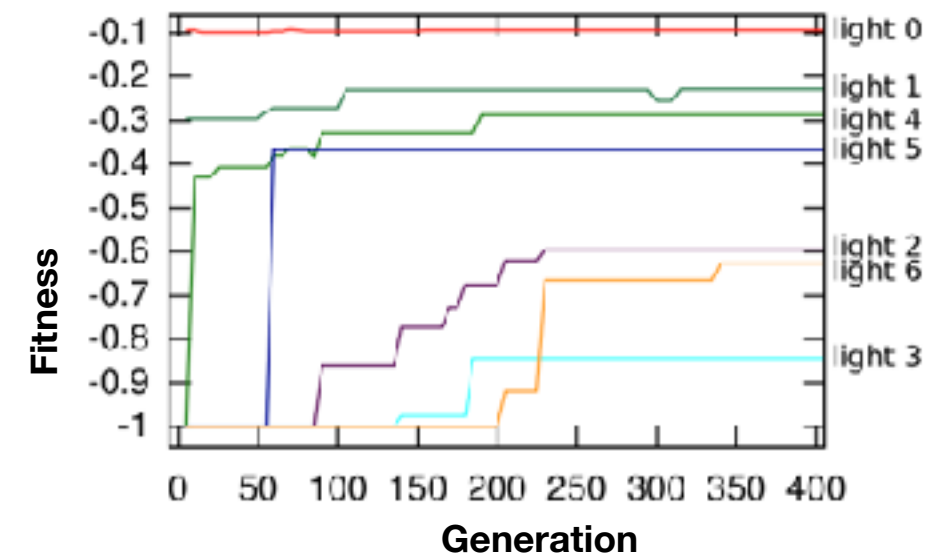
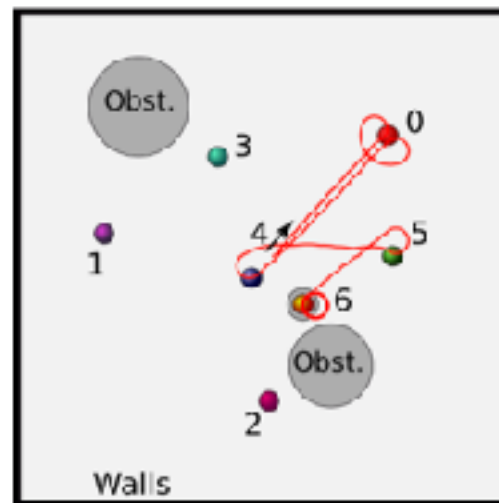
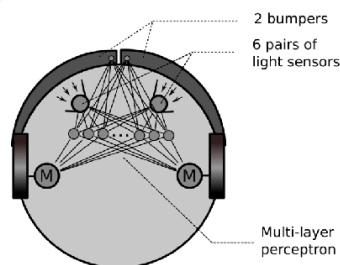
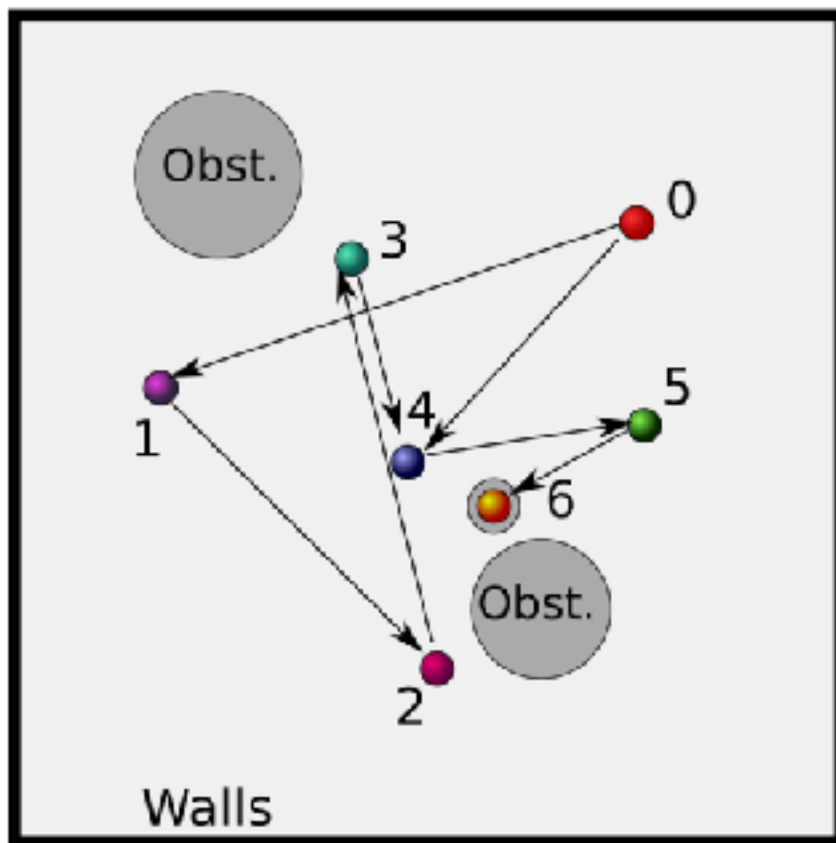
1. From single to multi-objective problem: **multi-objectivization**

- Decomposition of the single objective [Knowles et al 2001]

Beyond multi-objective problems resolution examples of use of multi-objective approaches

1. From single to multi-objective problem: **multi-objectivization**

- Decomposition of the single objective [Knowles et al 2001]
- Application to learning in robotics:



Beyond multi-objective problems resolution examples of use of multi-objective approaches

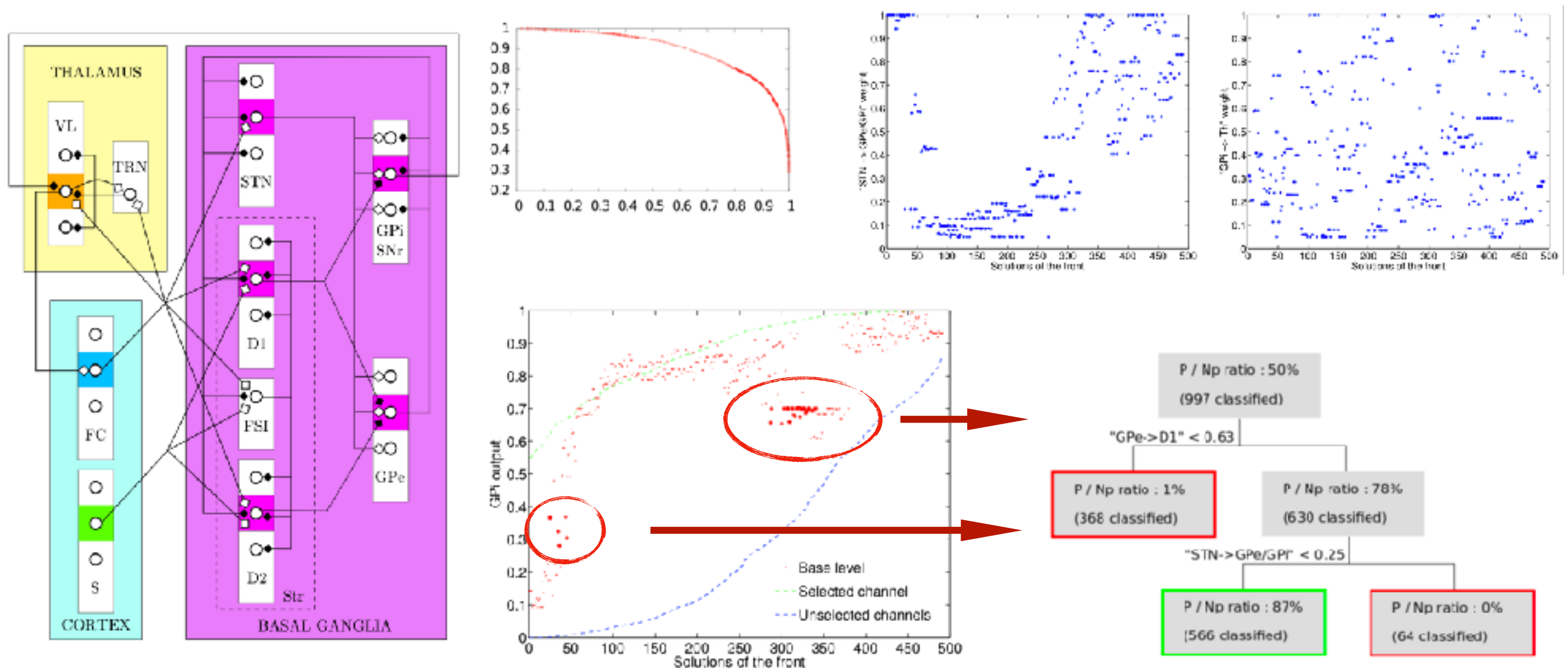
2. Getting more from the Pareto set and Pareto front: **Innovization**

- Using machine learning to go beyond the Pareto set found [Deb and Srinivasan 2006]

Beyond multi-objective problems resolution examples of use of multi-objective approaches

2. Getting more from the Pareto set and Pareto front: **Innovization**

- Using machine learning to go beyond the Pareto set found [Deb and Srinivasan 2006]
- Application to the study of a computational model of action selection:



Overview



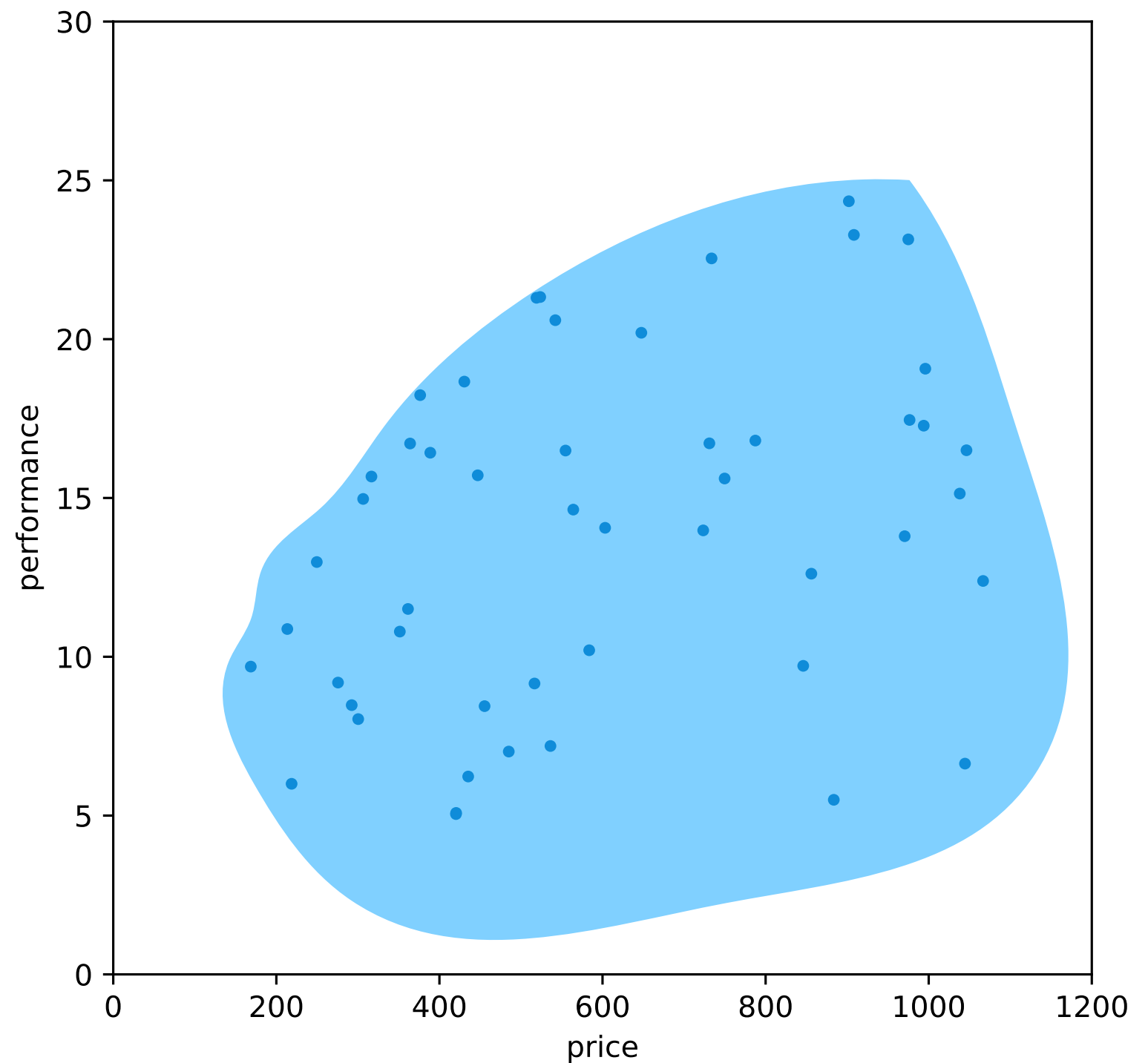
- Introduction
- Back to single-objective
- Multi-objective optimization
- Many-objective optimization

Overview

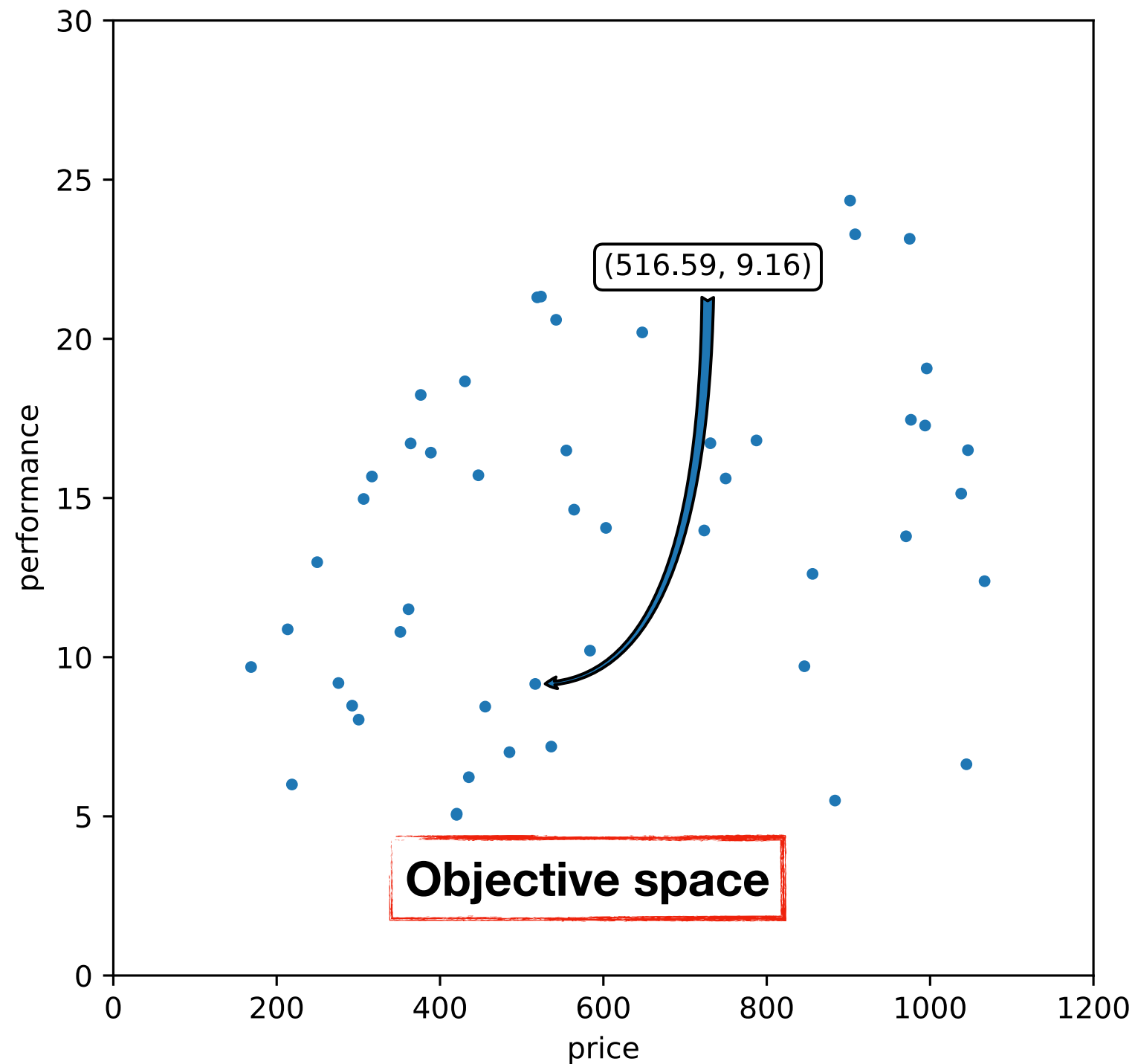


- Introduction
- Back to single-objective
- Multi-objective optimization
- Many-objective optimization

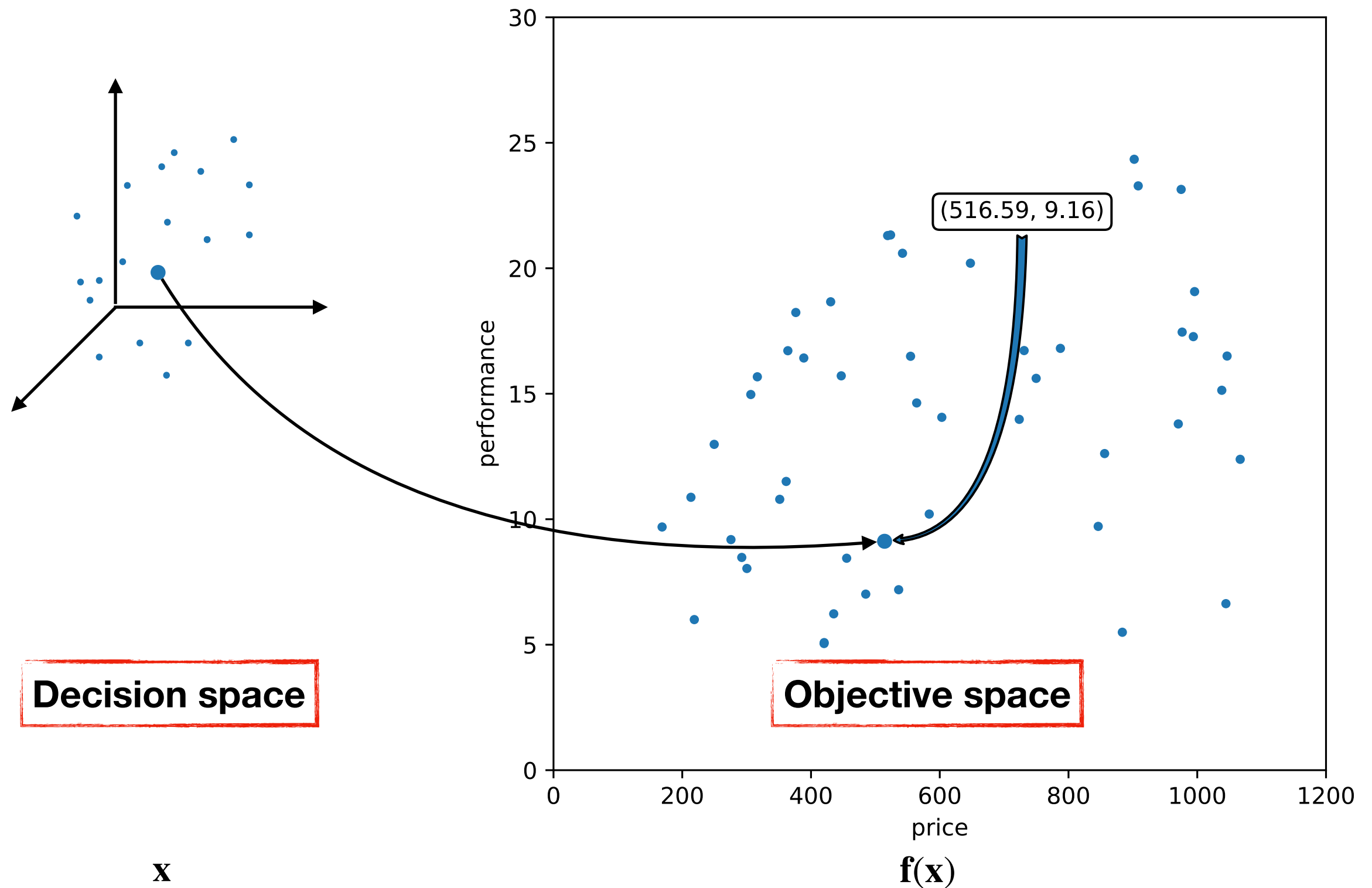
Multi-Objective problem



Multi-Objective problem



Multi-Objective problem



Multi-Objective problem

- Mathematical formulation:

$$\text{minimize/maximize } \mathbf{F}(x) = (f_1(x), \dots, f_m(x))^T$$

$$\text{subject to } g_j(x) \geq a_j, j = 1, \dots, q$$

$$h_j(x) = b_j, j = 1, \dots, l$$

$$x \in \Omega$$

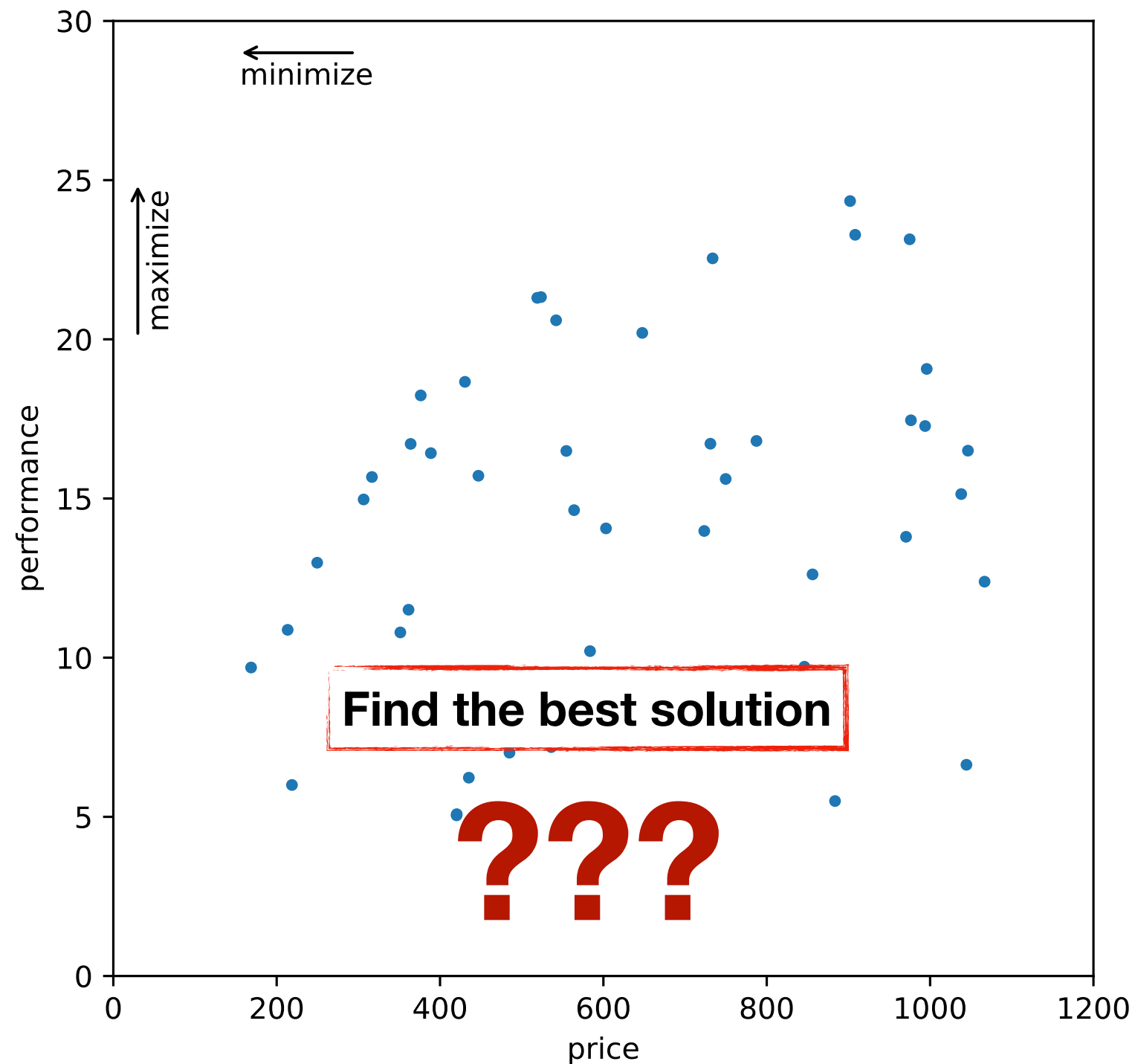
x : decision variable

$\mathbf{F}$: objective vector

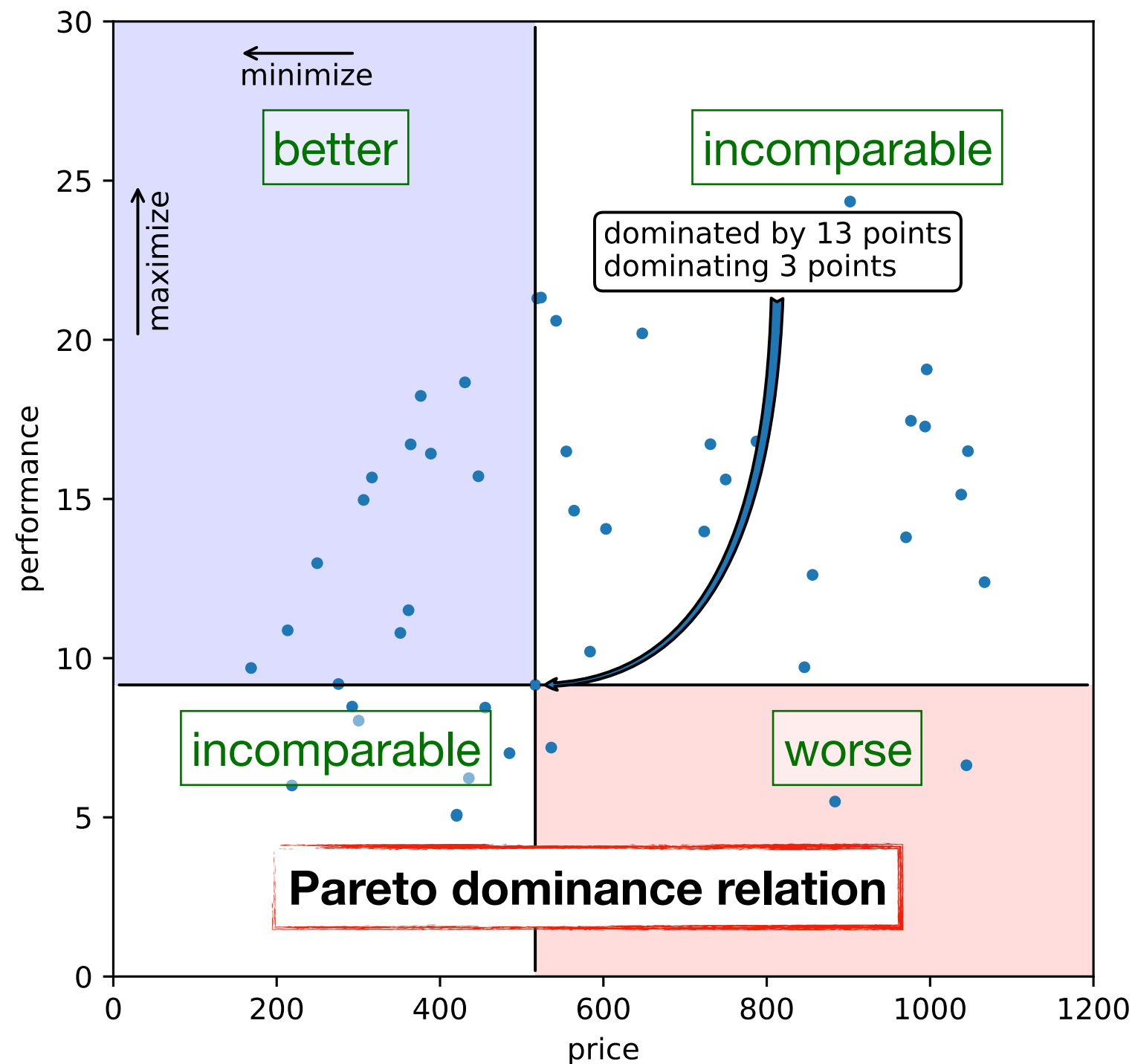
Ω : decision space

$\Omega \rightarrow \mathbb{R}^m$: objective space

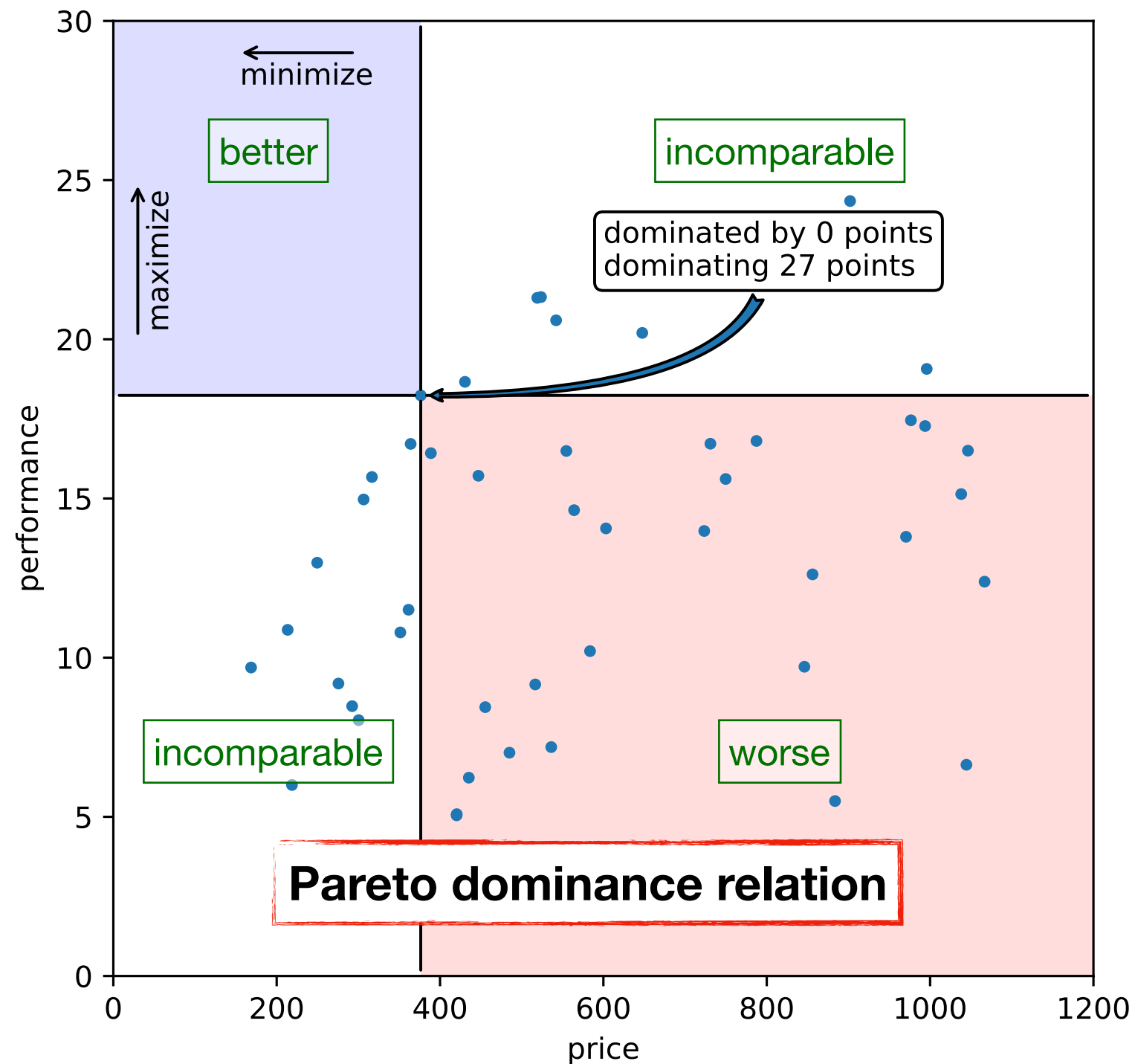
Multi-Objective problem



Multi-Objective problem



Multi-Objective problem



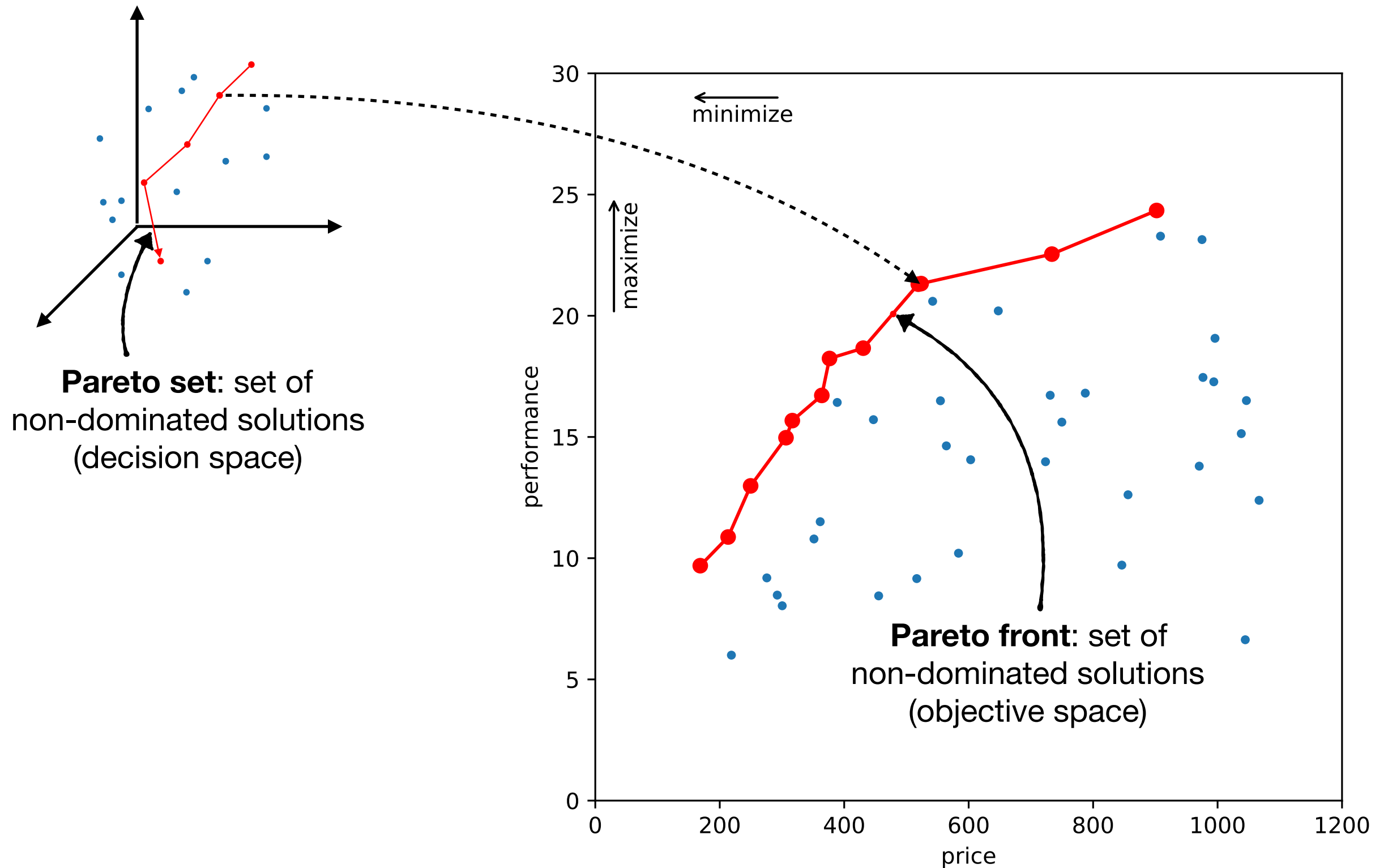
Pareto dominance relation

- **Comparison in the objective space**
- A solution **a** dominates a solution **b** (minimization) if:

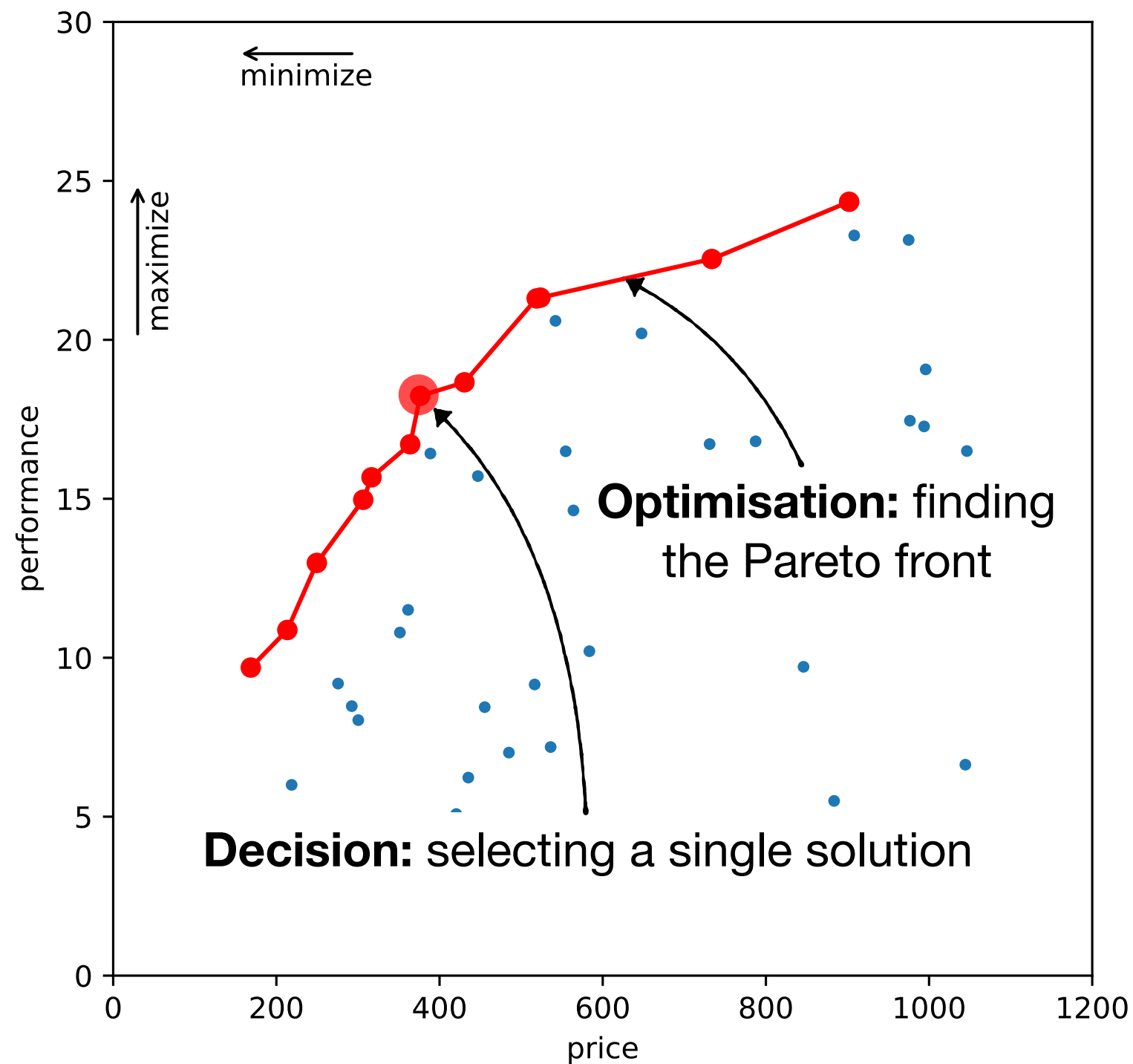
$$\forall i \in \{1, \dots, m\} : f_i(\mathbf{a}) \leq f_i(\mathbf{b})$$

$$\exists j \in \{1, \dots, m\} : f_j(\mathbf{a}) < f_j(\mathbf{b})$$

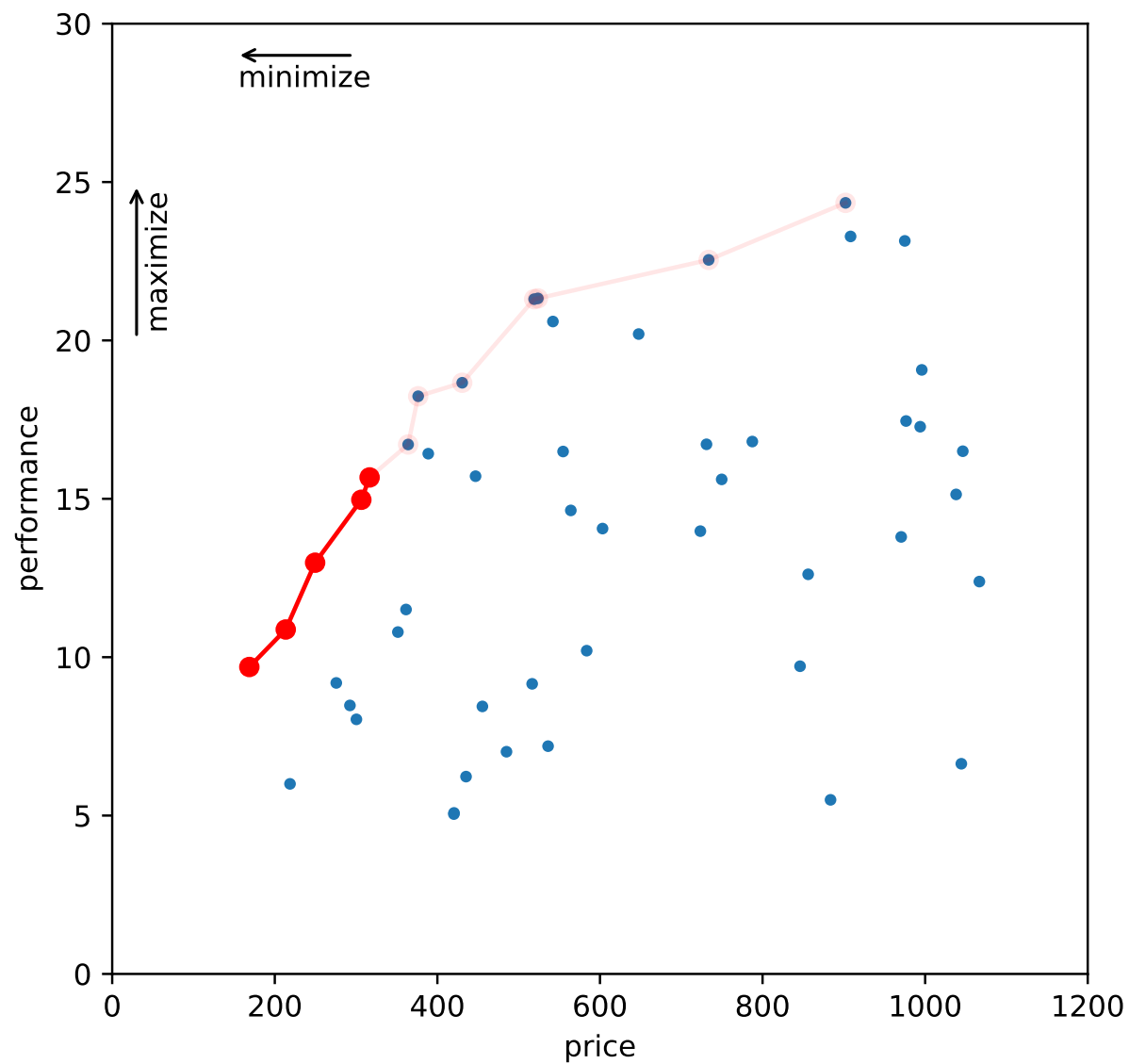
Pareto front



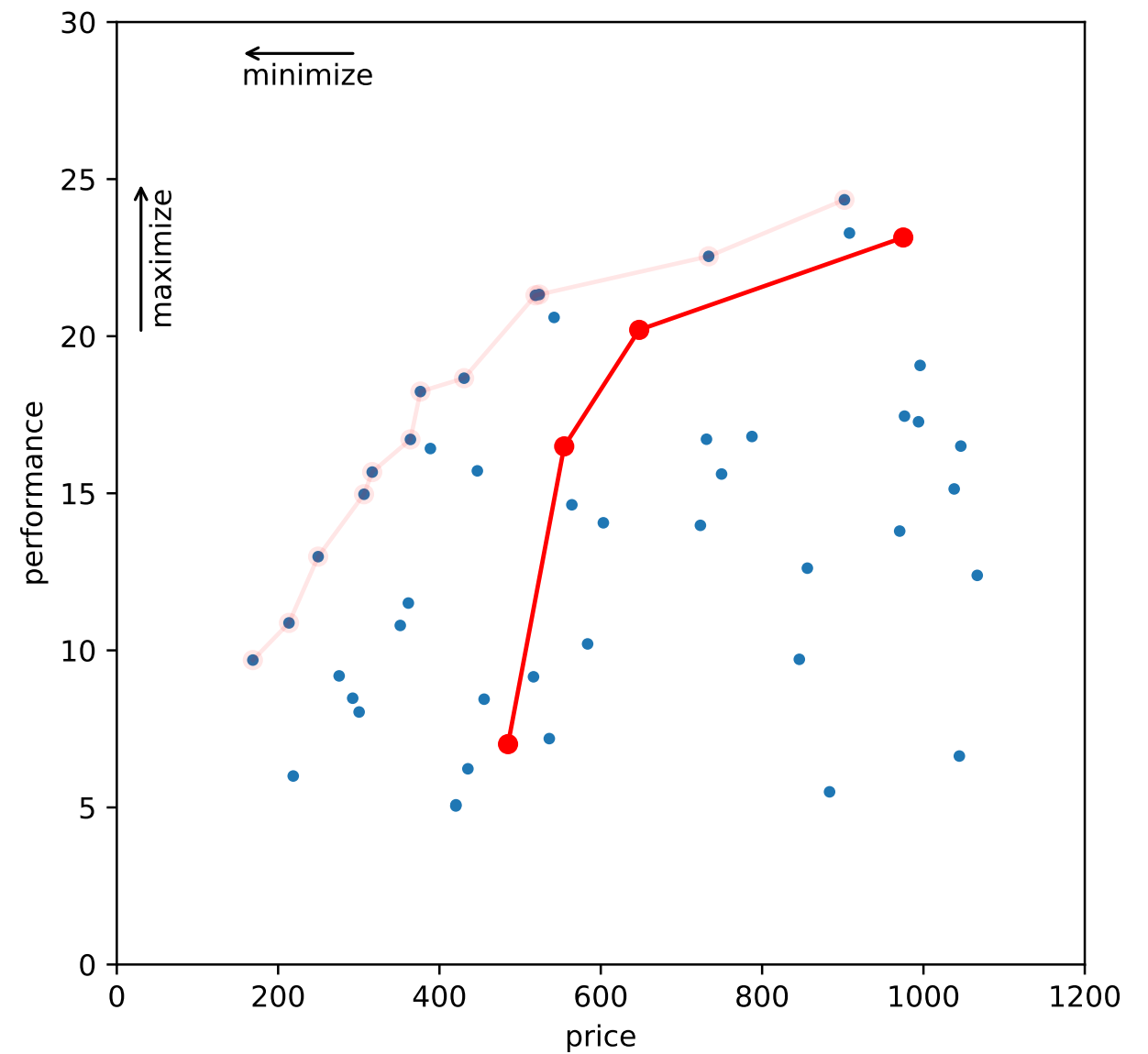
Decision vs Optimisation



Optimisation: convergence & diversity

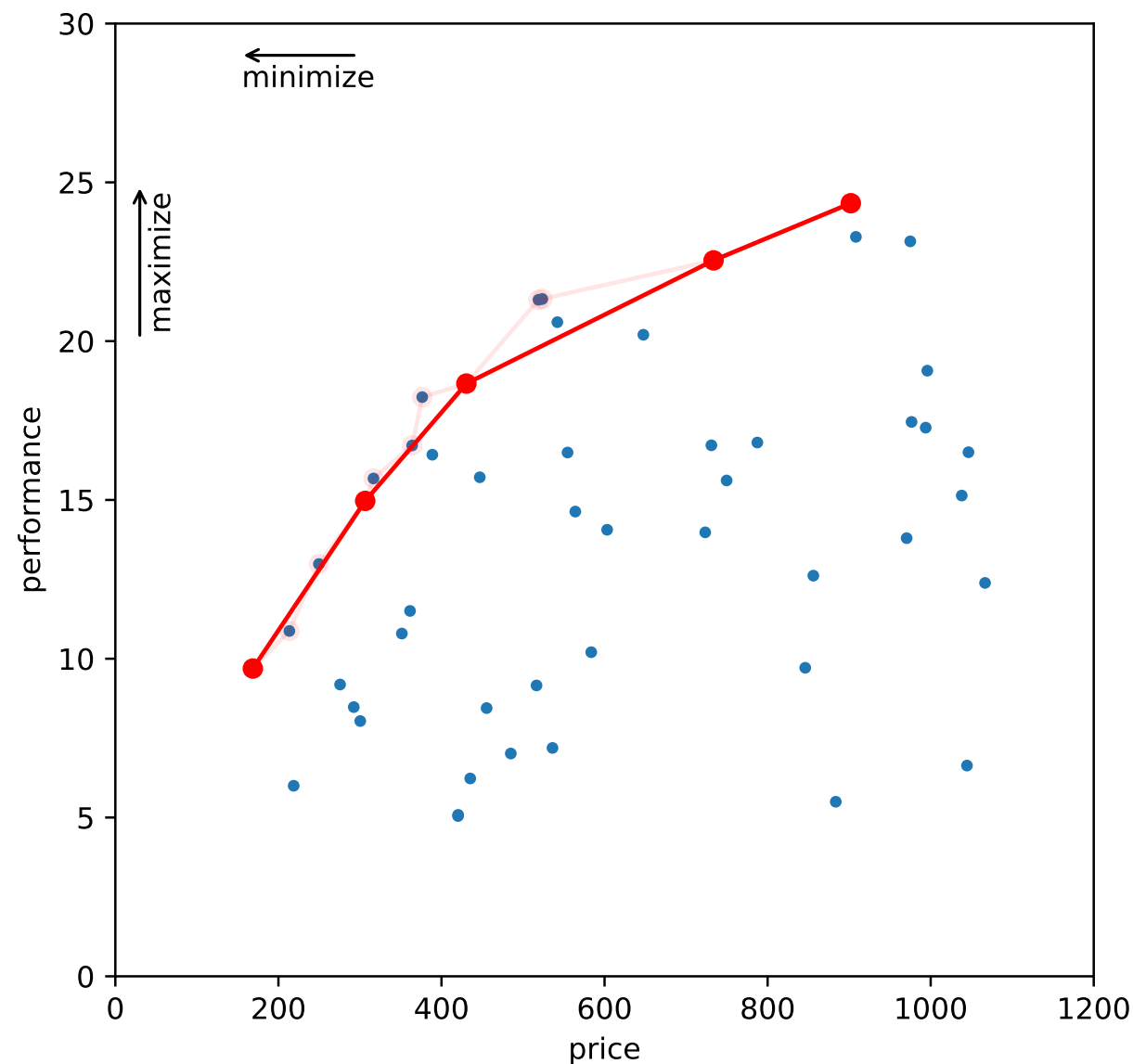


Poor diversity



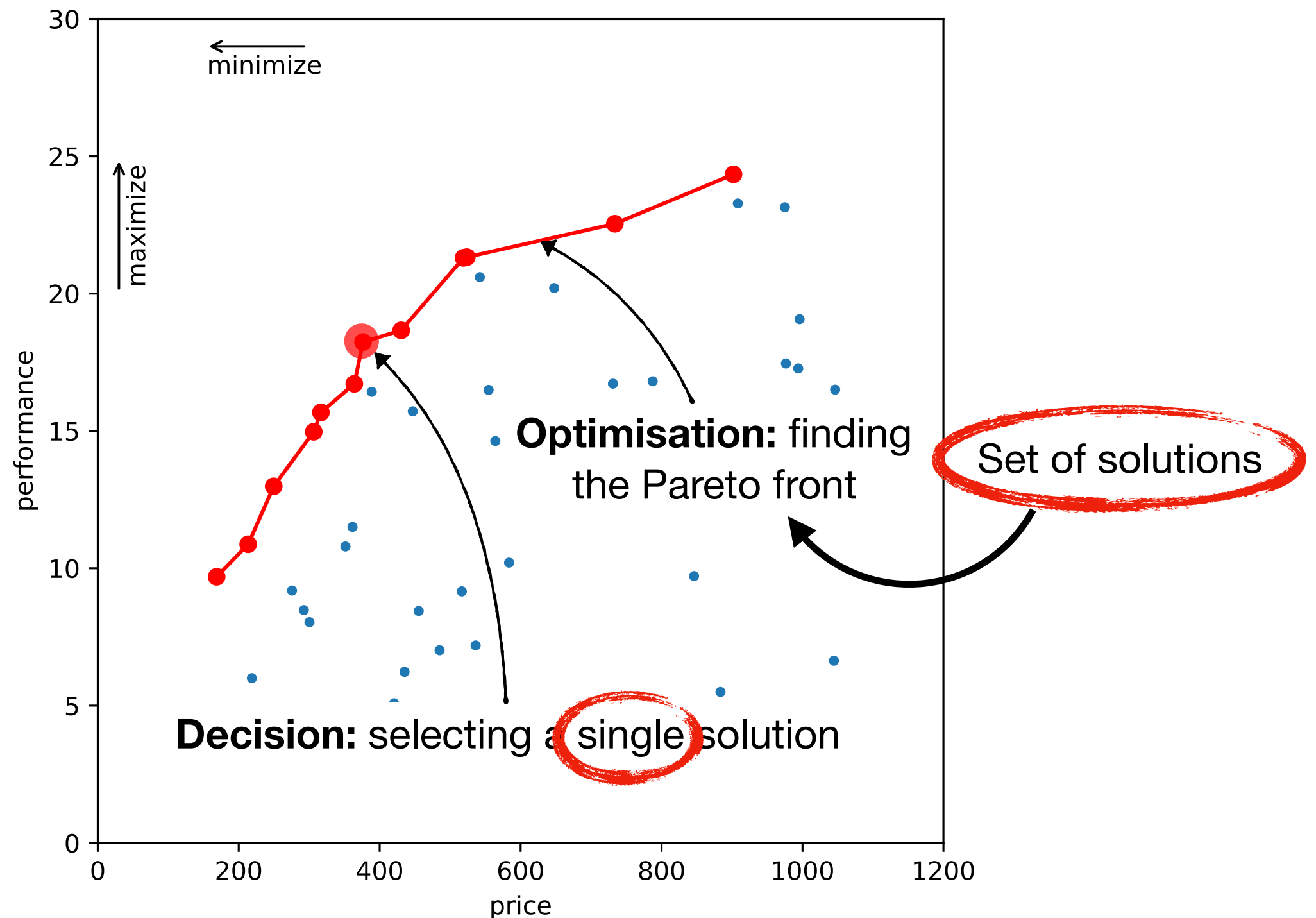
Poor convergence

Optimisation: convergence & diversity



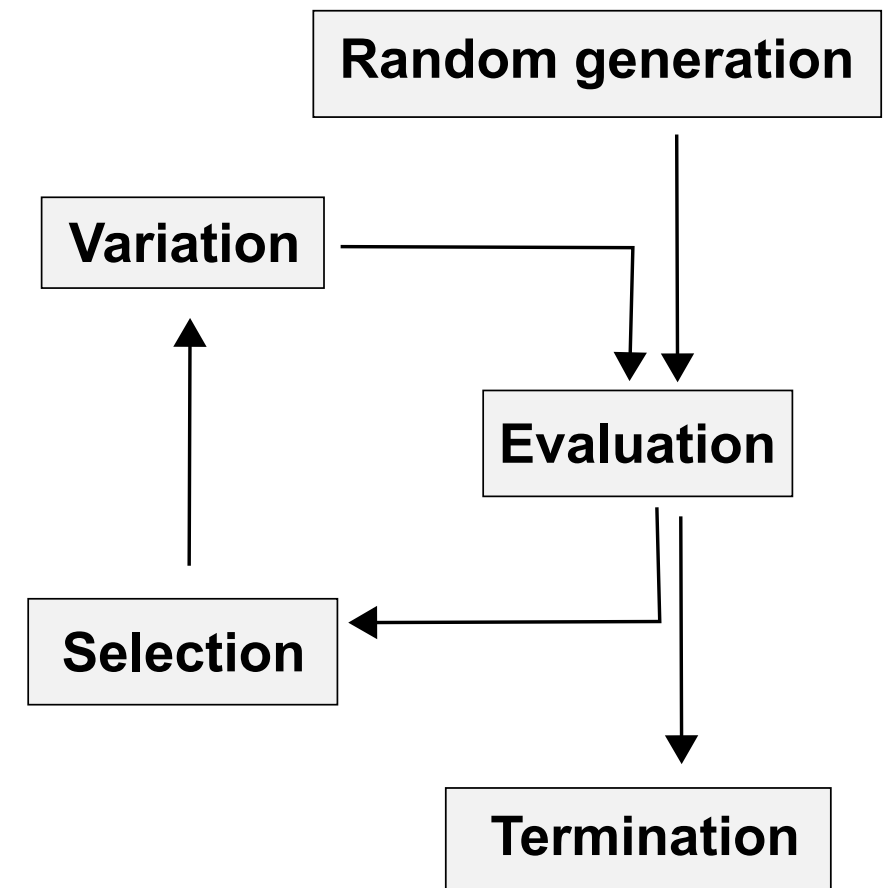
Good balance between convergence and diversity

Decision vs Optimisation

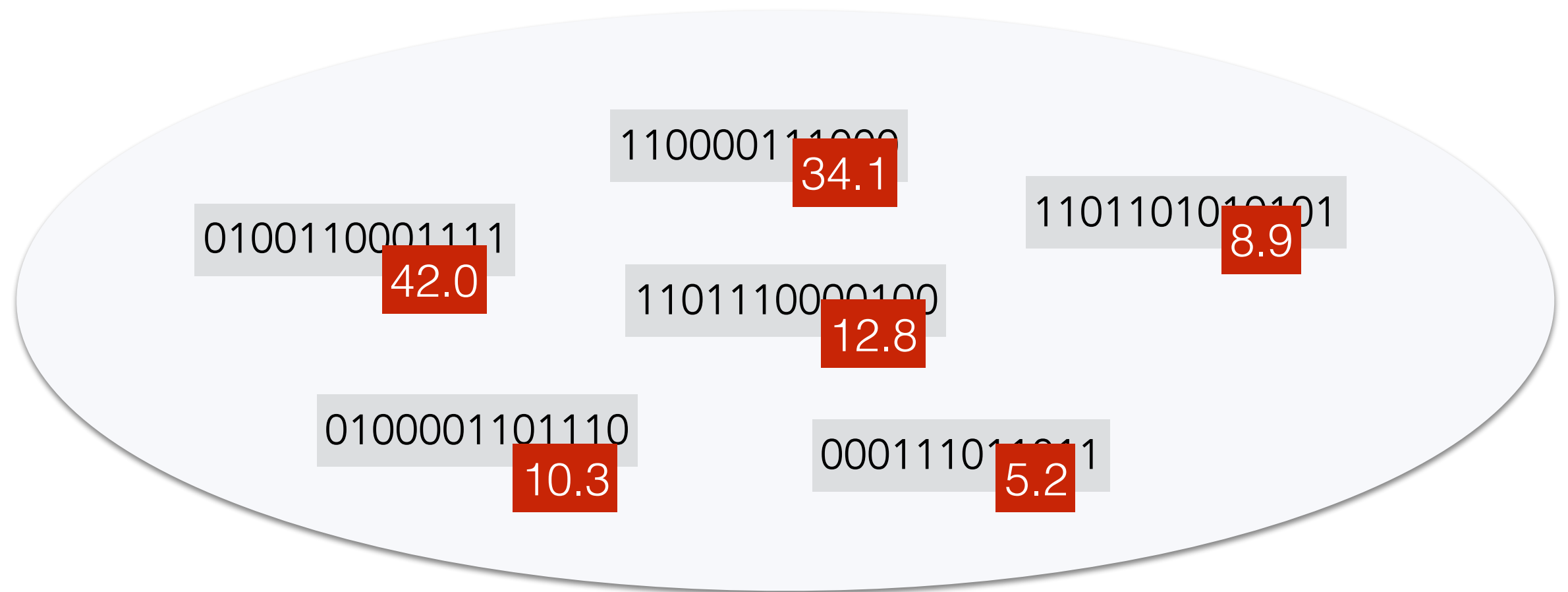


How to find optimal solutions ?

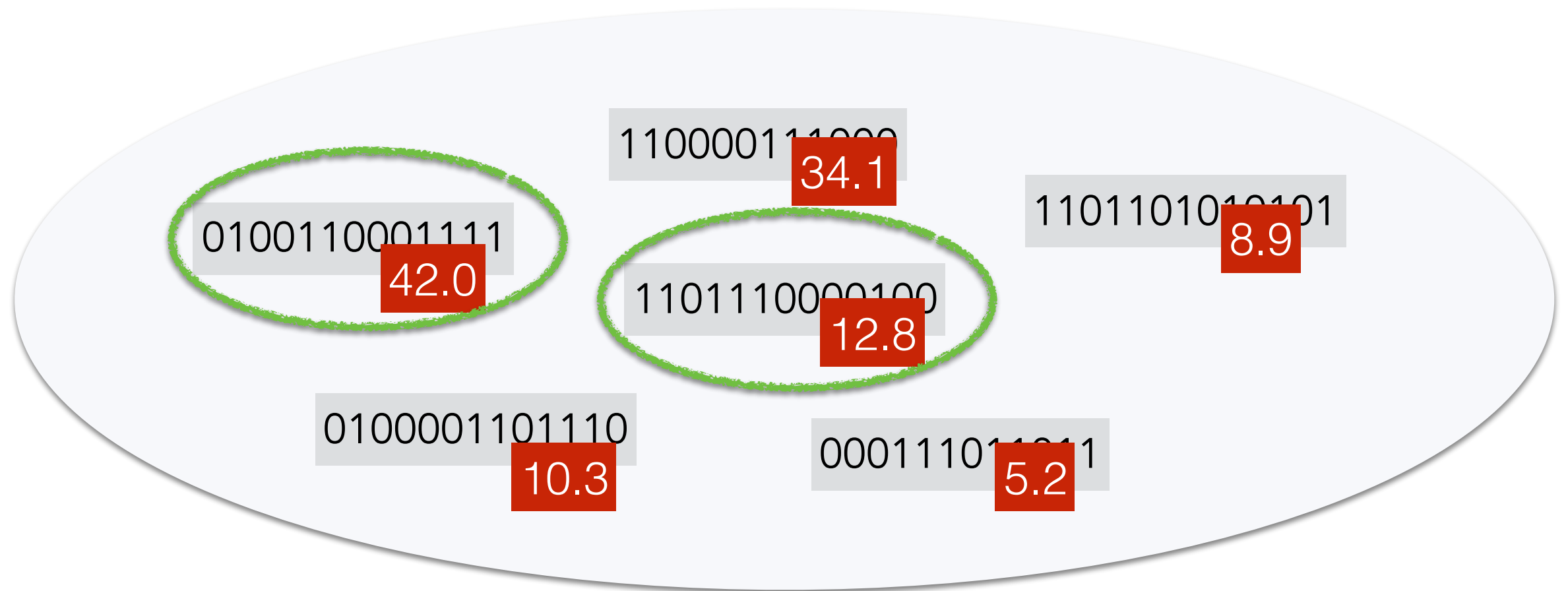
- Evolutionary Algorithms:
 - Population-based
 - Black-box optimization
 - Robust to noise and local optima
 - Versatile w.r.t. what is optimized



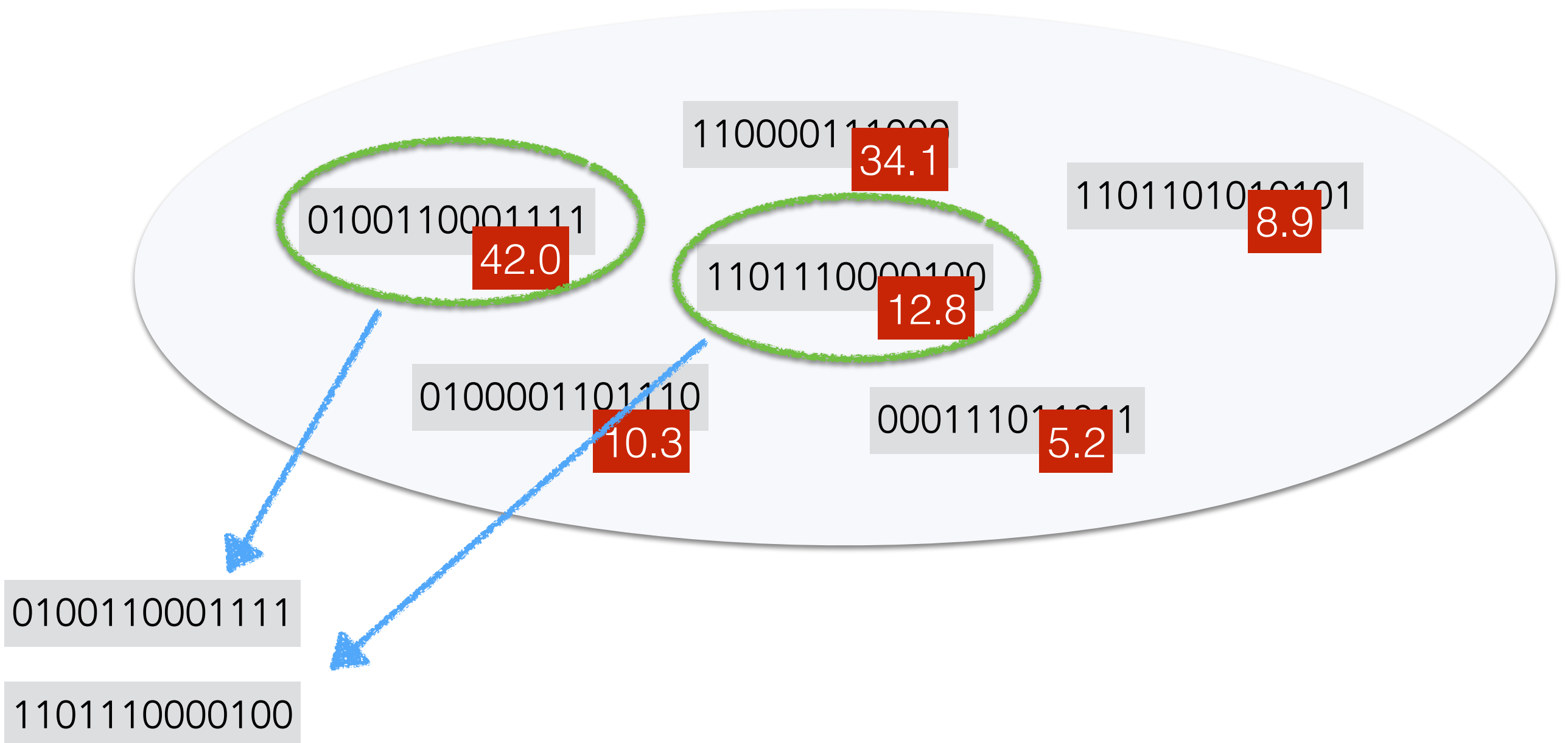
Components of an Evolutionary Algorithm: Population



Components of an Evolutionary Algorithm: Parent selection mech.



Components of an Evolutionary Algorithm: Parent selection mech.



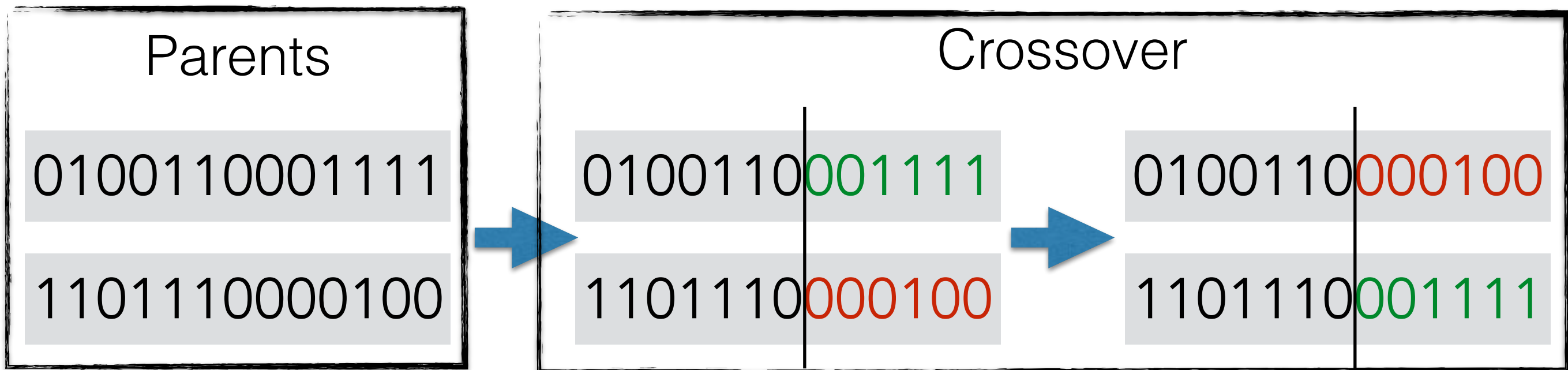
Components of an Evolutionary Algorithm: Variation operators

Parents

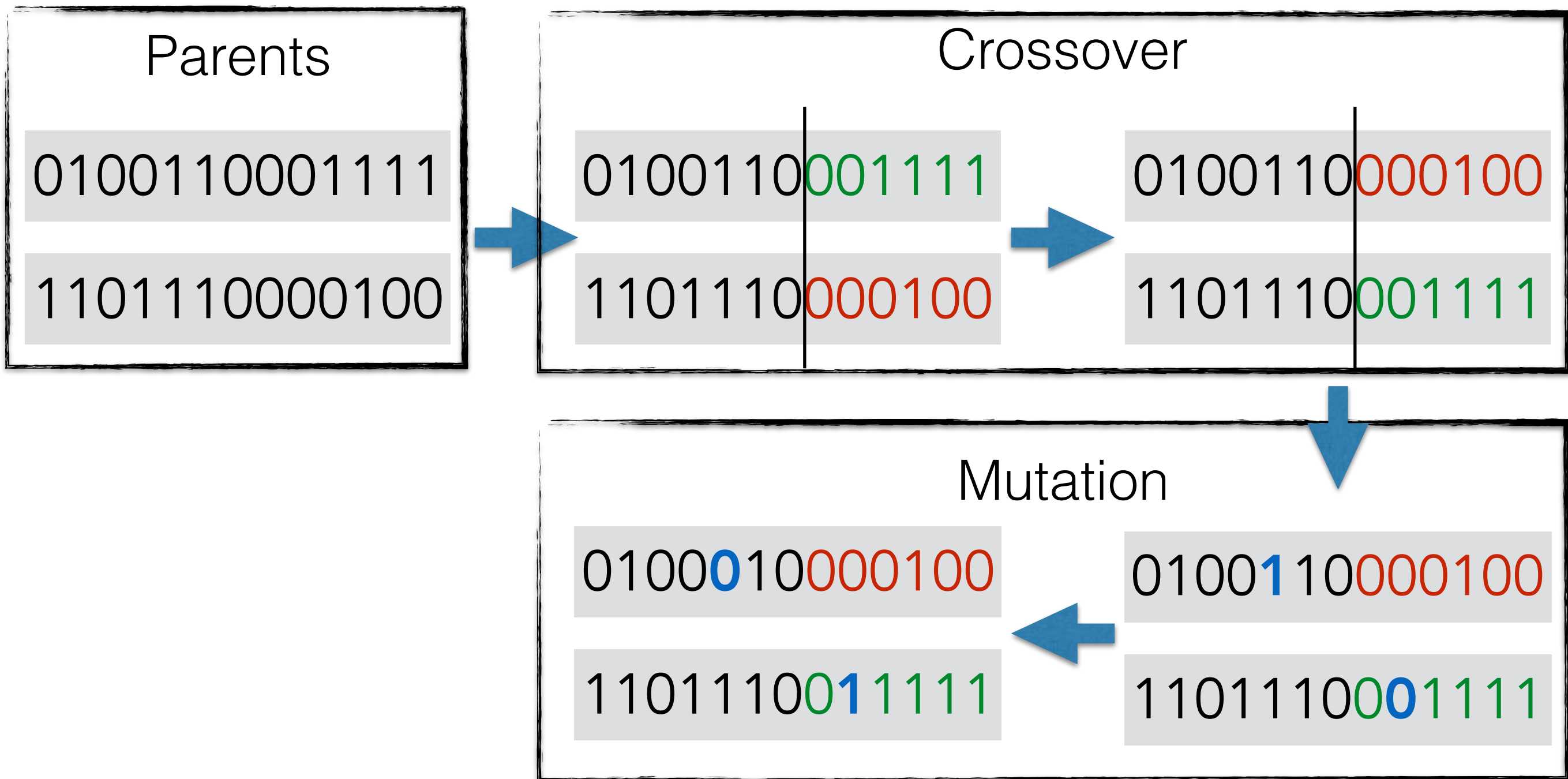
0100110001111

1101110000100

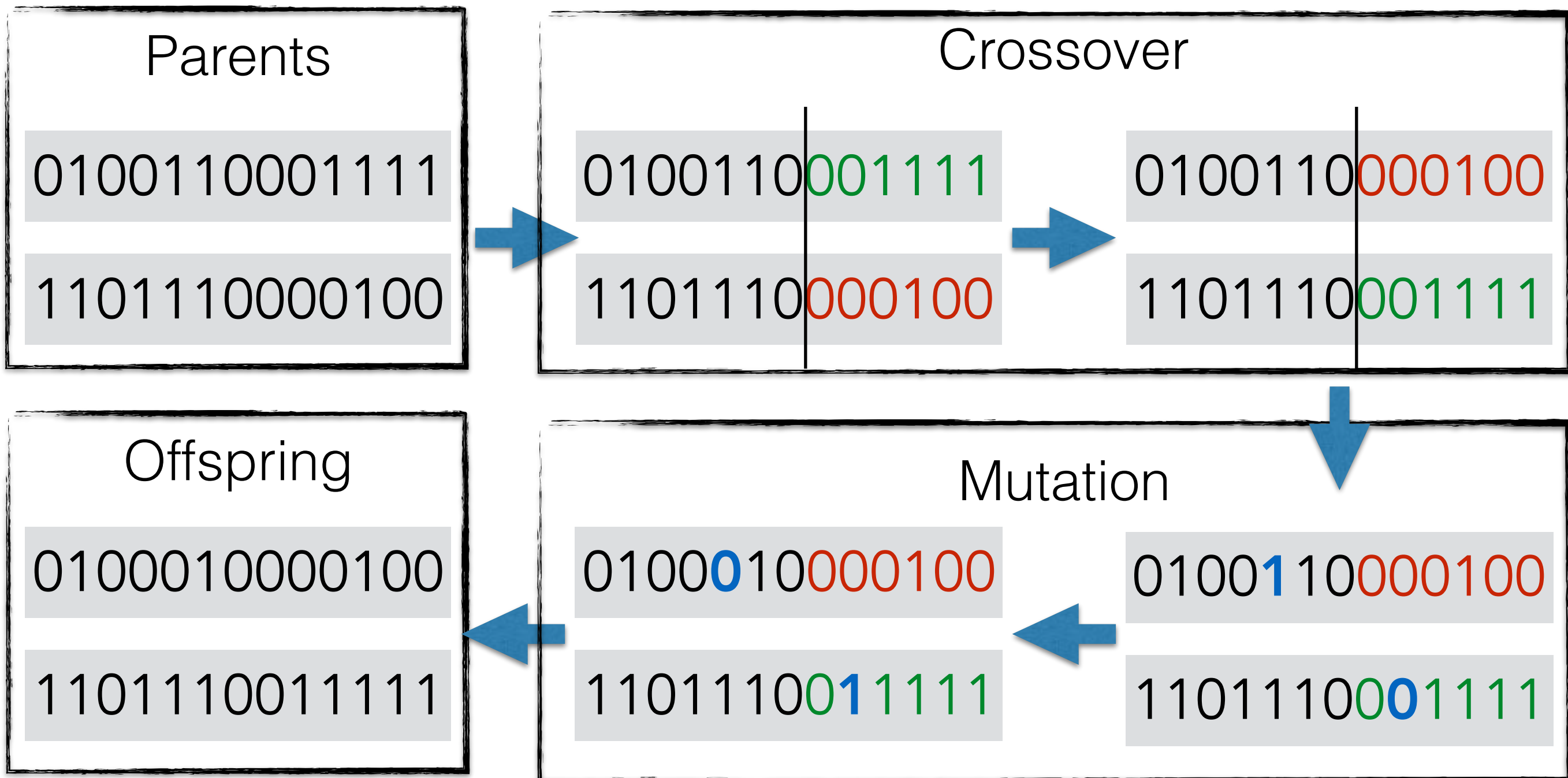
Components of an Evolutionary Algorithm: Variation operators



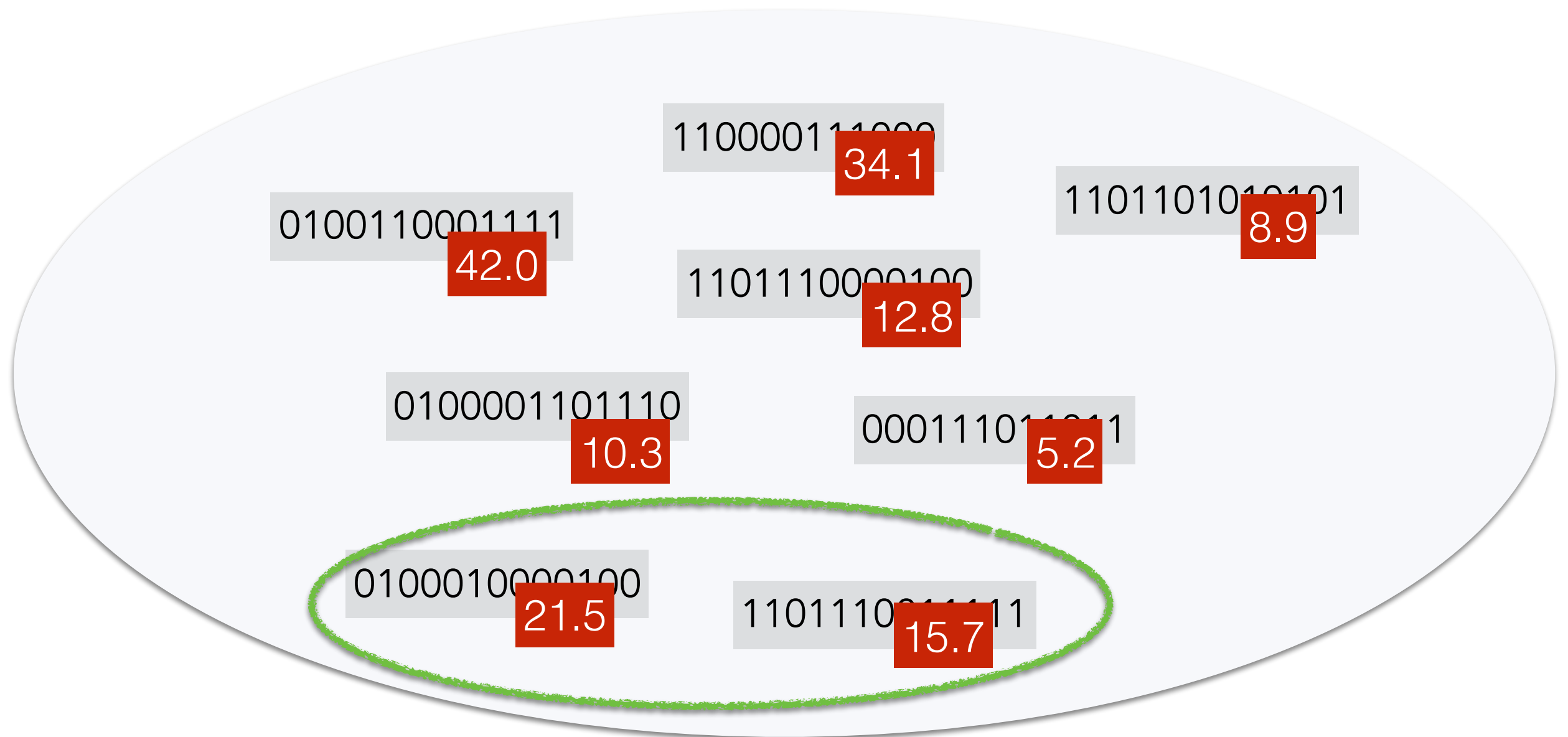
Components of an Evolutionary Algorithm: Variation operators



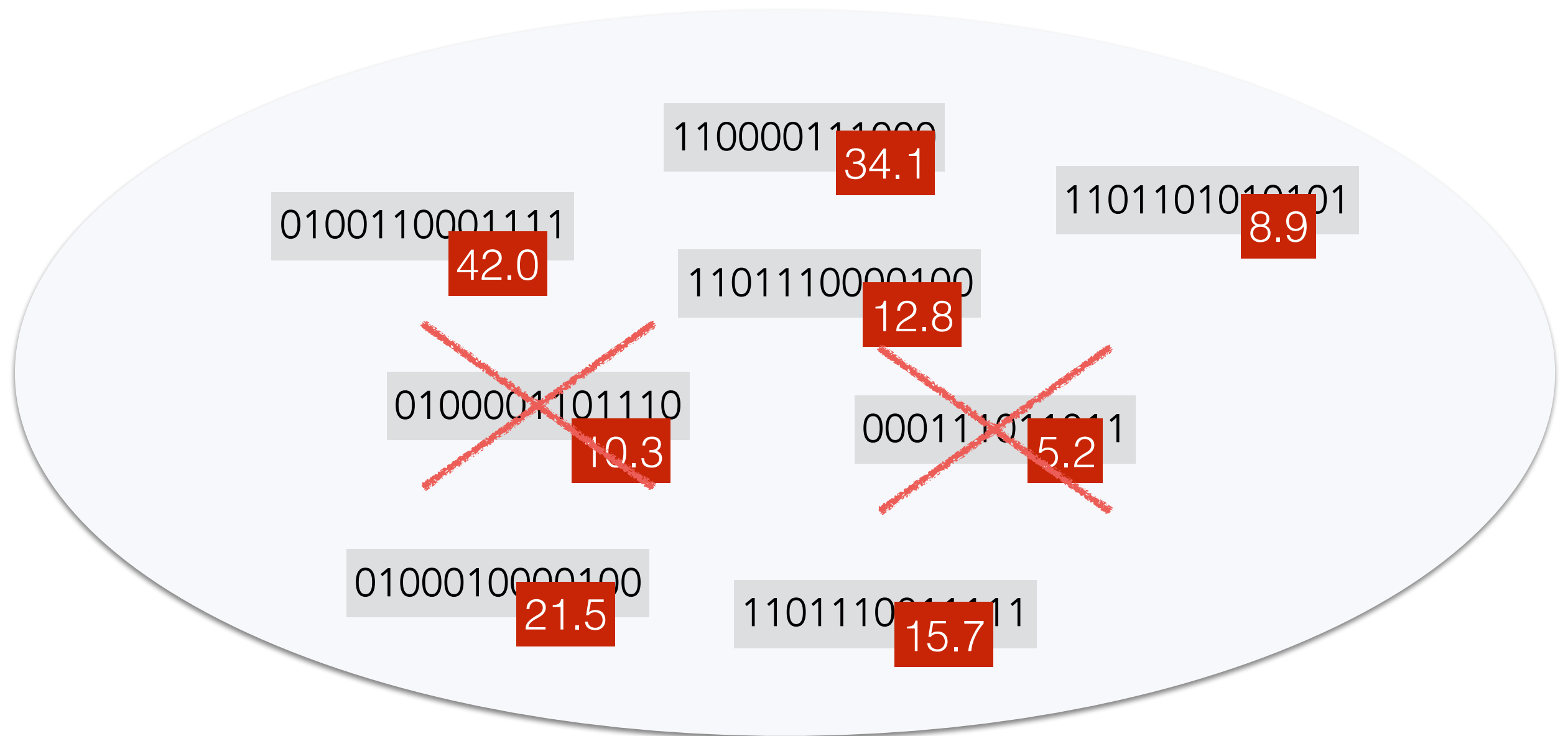
Components of an Evolutionary Algorithm: Variation operators



Components of an Evolutionary Algorithm: Variation operators



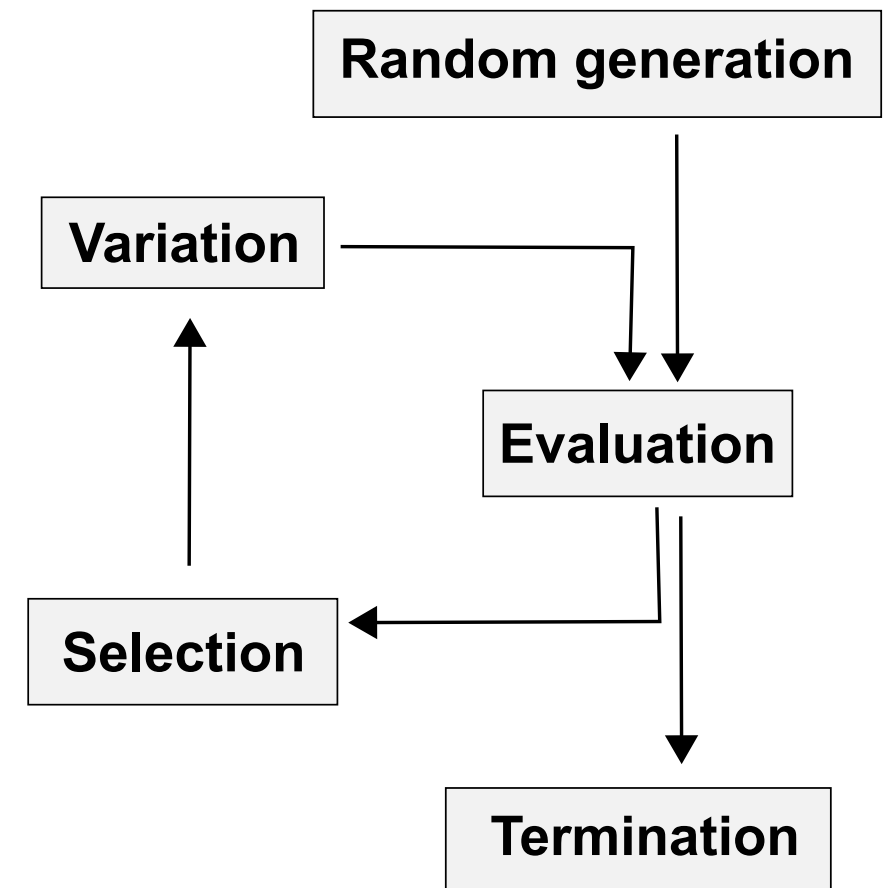
Components of an Evolutionary Algorithm: Survivor selection mech.



Evolutionary algorithms

Main components:

- individual
- population
- evaluation function (or fitness function)
- parent selection mechanism
- variation operators, recombination and mutation
- survivor selection mechanism



Interlude...

Introduction to DEAP library

0_EA_basics.ipynb

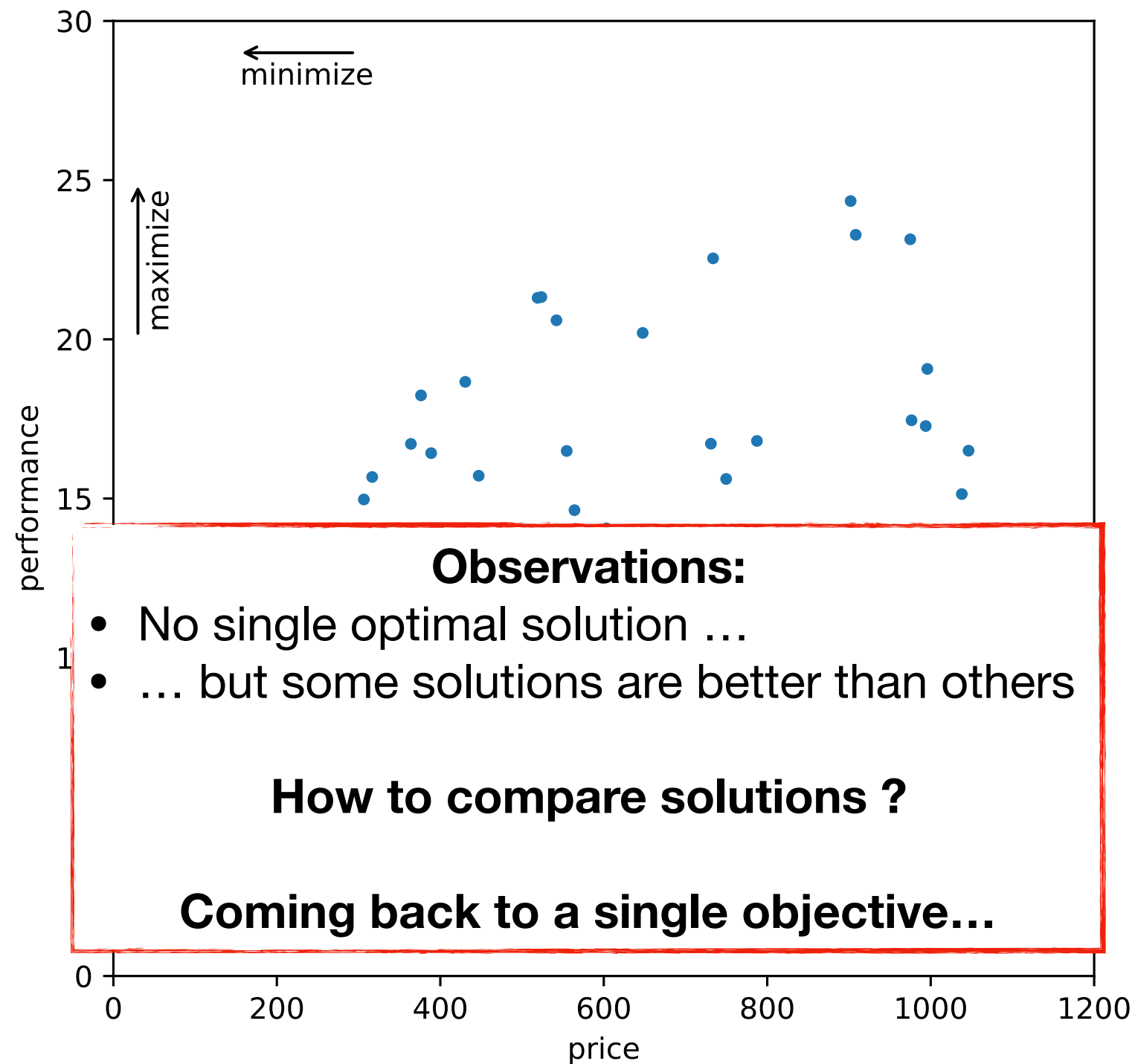
1_EA_stats.ipynb

Overview

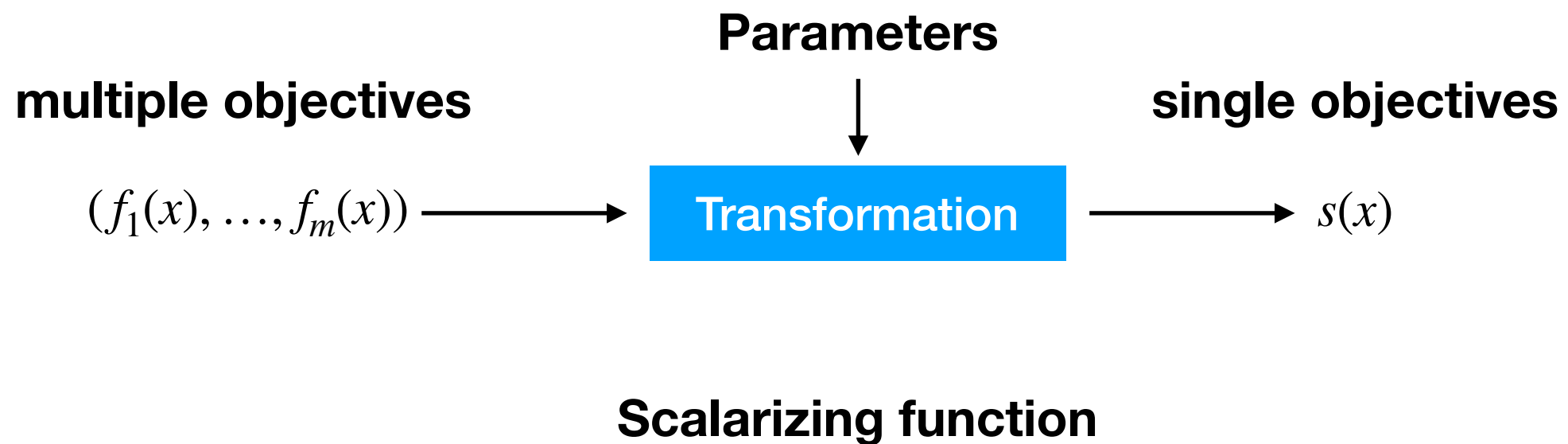


- Introduction
- Back to single-objective
- Multi-objective optimization
- Many-objective optimization

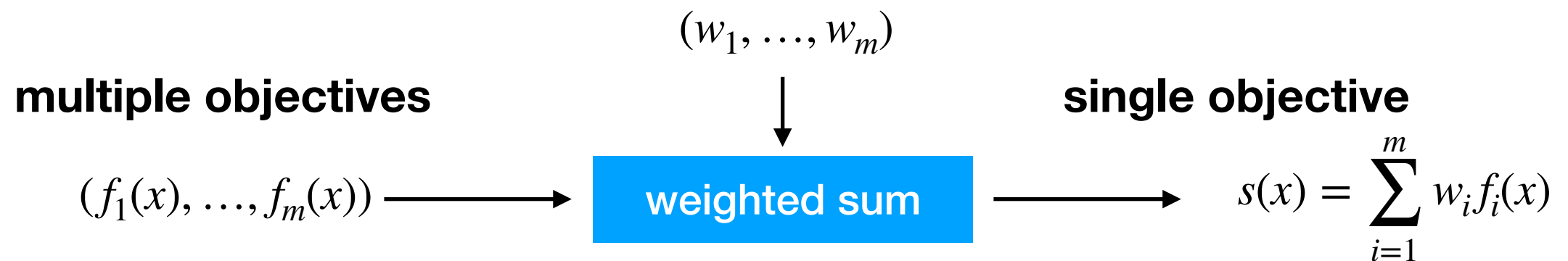
Multi-Objective decision



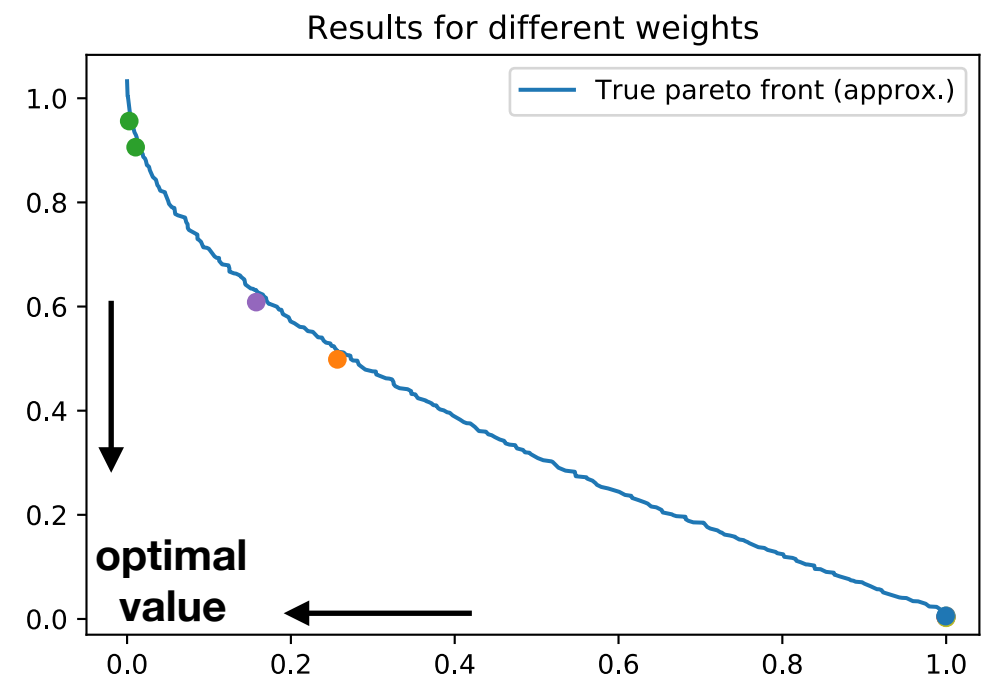
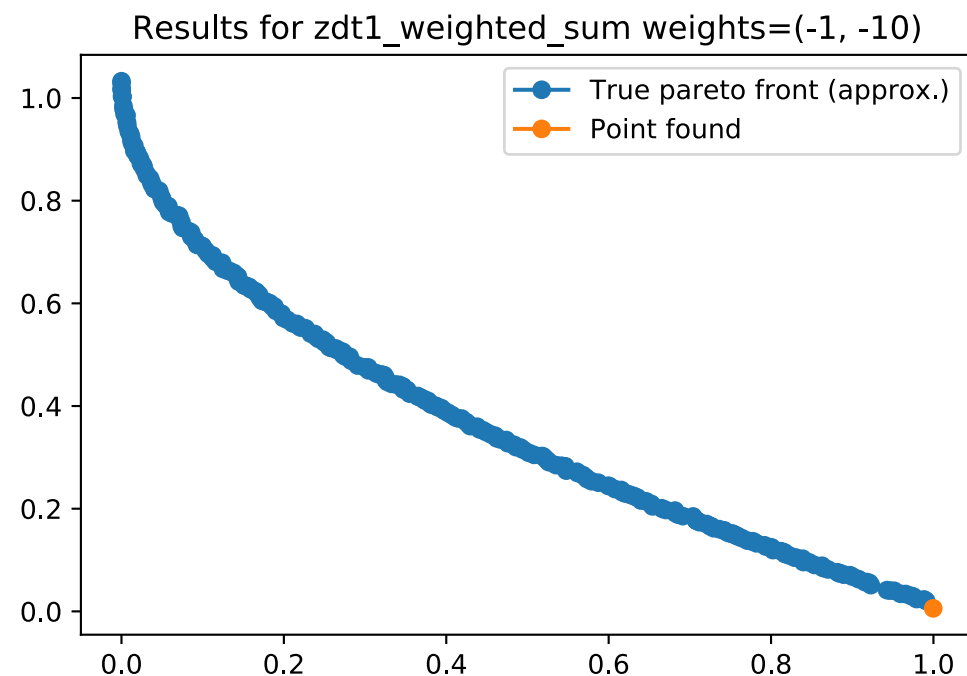
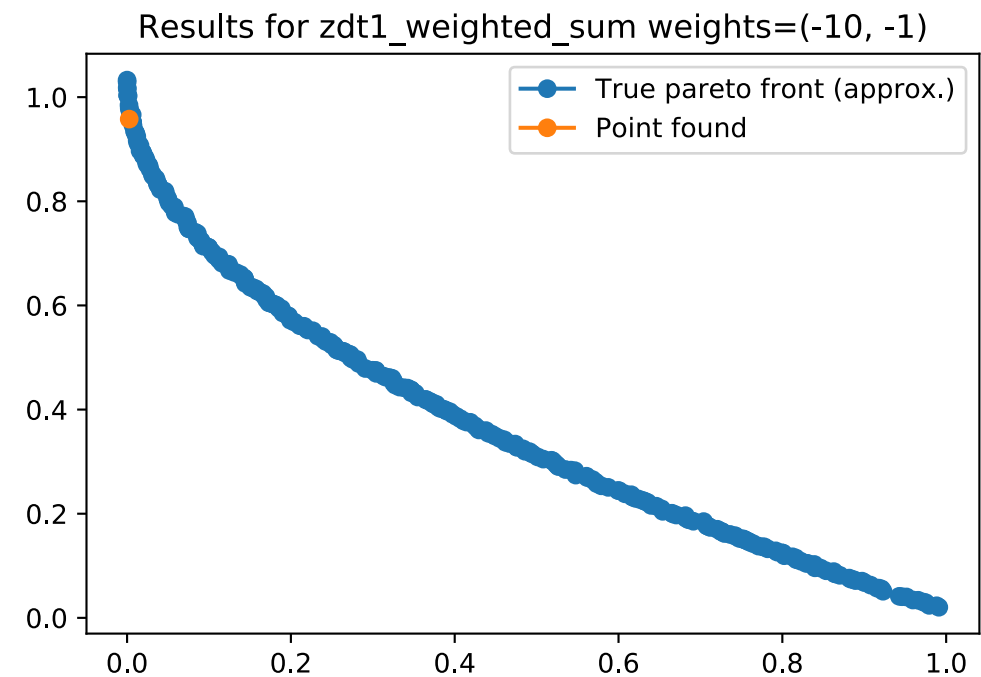
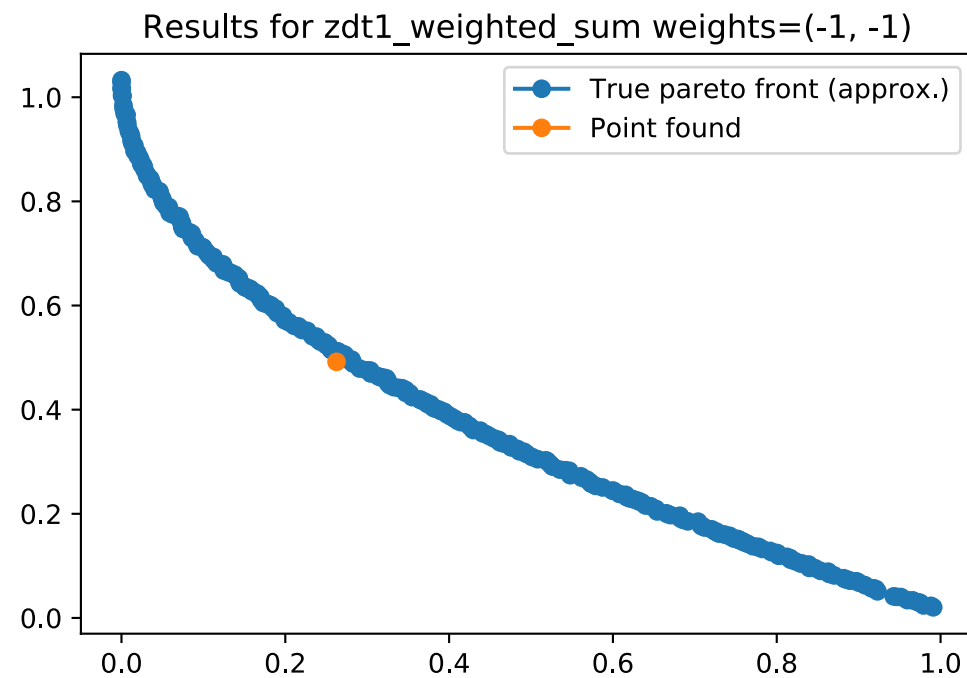
Back to a single objective (decision)



Back to a single objective: weighted sum

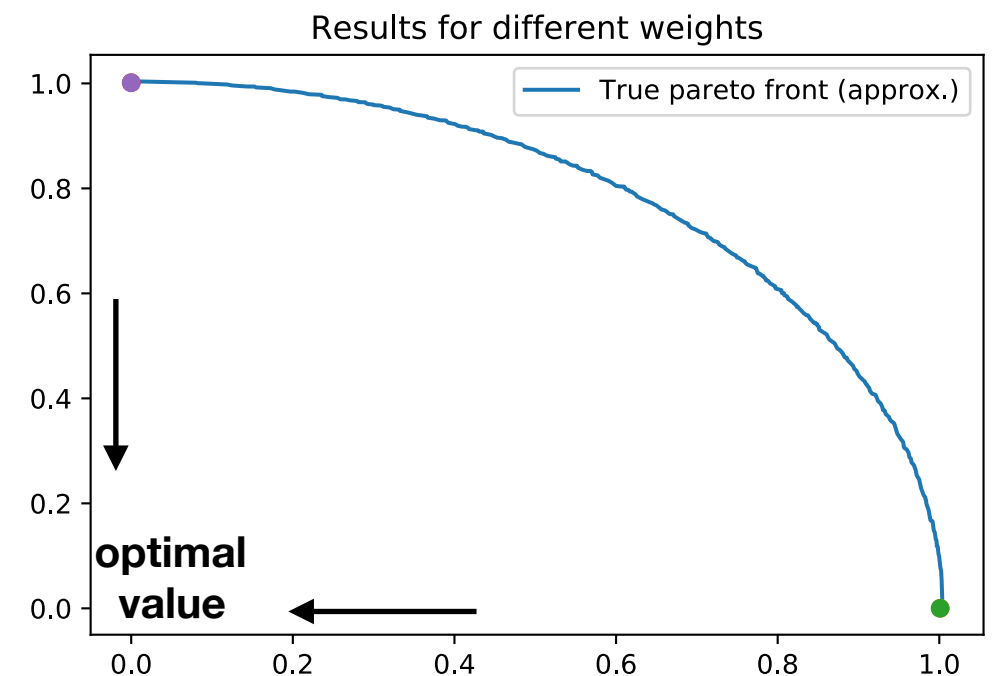
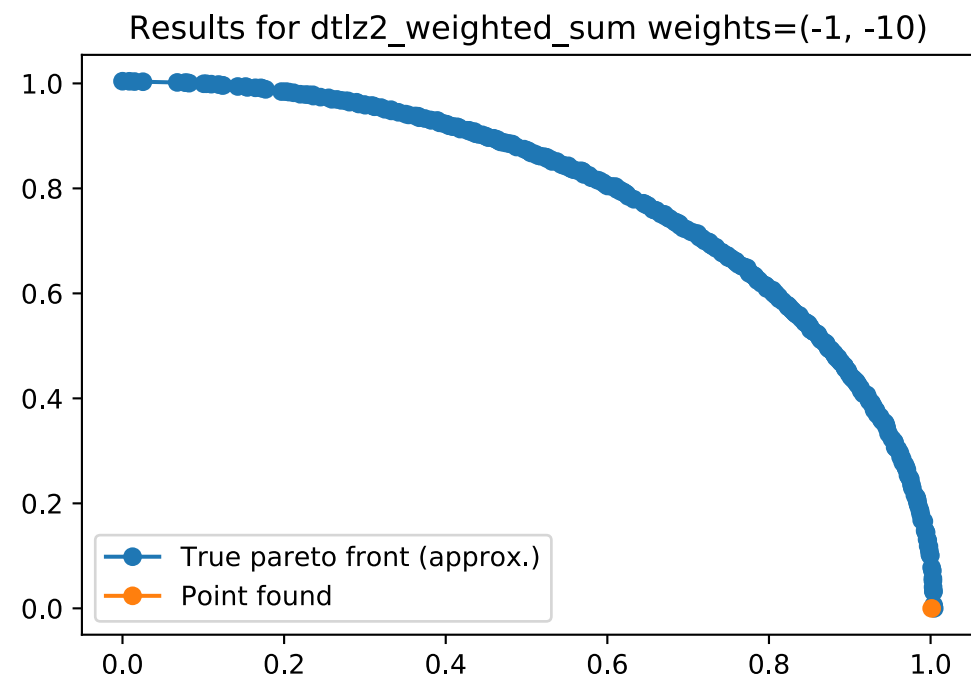
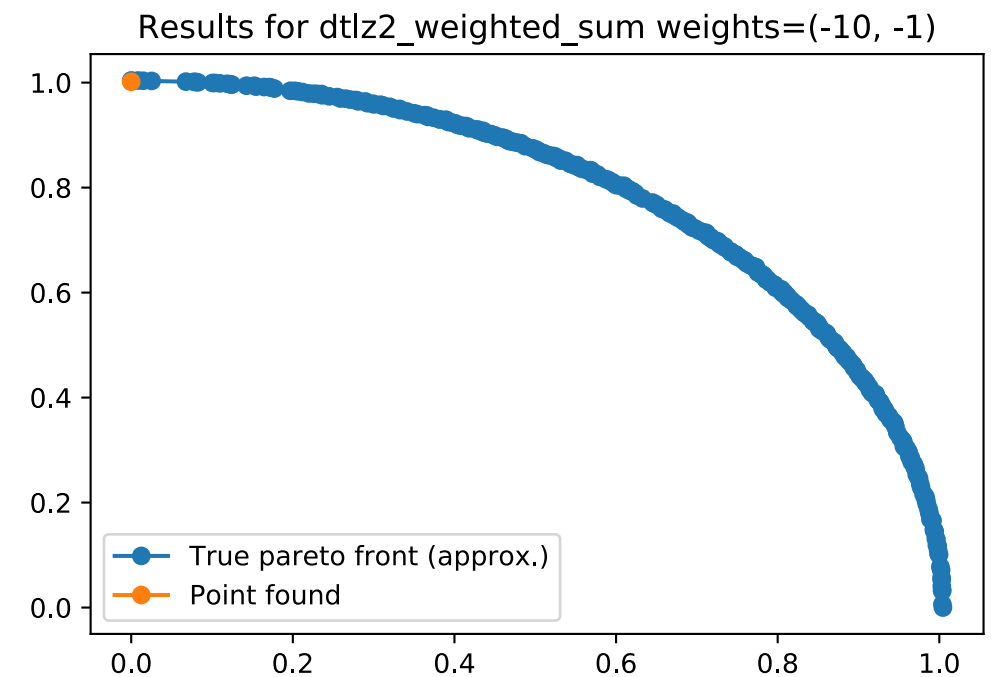
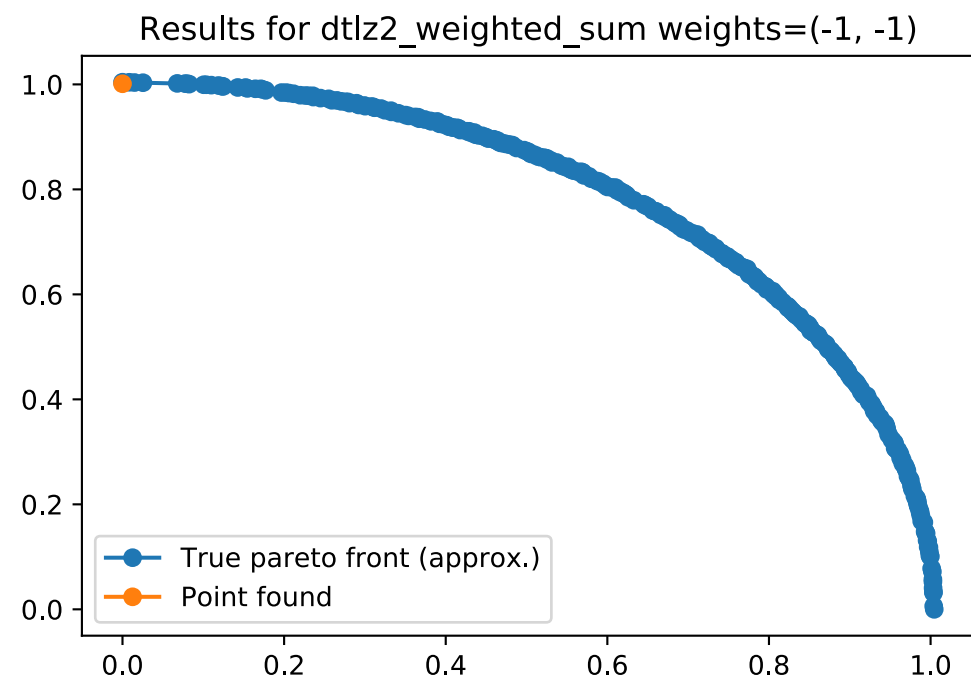


Example



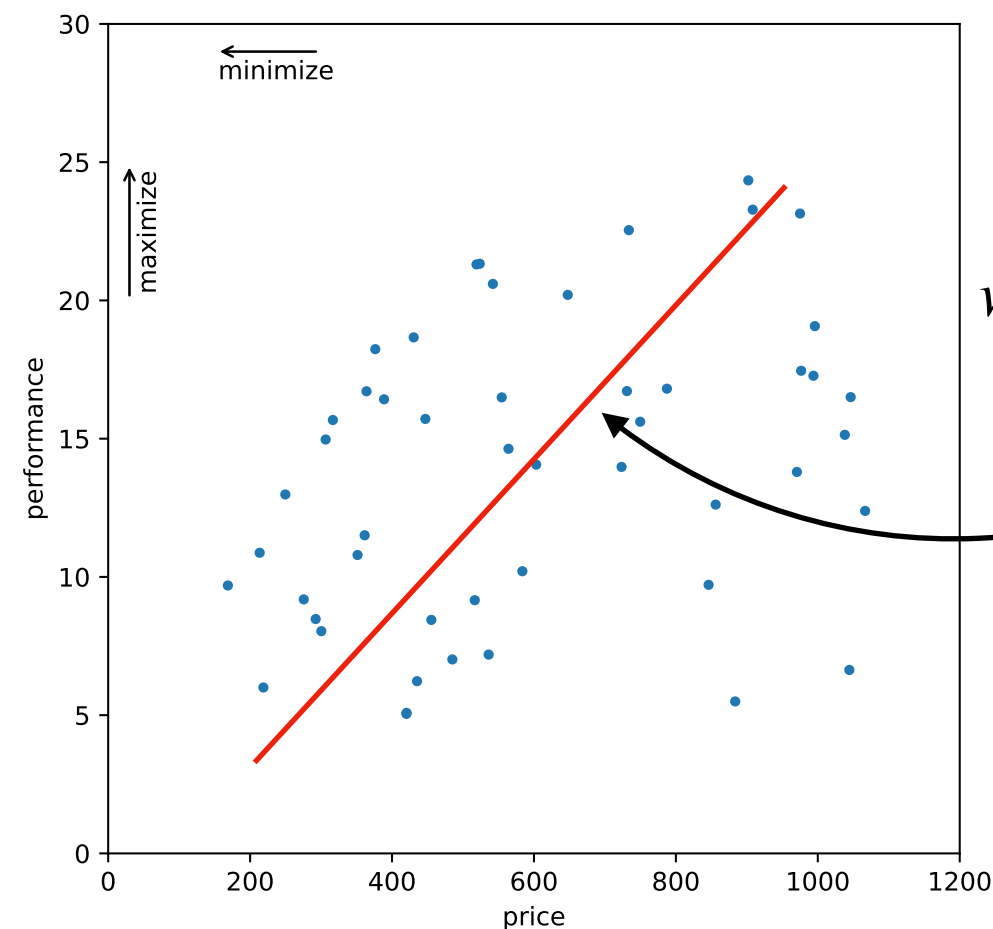
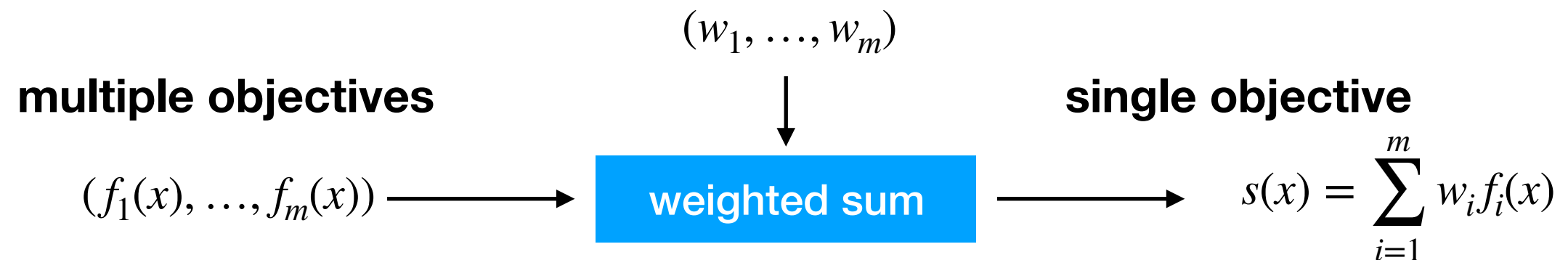
zdt1 benchmark function, see 2_EA_MO_single.ipynb

Example



dtlz2 benchmark function, see 2_EA_MO_single.ipynb

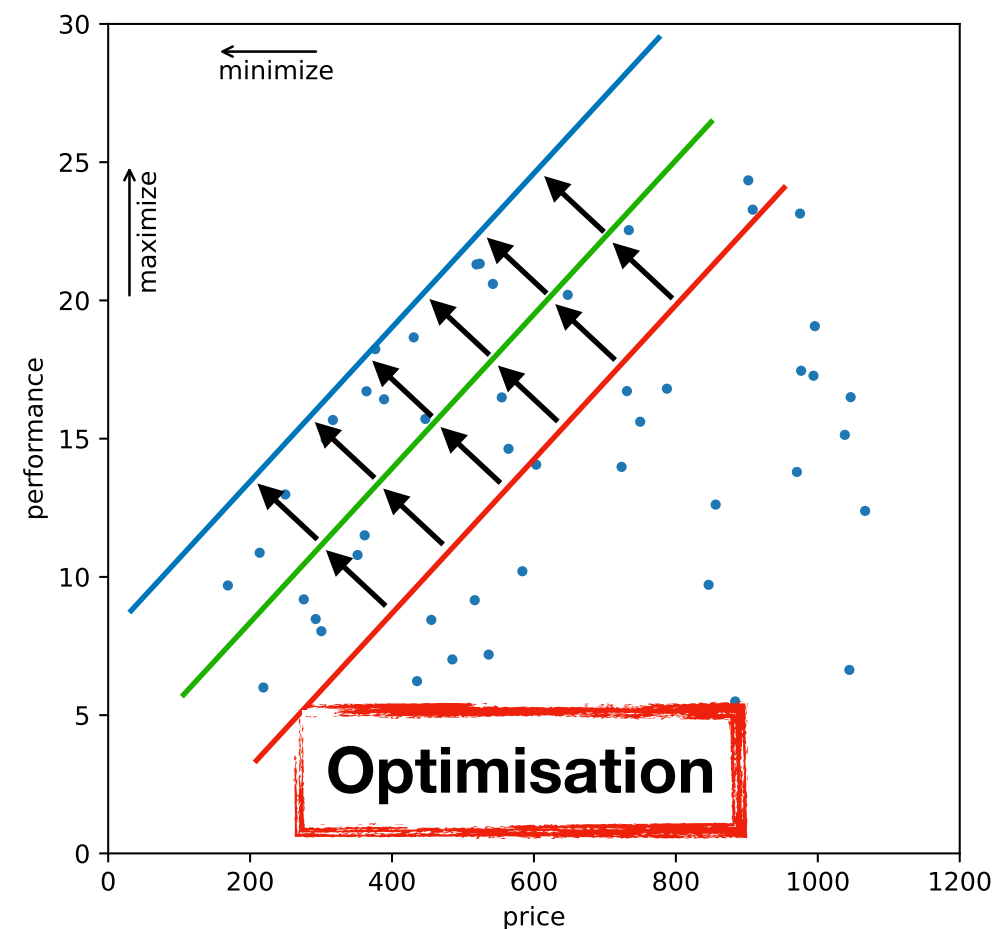
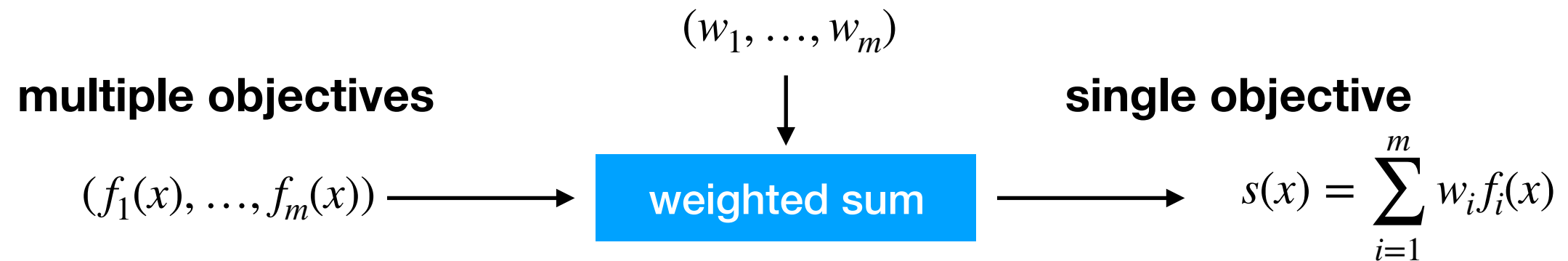
Back to a single objective: weighted sum



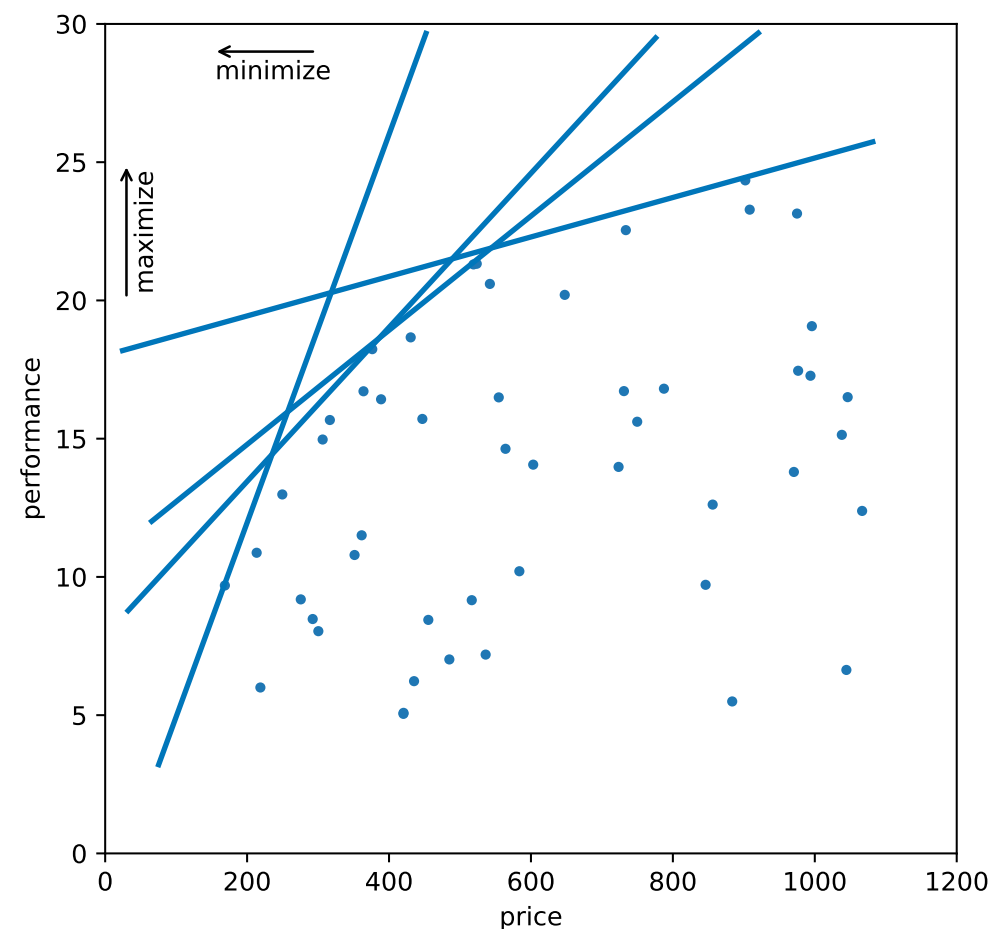
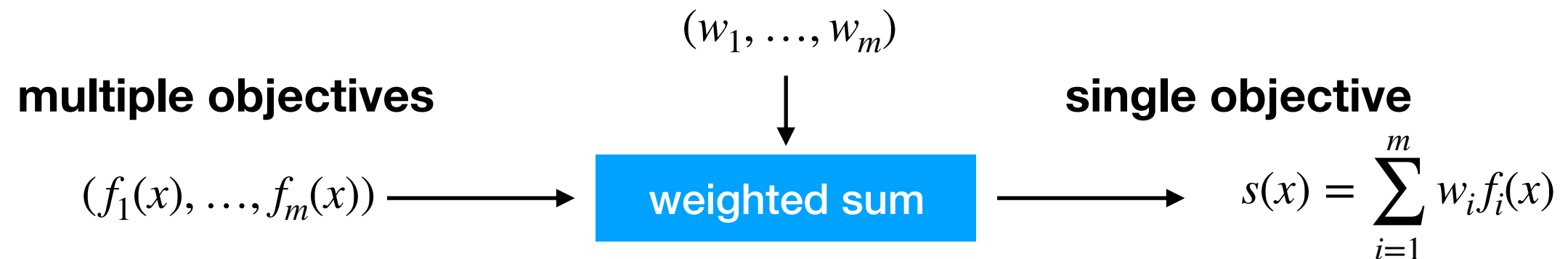
$$w_1 \text{ price}(x) + w_2 \text{ perf}(x) = \text{Cte}$$

Rq: $w_1 \cdot w_2 < 0$

Back to a single objective: weighted sum

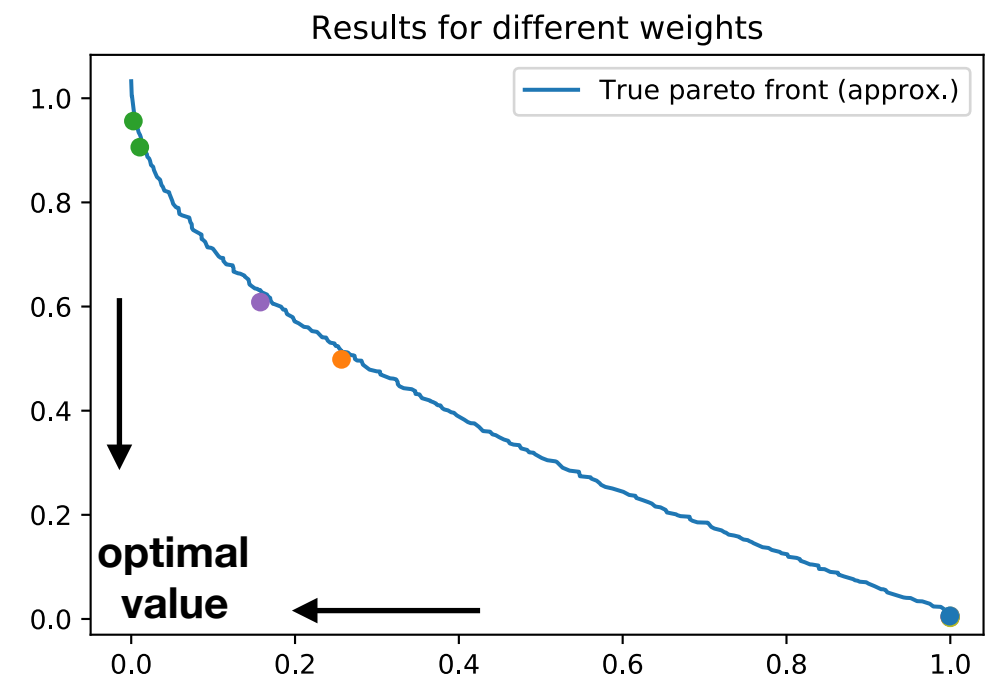
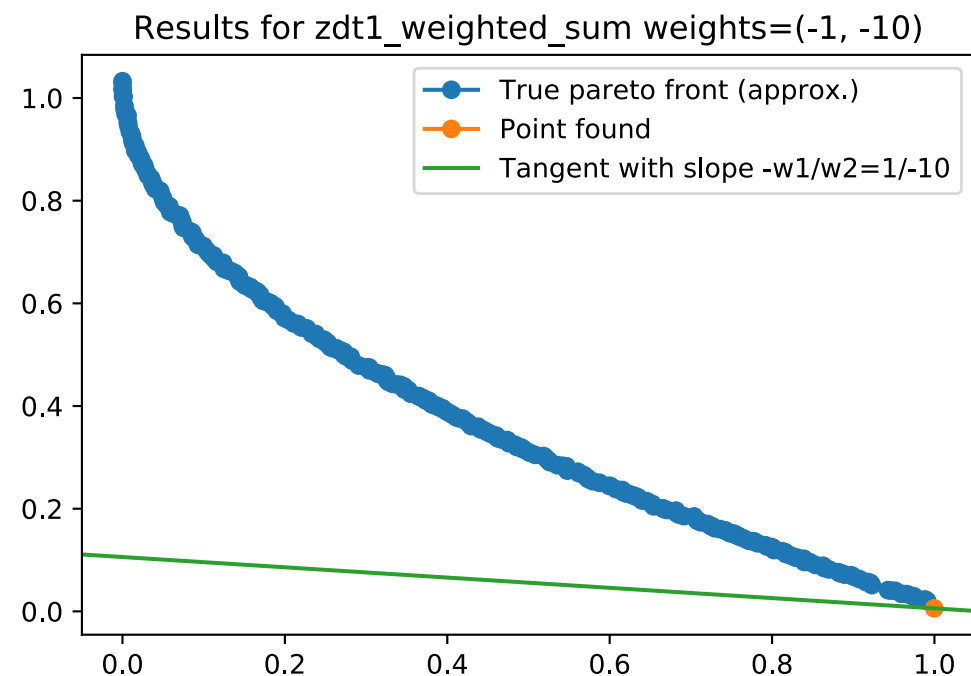
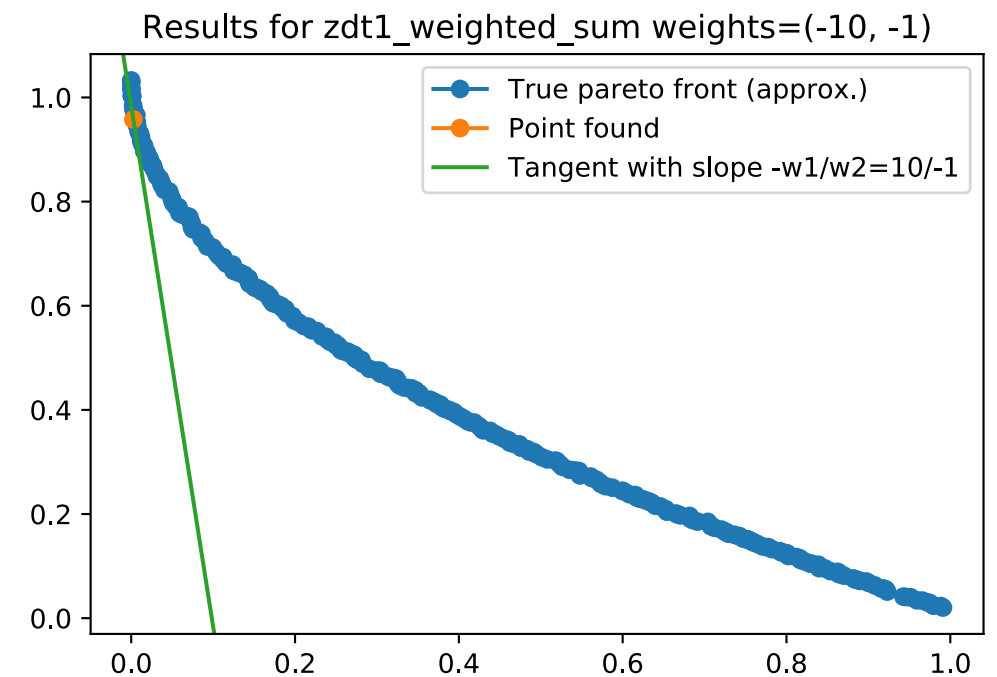
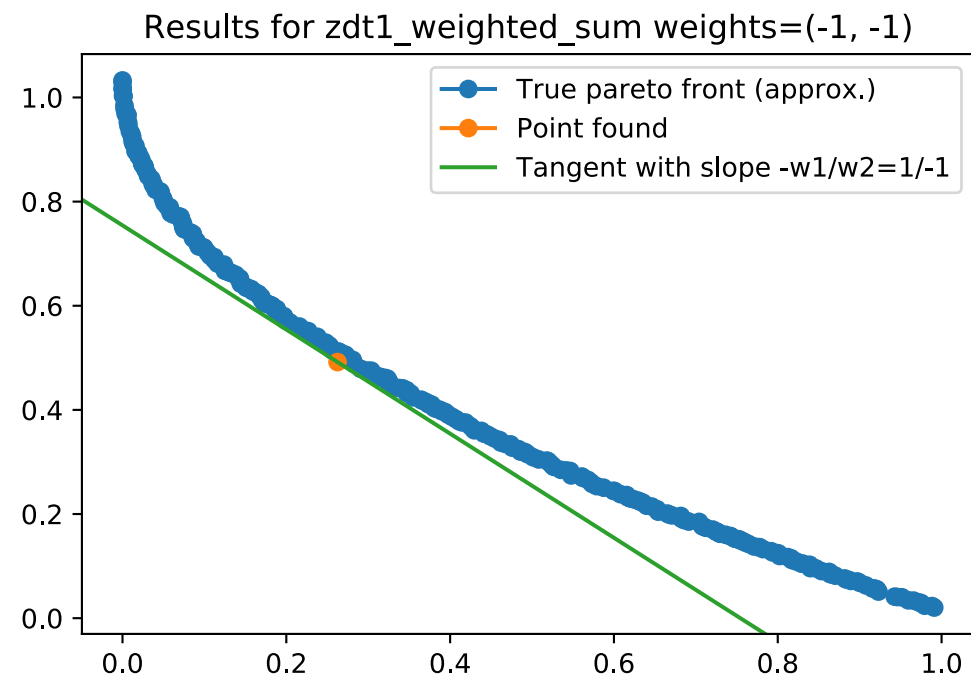


Back to a single objective: weighted sum



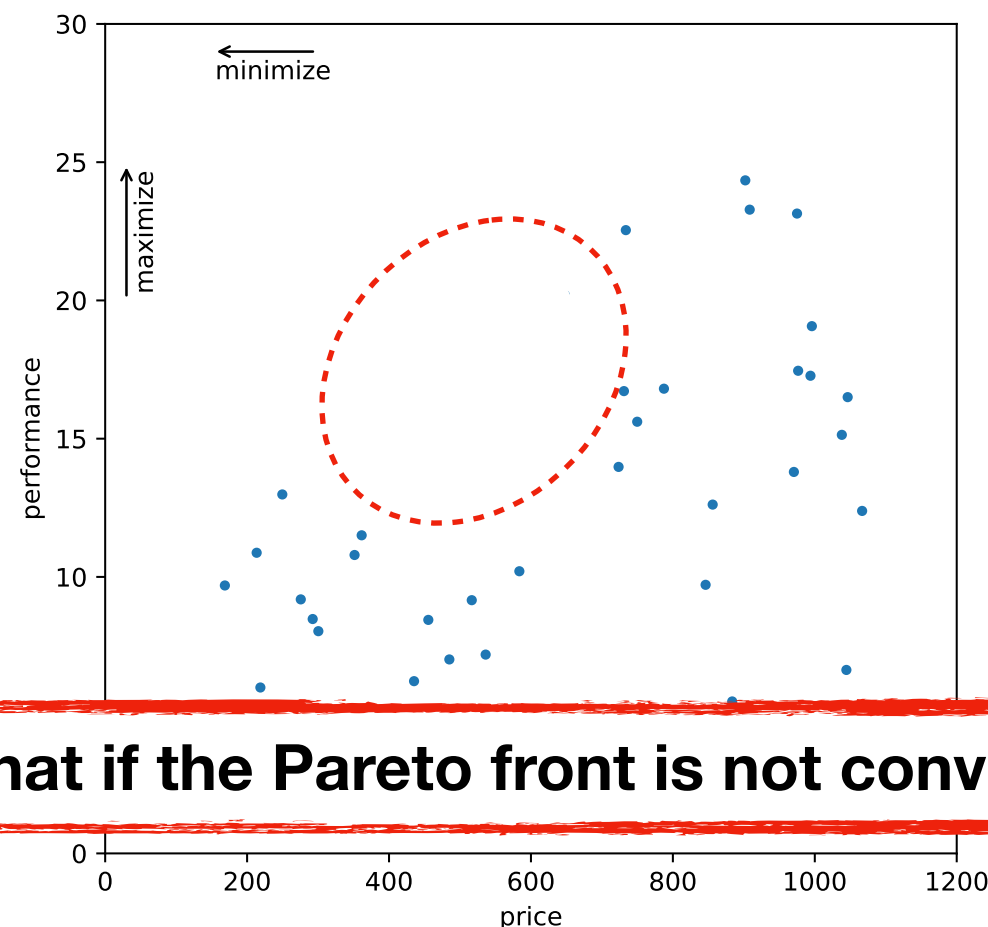
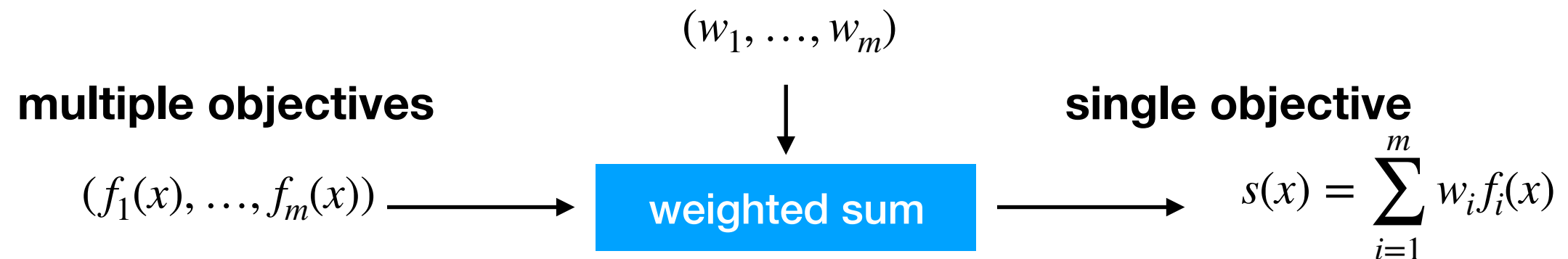
Each set of weights makes
optimisation converge to
a particular point
of the Pareto front

Example



zdt1 benchmark function, see 2_EA_MO_single.ipynb

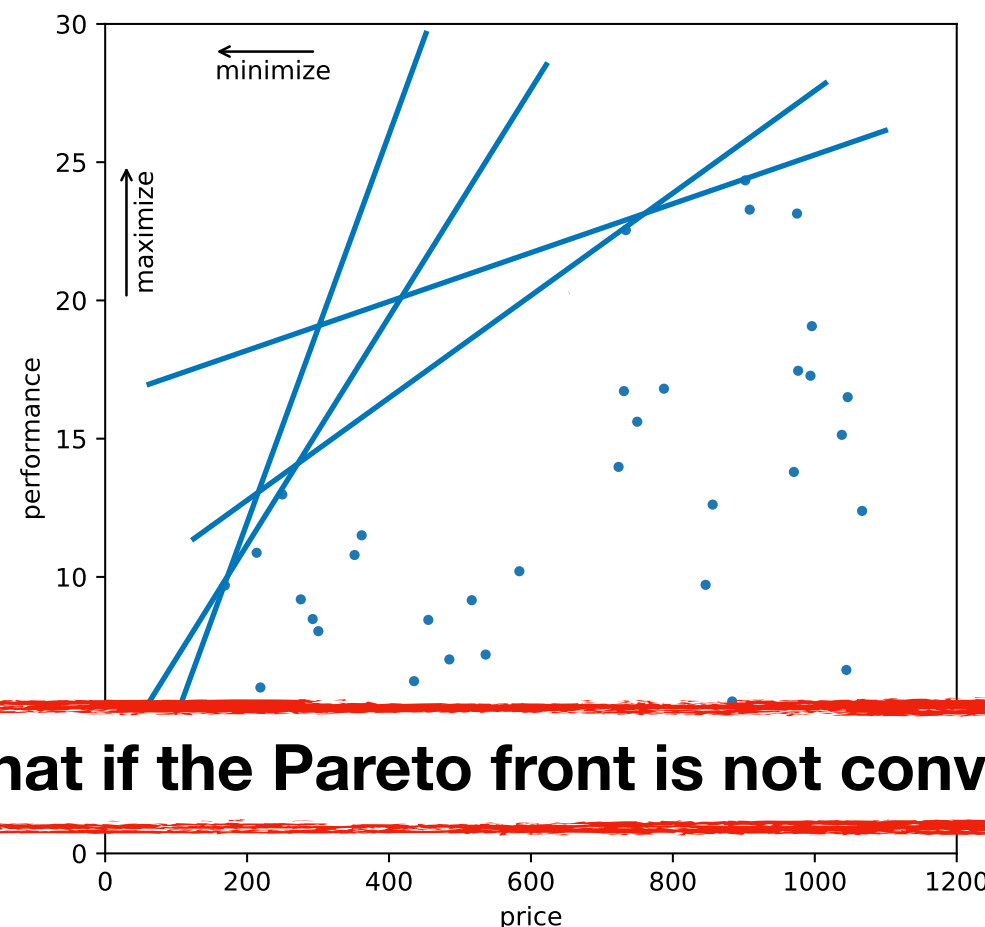
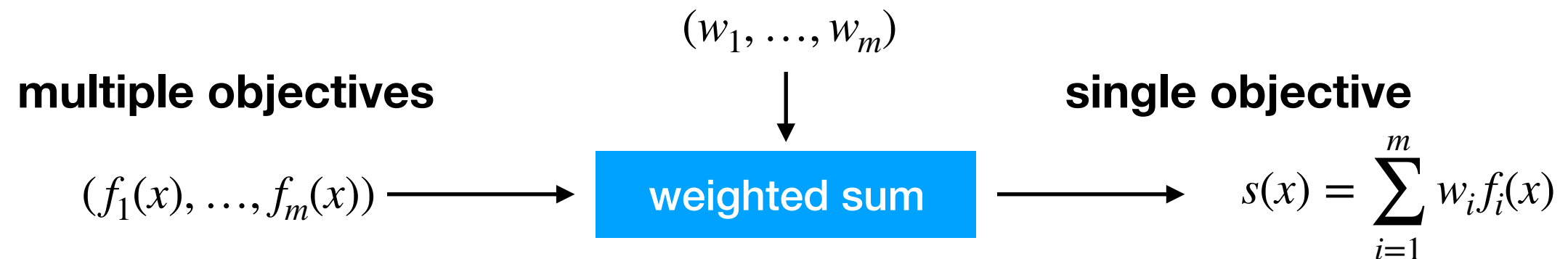
Back to a single objective: weighted sum



Each set of weights makes optimisation converge to a particular point of the Pareto front

What if the Pareto front is not convex ?

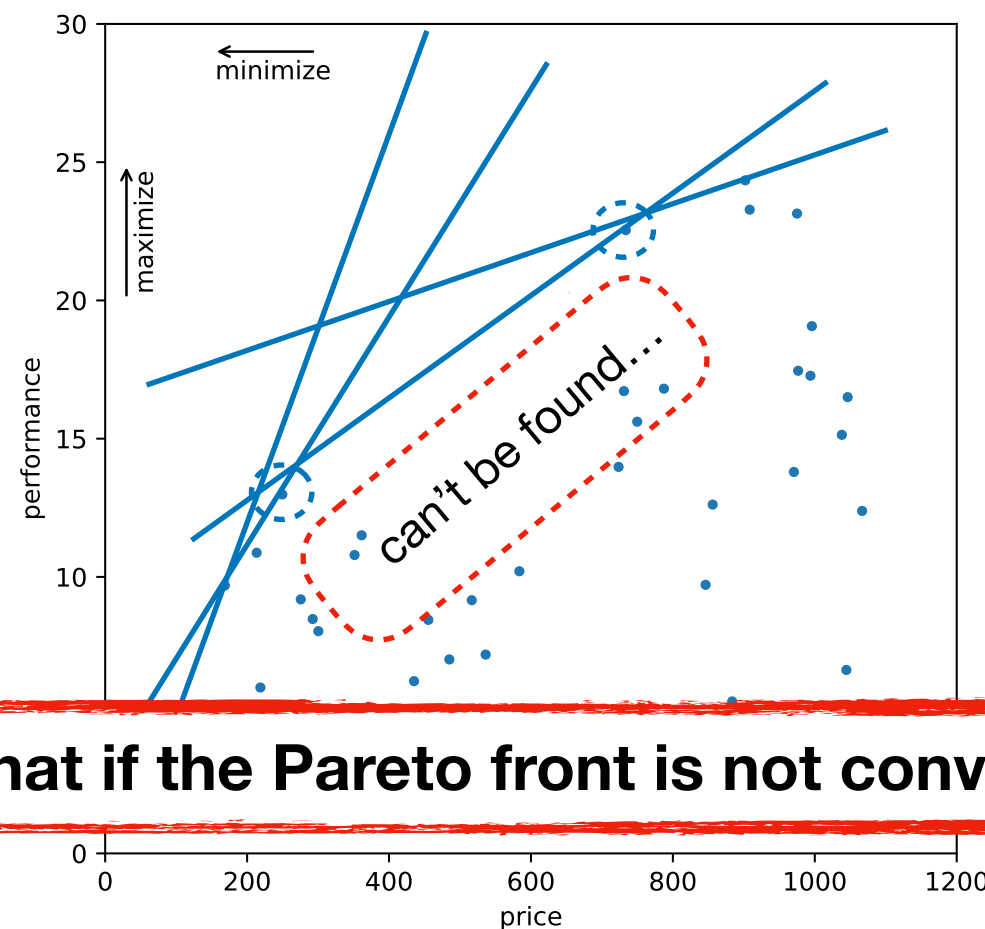
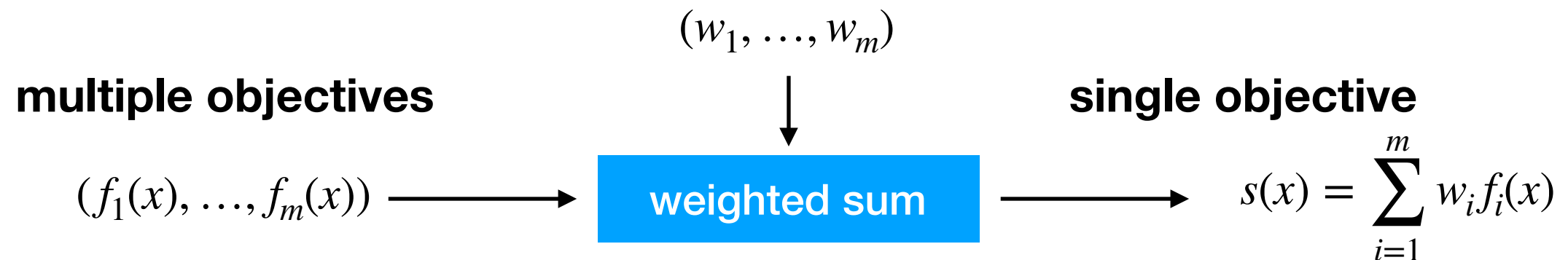
Back to a single objective: weighted sum



Each set of weights makes optimisation converge to a particular point of the Pareto front

What if the Pareto front is not convex ?

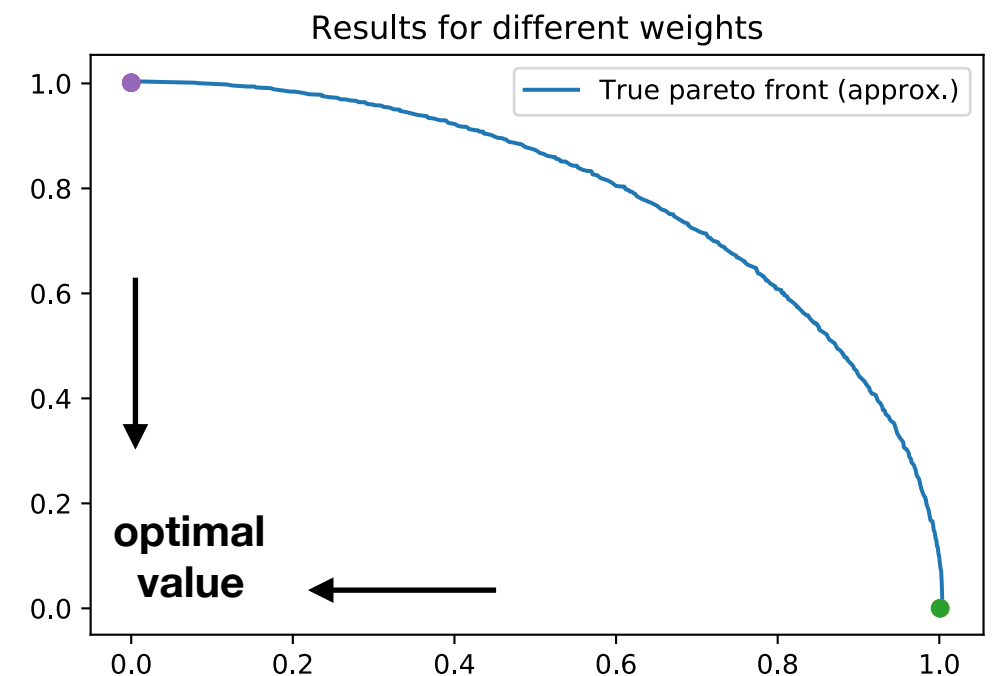
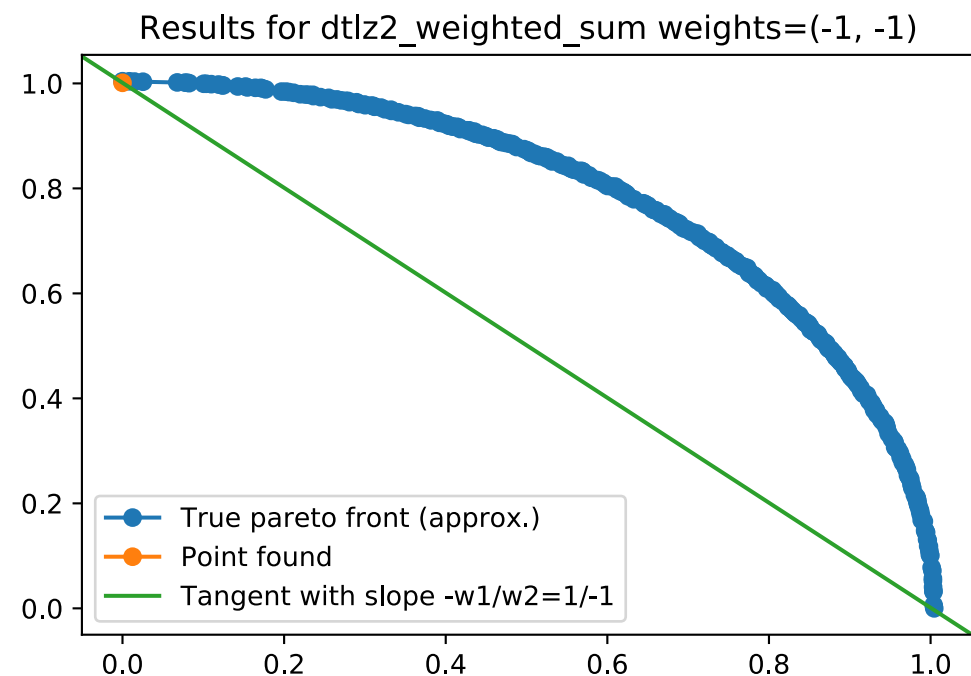
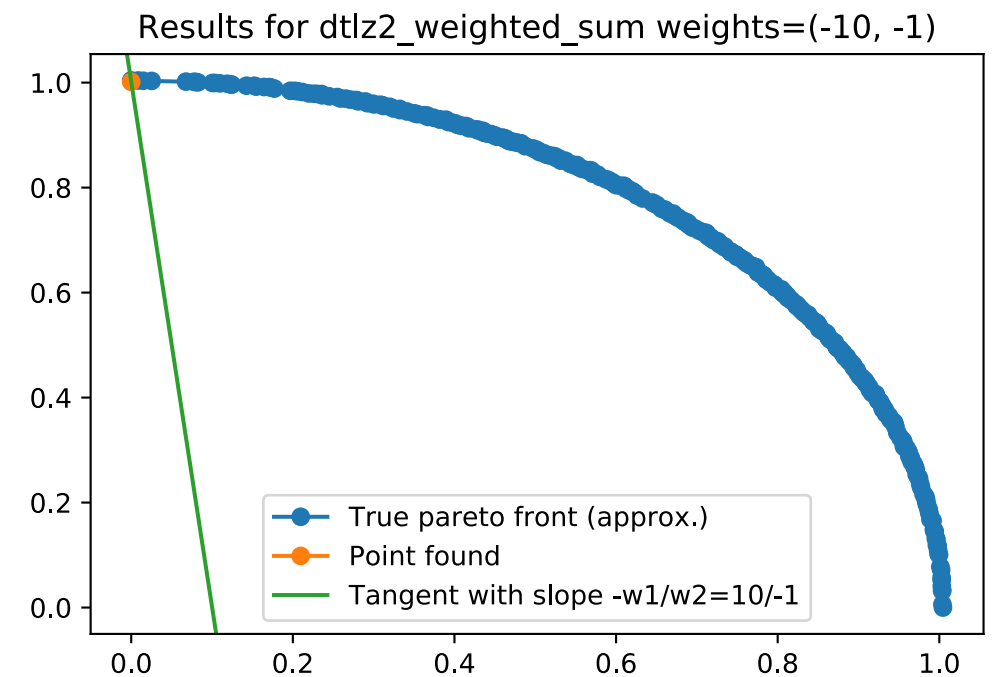
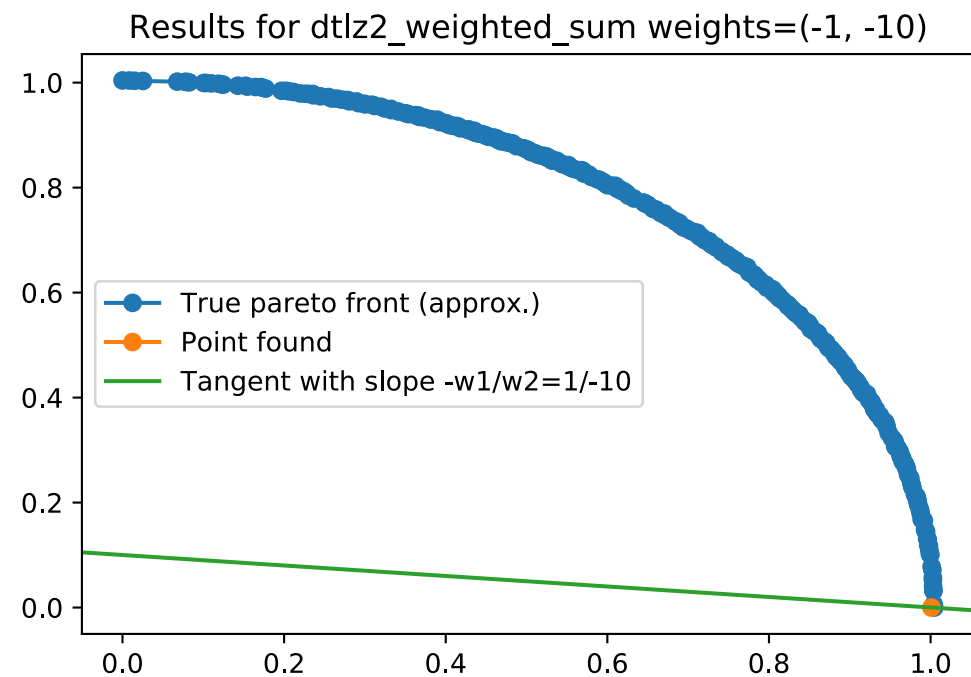
Back to a single objective: weighted sum



Each set of weights makes optimisation converge to a particular point of the Pareto front

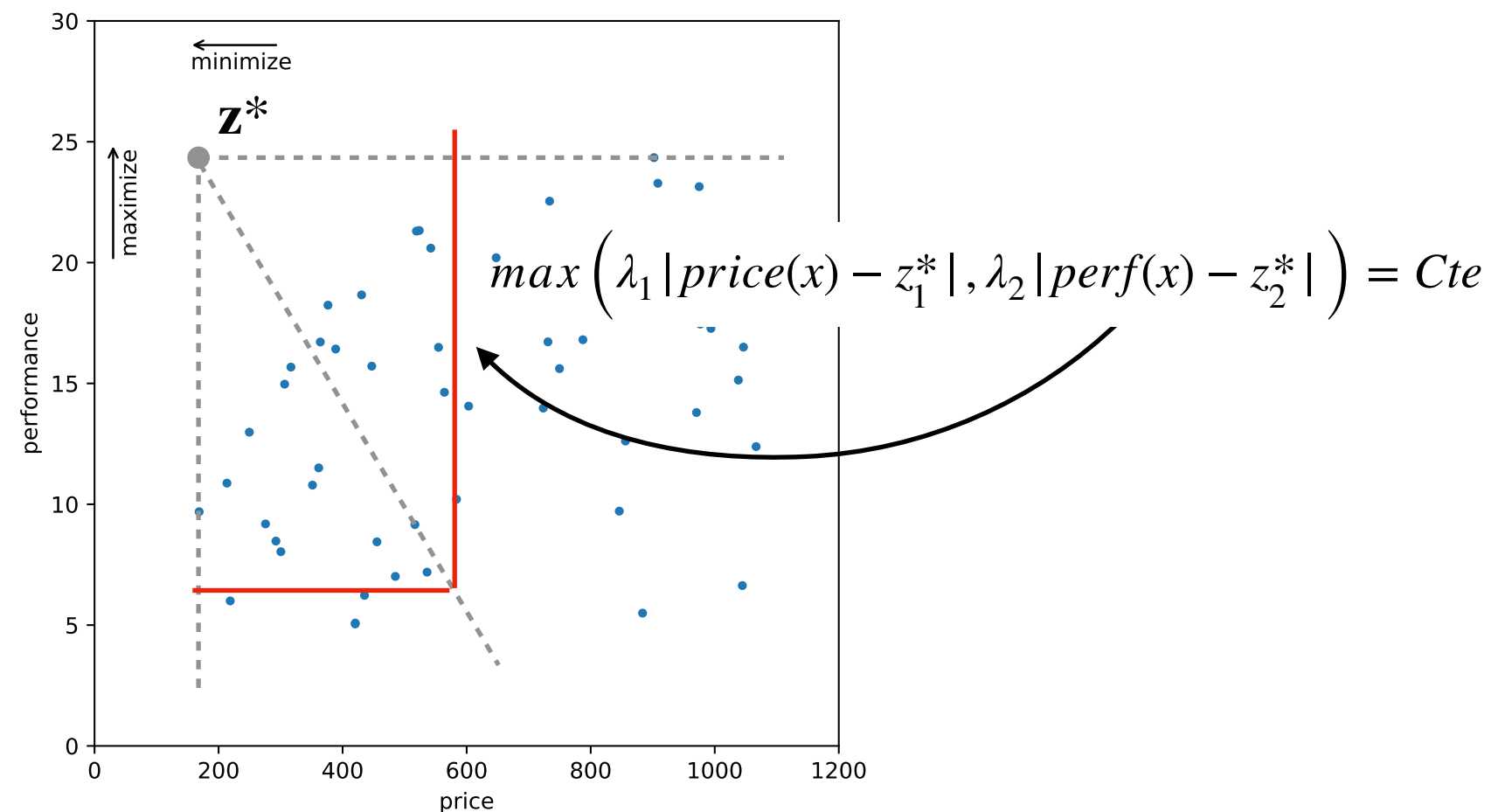
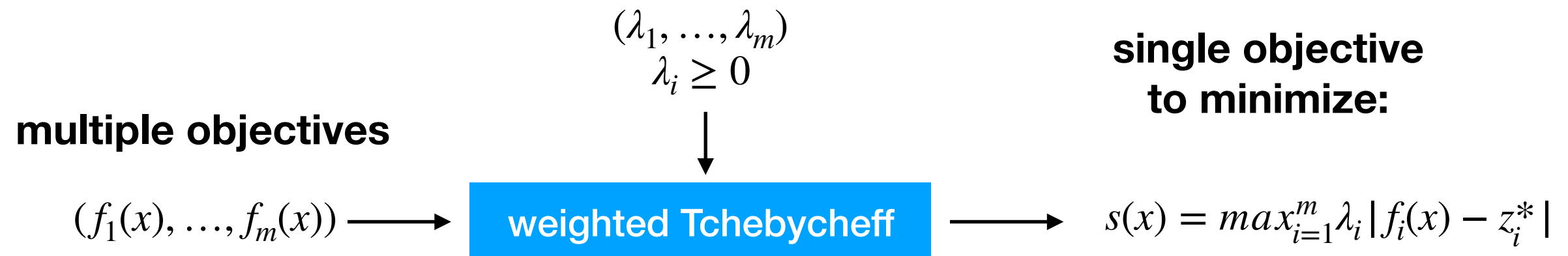
What if the Pareto front is not convex ?

Example

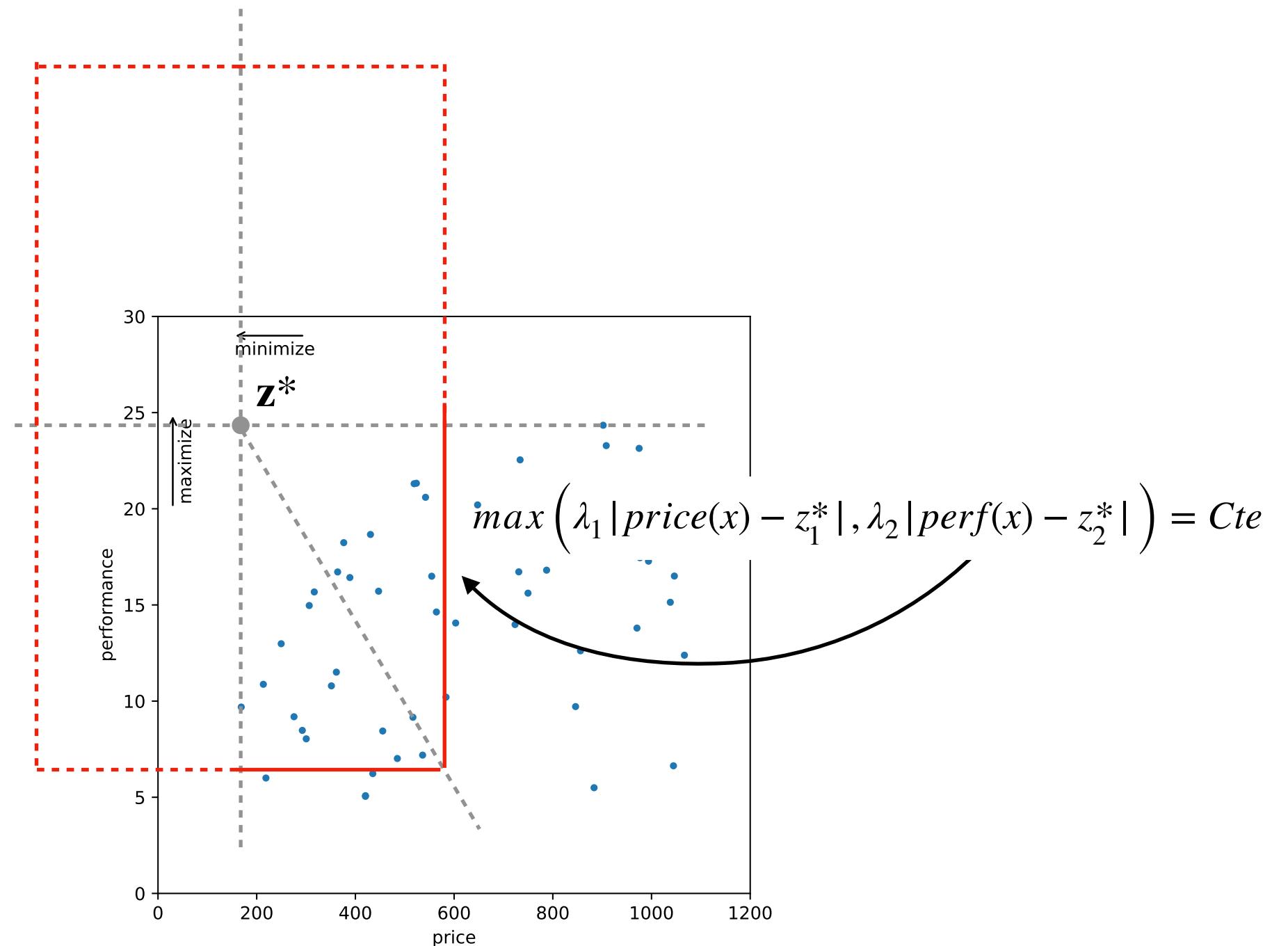


dtlz2 benchmark function, see 2_EA_MO_single.ipynb

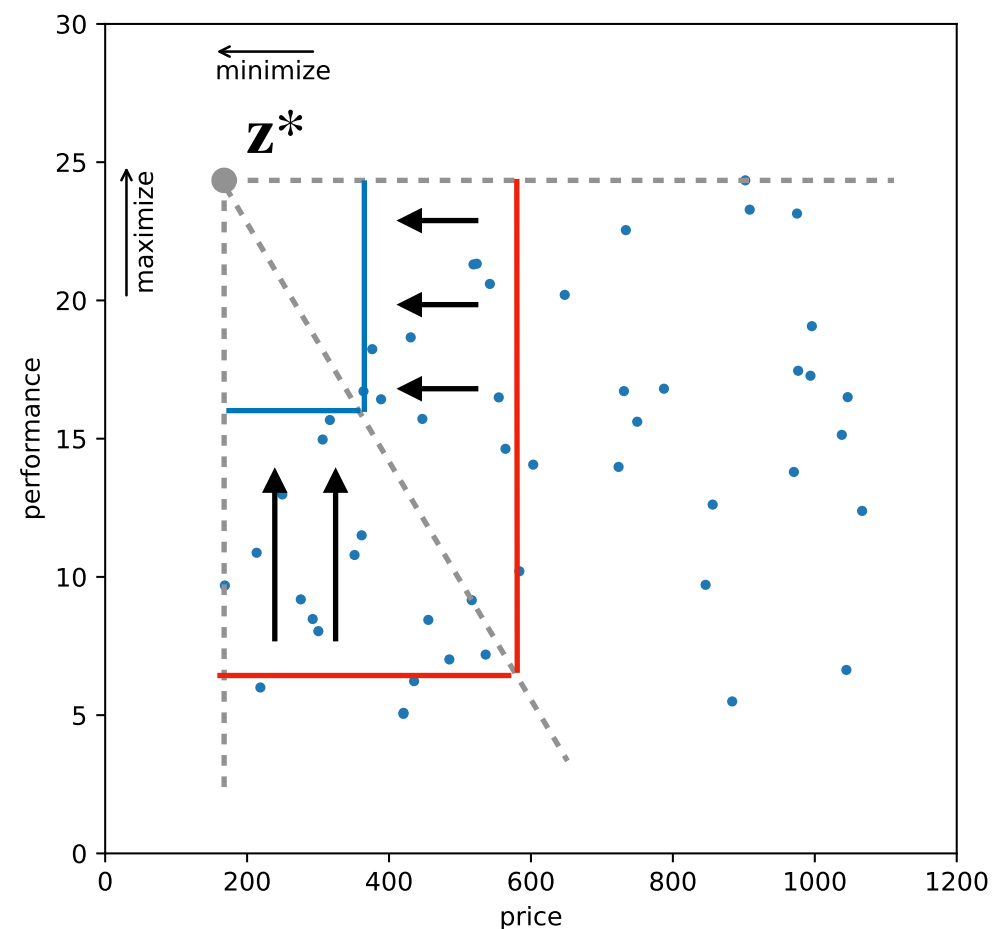
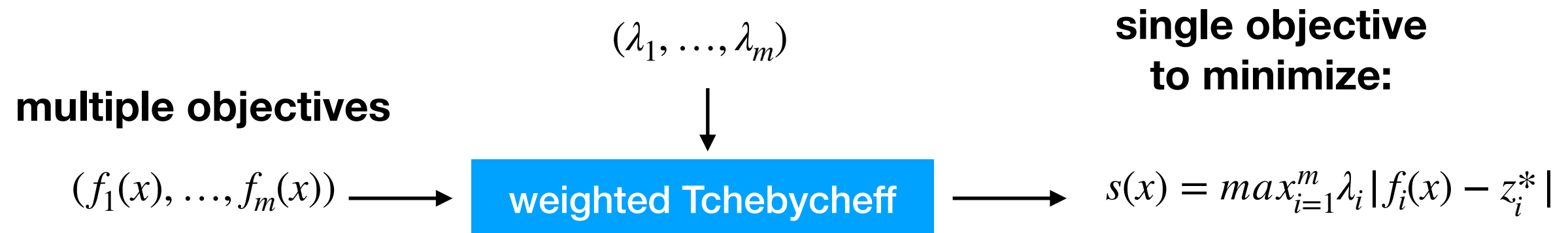
Back to a single objective: weighted Tchebycheff



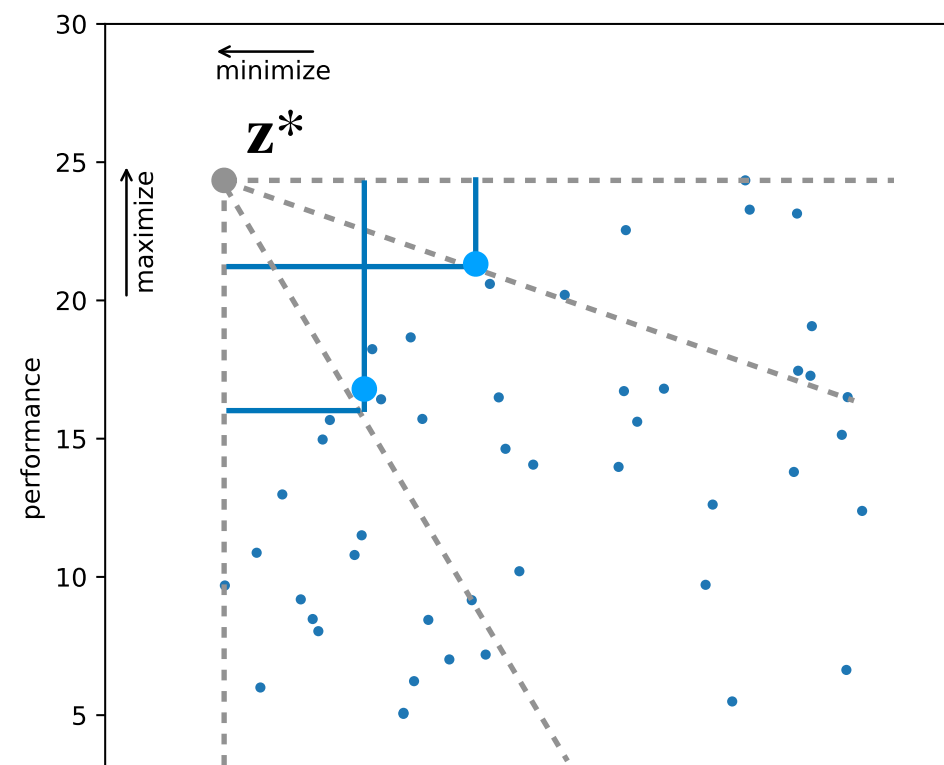
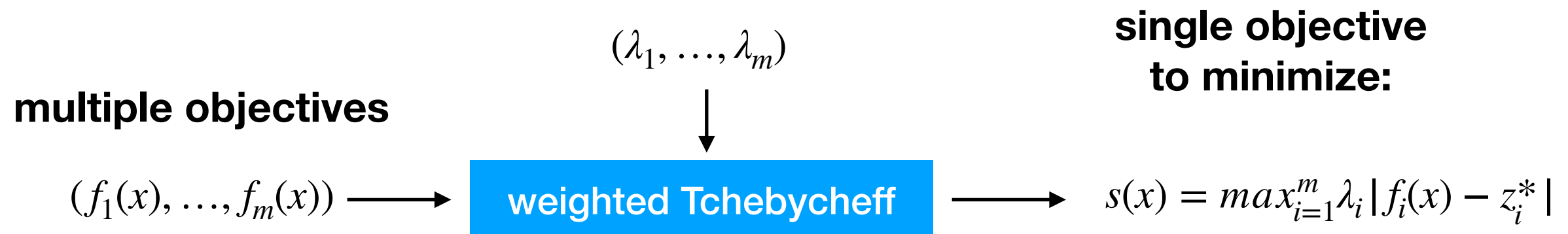
Back to a single objective: weighted Tchebycheff



Back to a single objective: weighted Tchebycheff



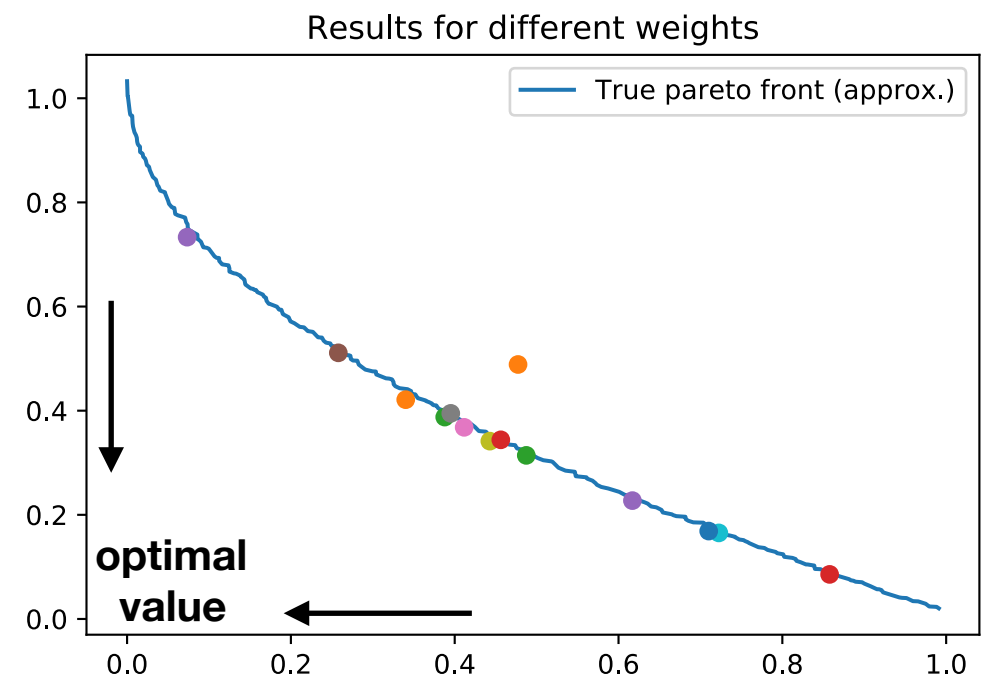
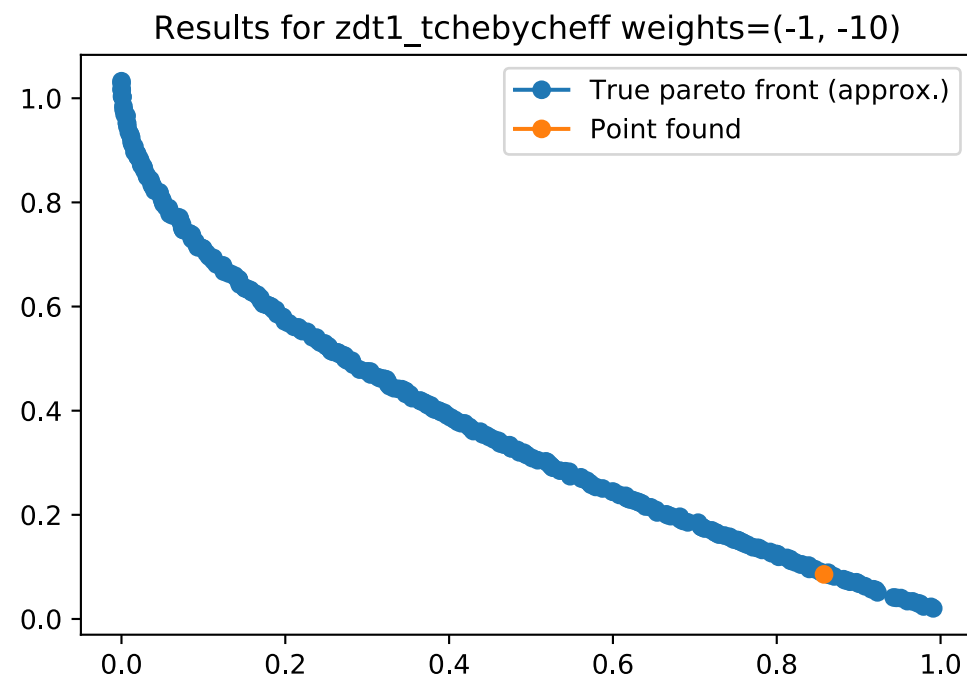
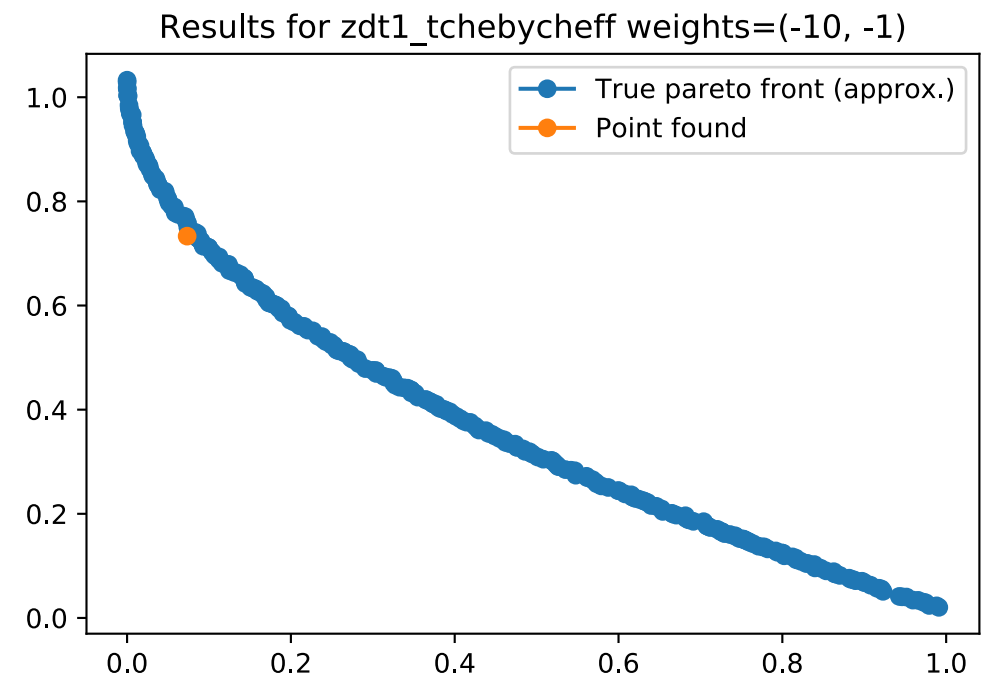
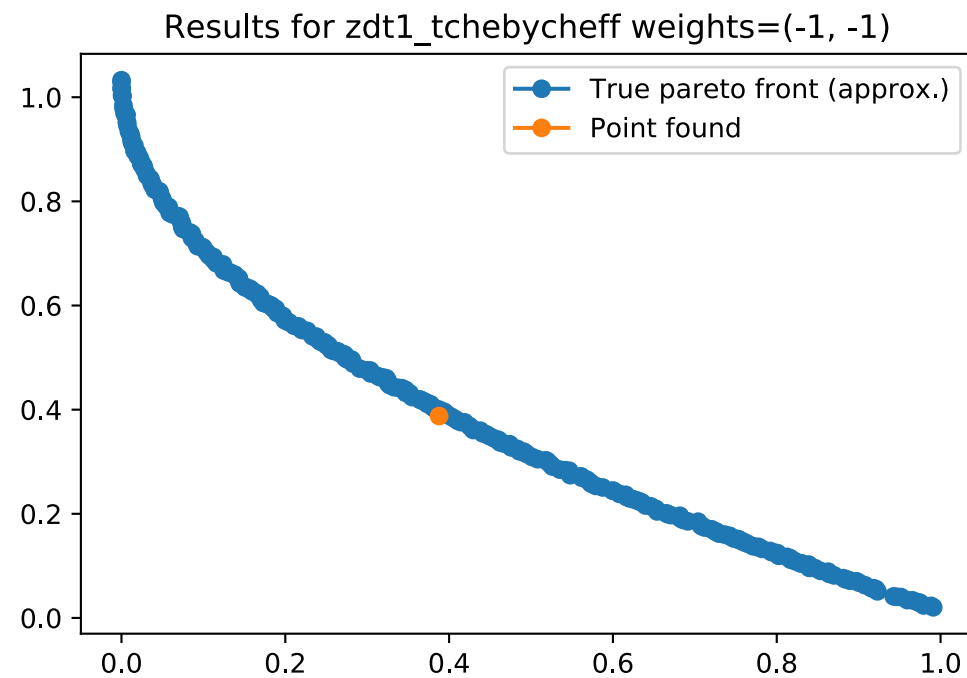
Back to a single objective: weighted Tchebycheff



Each set of weights makes optimisation converge to a particular point of the Pareto front

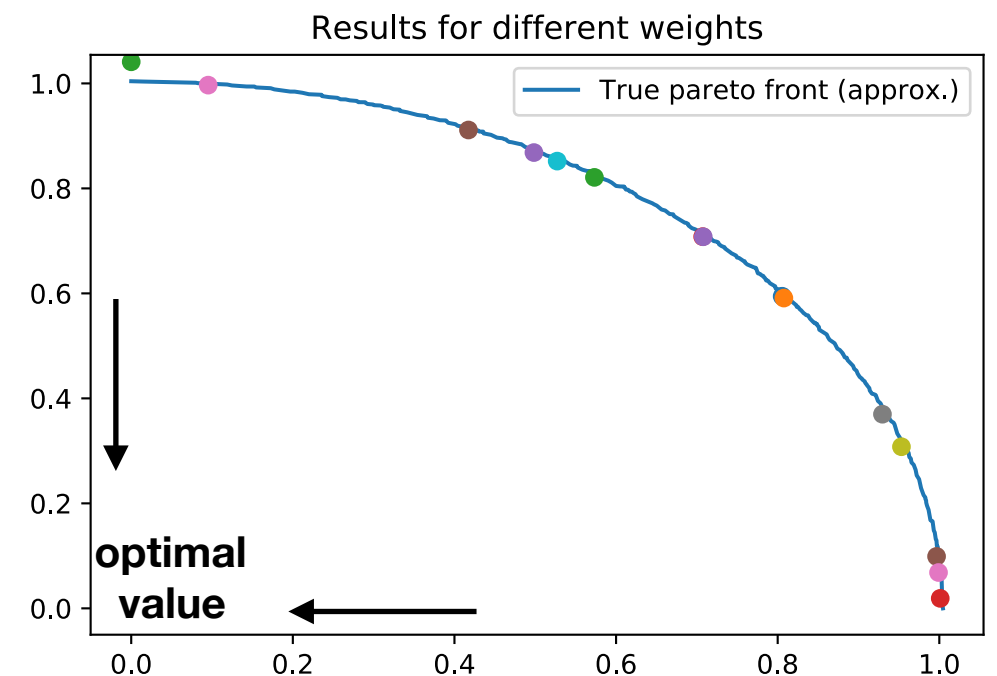
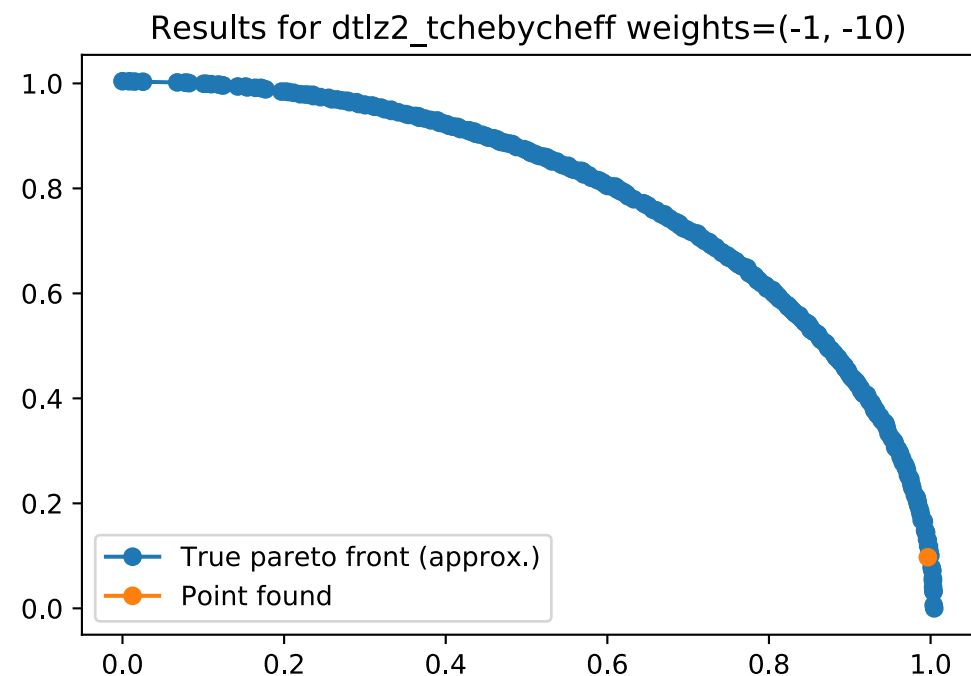
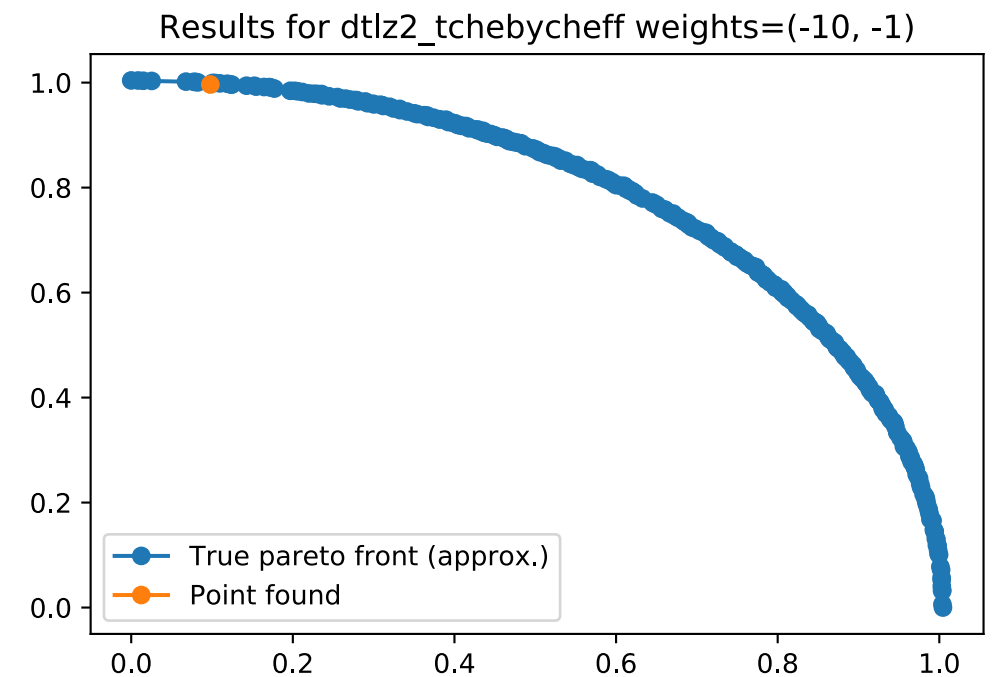
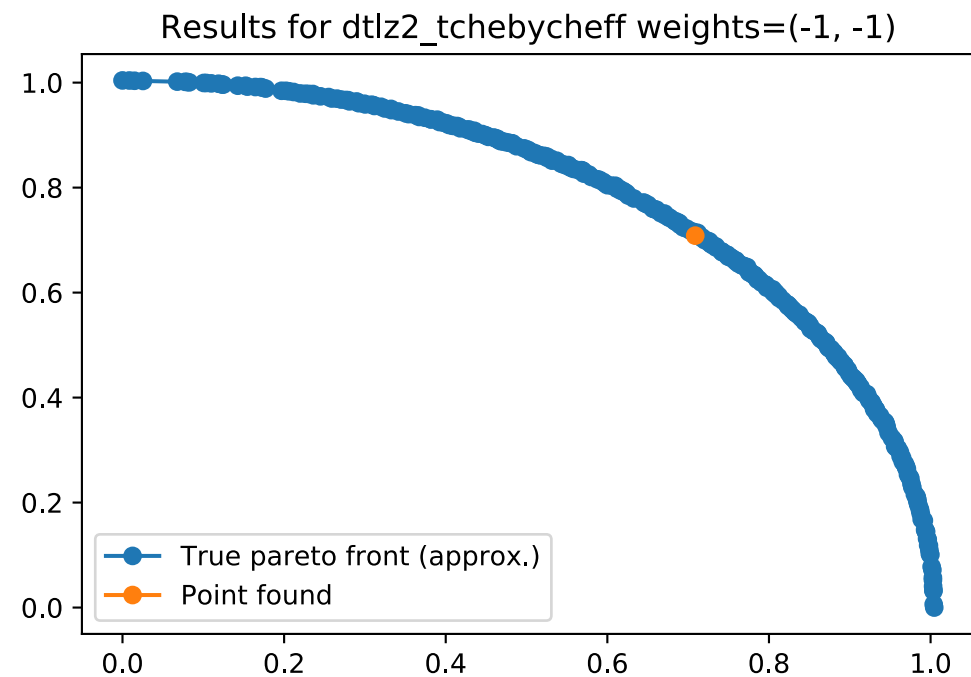
**Can find any point of the Pareto front,
no matter its shape**

Example: zdt1 function



see 2_EA_MO_single.ipynb (larger nbgen and popsize)

Example: dtlz2 function



see 2_EA_MO_single.ipynb

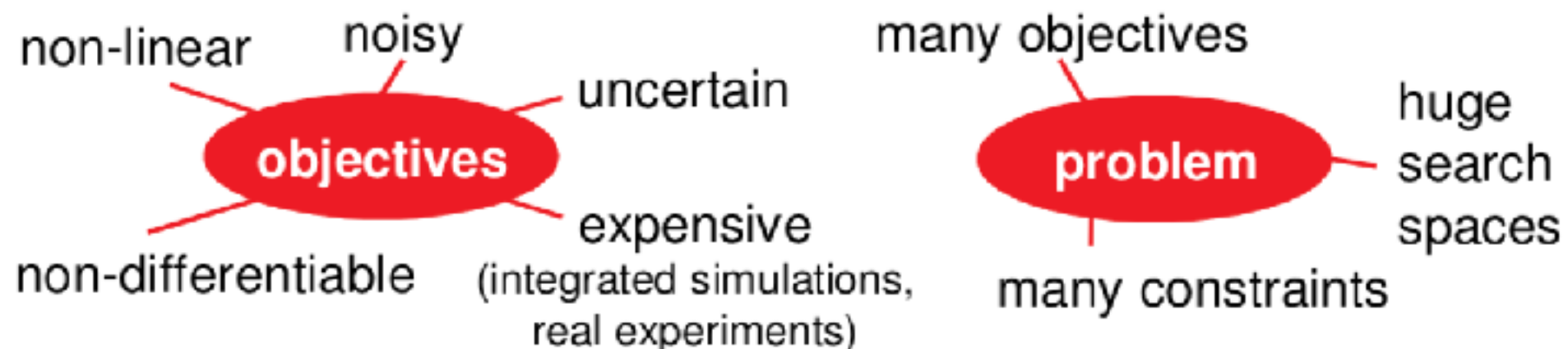
Overview



- Introduction
- Back to single-objective
- Multi-objective optimization
- Many-objective optimization

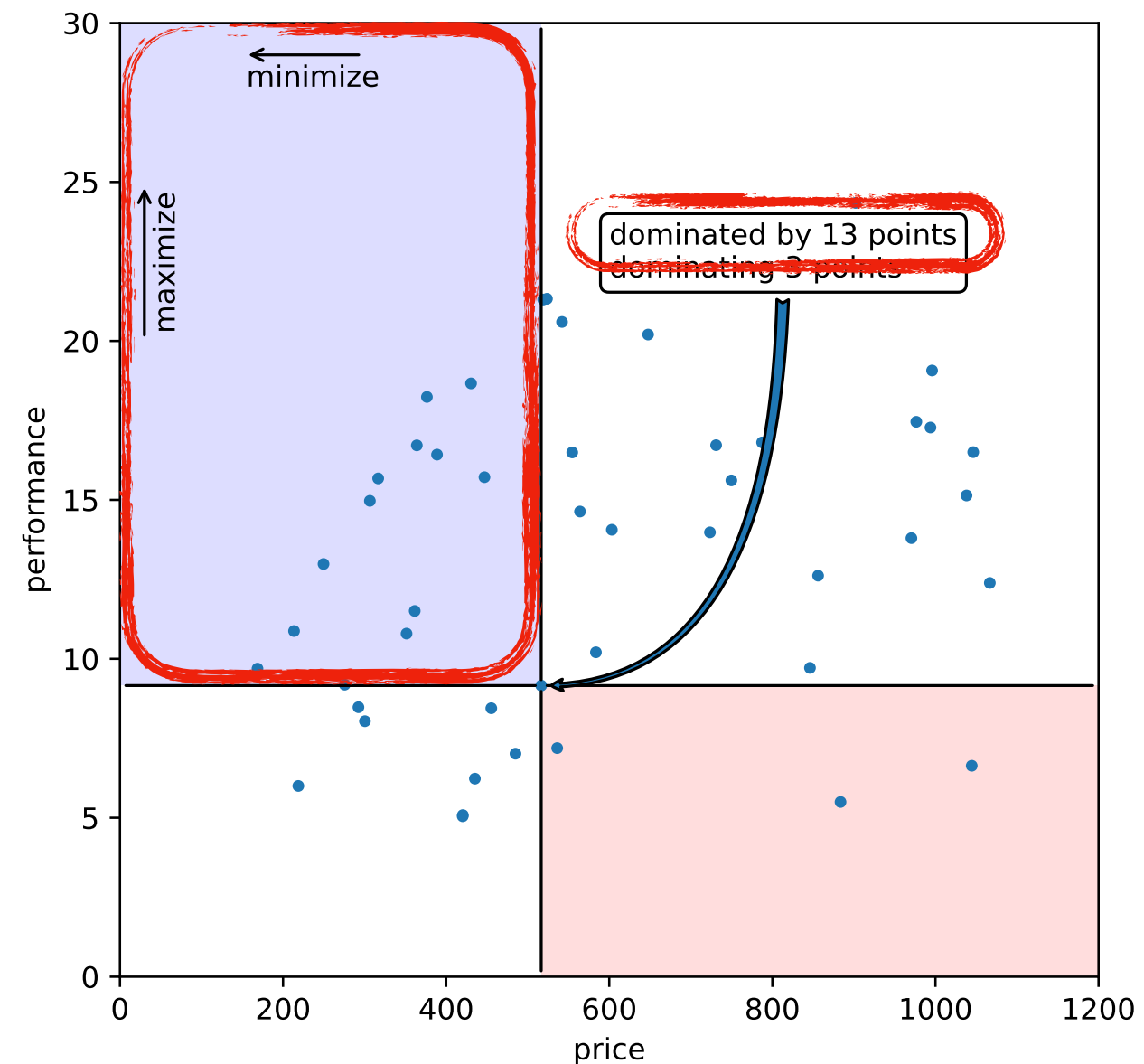
Evolutionary Multiobjective Optimization

- Evolutionary algorithms well suited to multi-objective optimization: **population-based**, can approximate the Pareto front in one run !
- Black box algorithms: adapted to real-world engineering problems



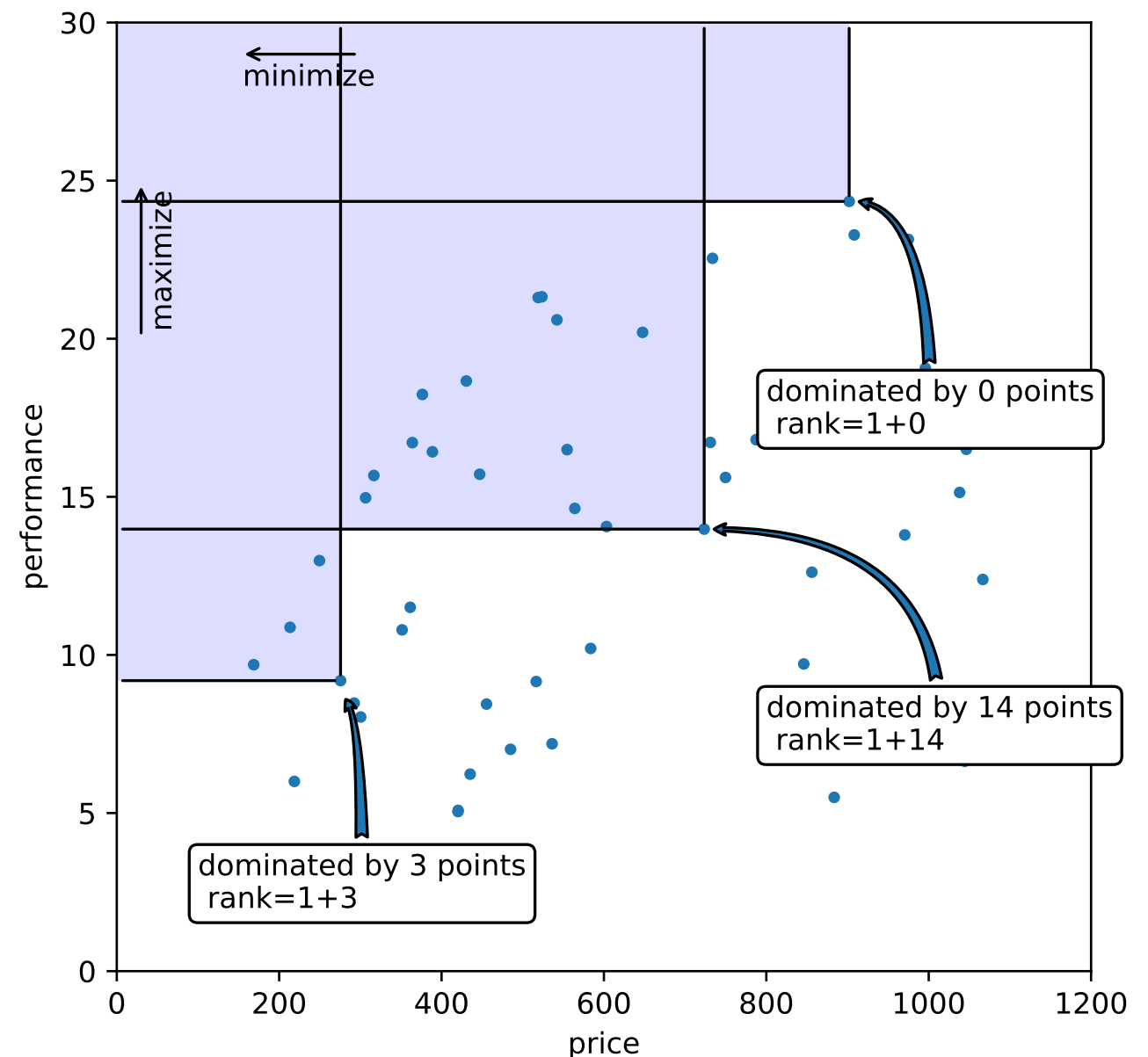
Multi-objective selection

- **Dominance rank:** by how many individuals is an individual dominated ?
(MOGA, NPGA)

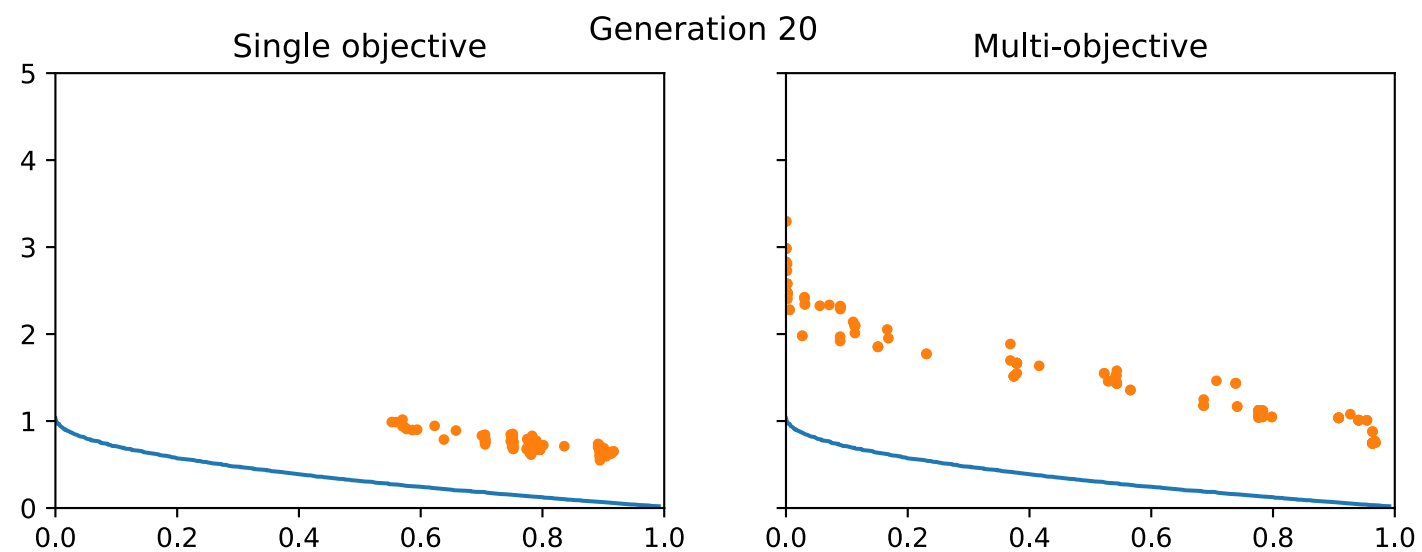
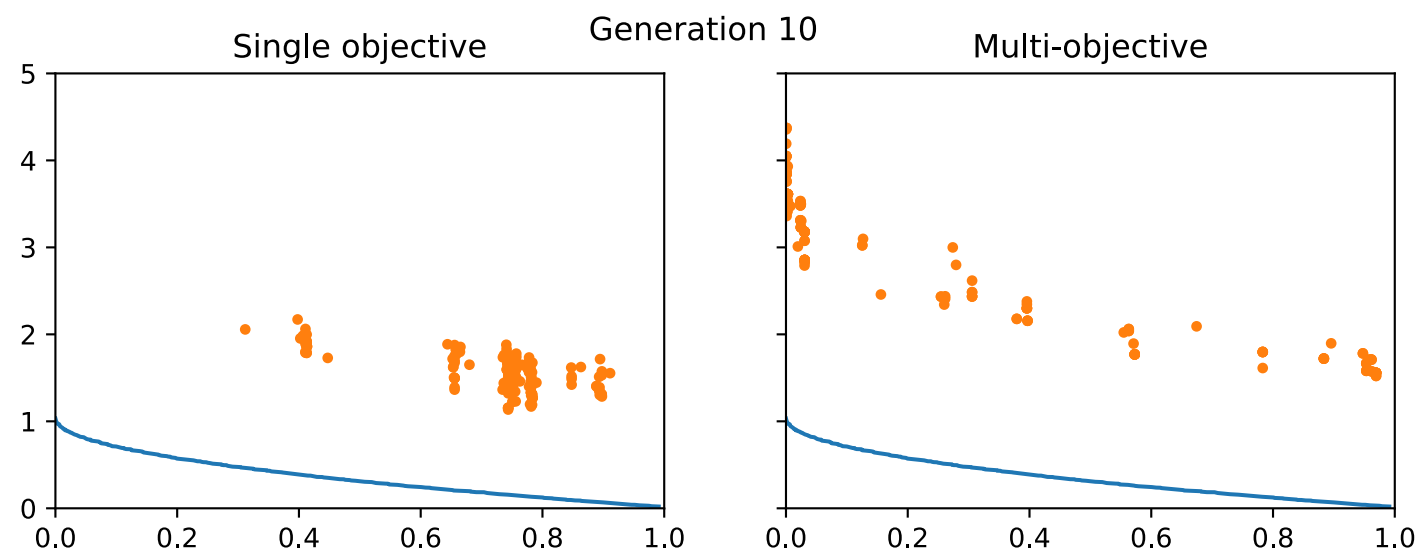
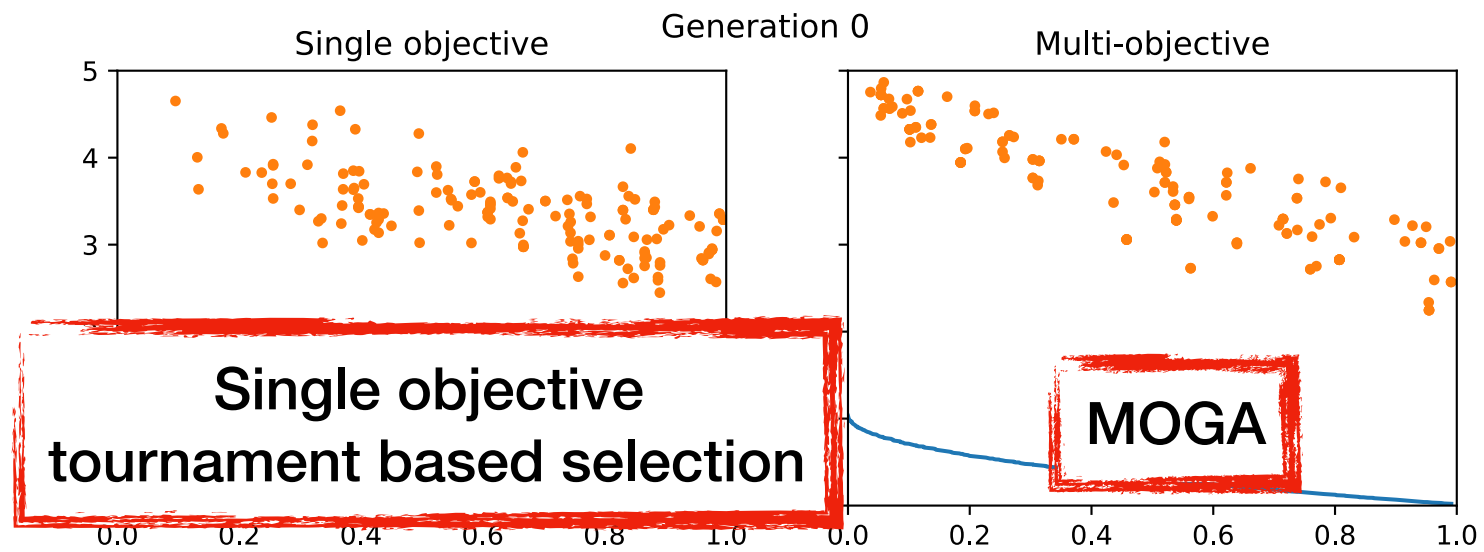


Multi-Objective Genetic Algorithm (MOGA)

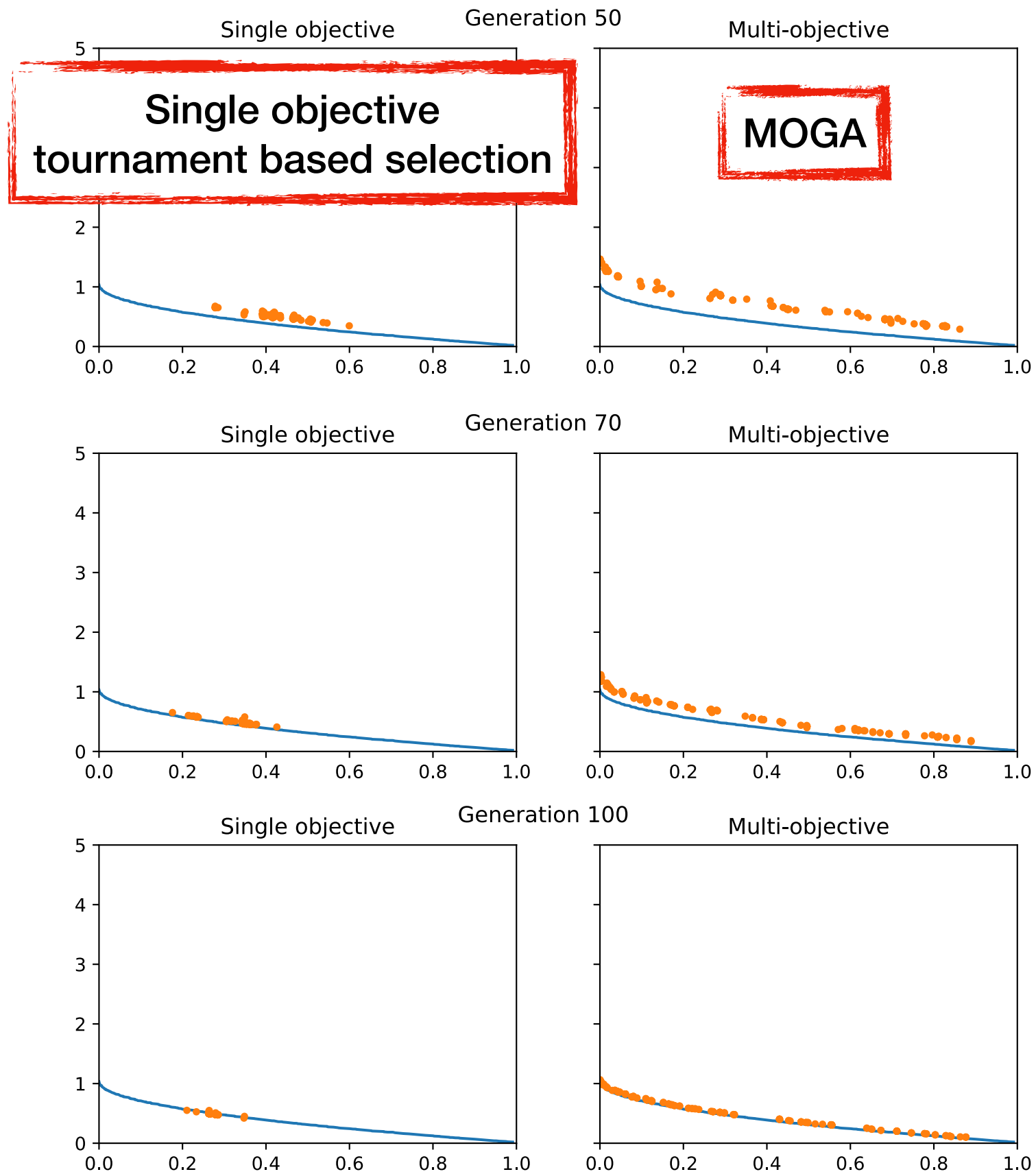
- Rank based fitness
- Adds a niche count factor to keep diversity among non dominated solutions



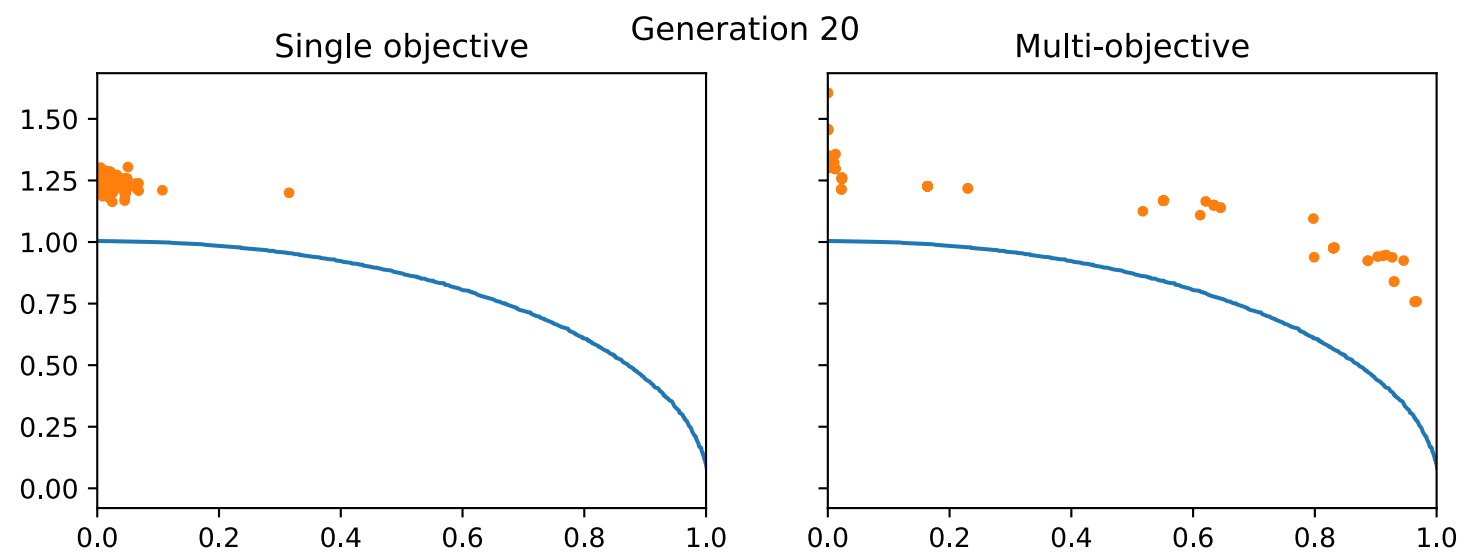
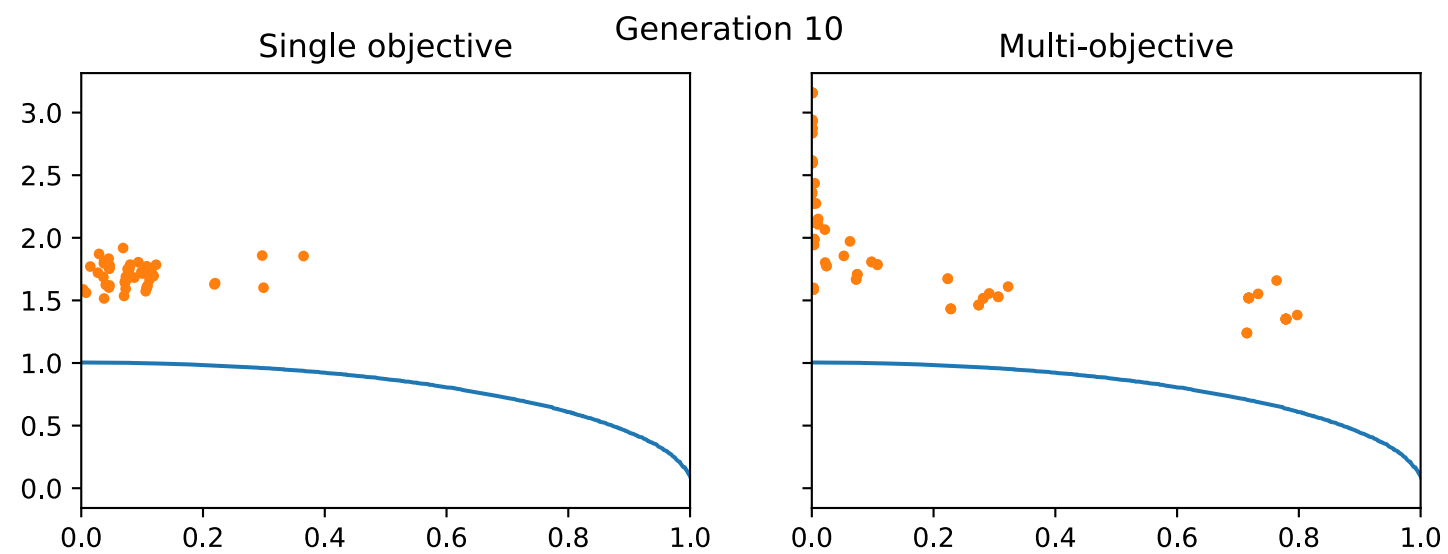
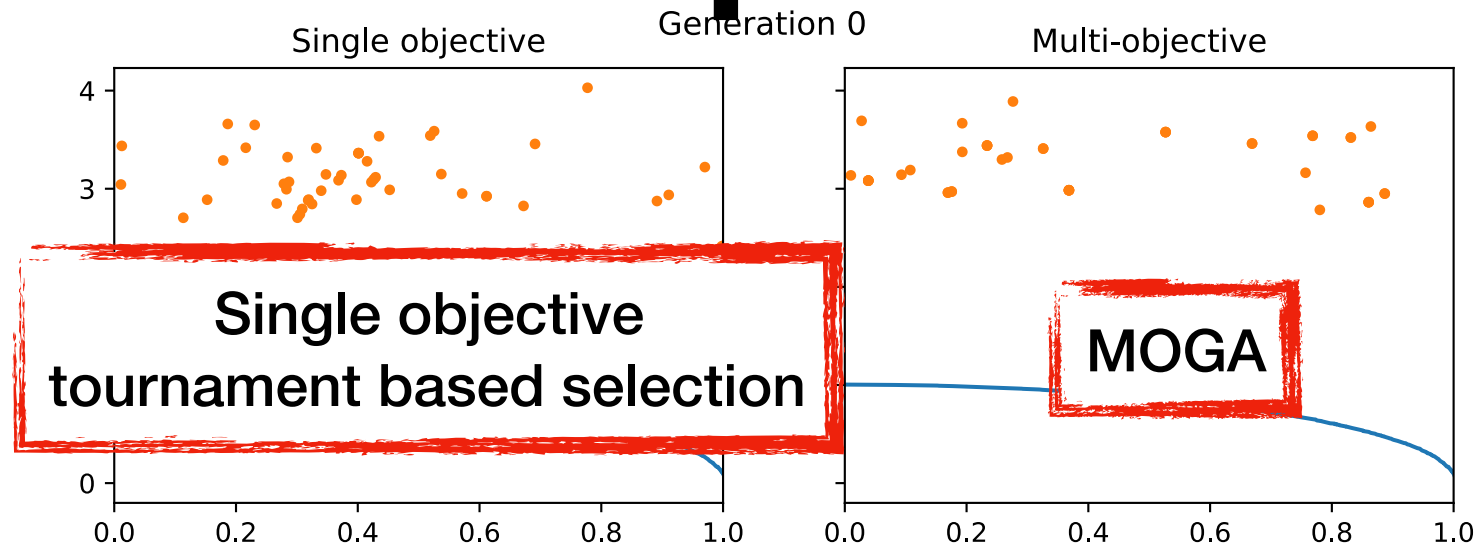
Example: zdt1



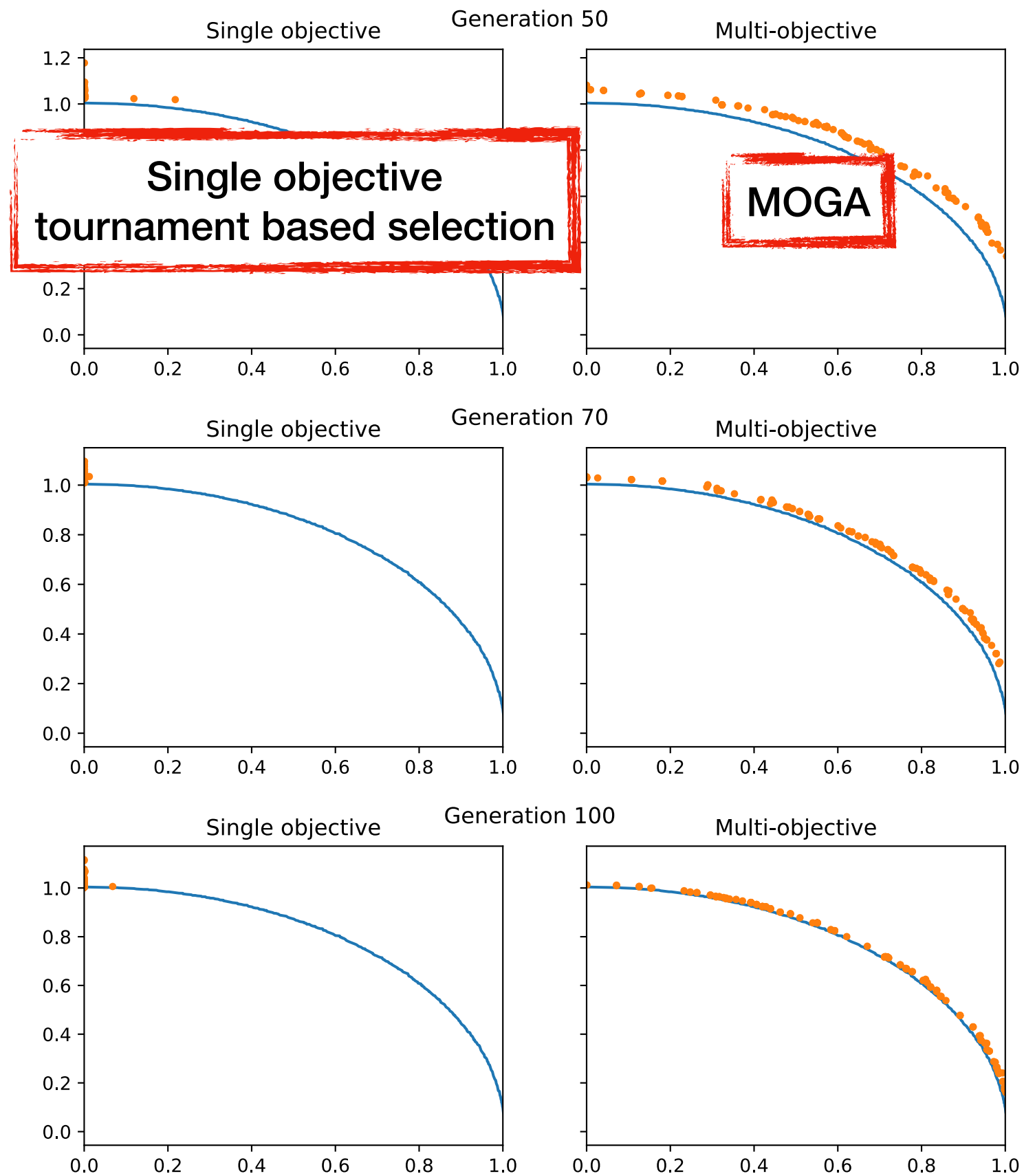
Example: zdt1



Example: dtlz2

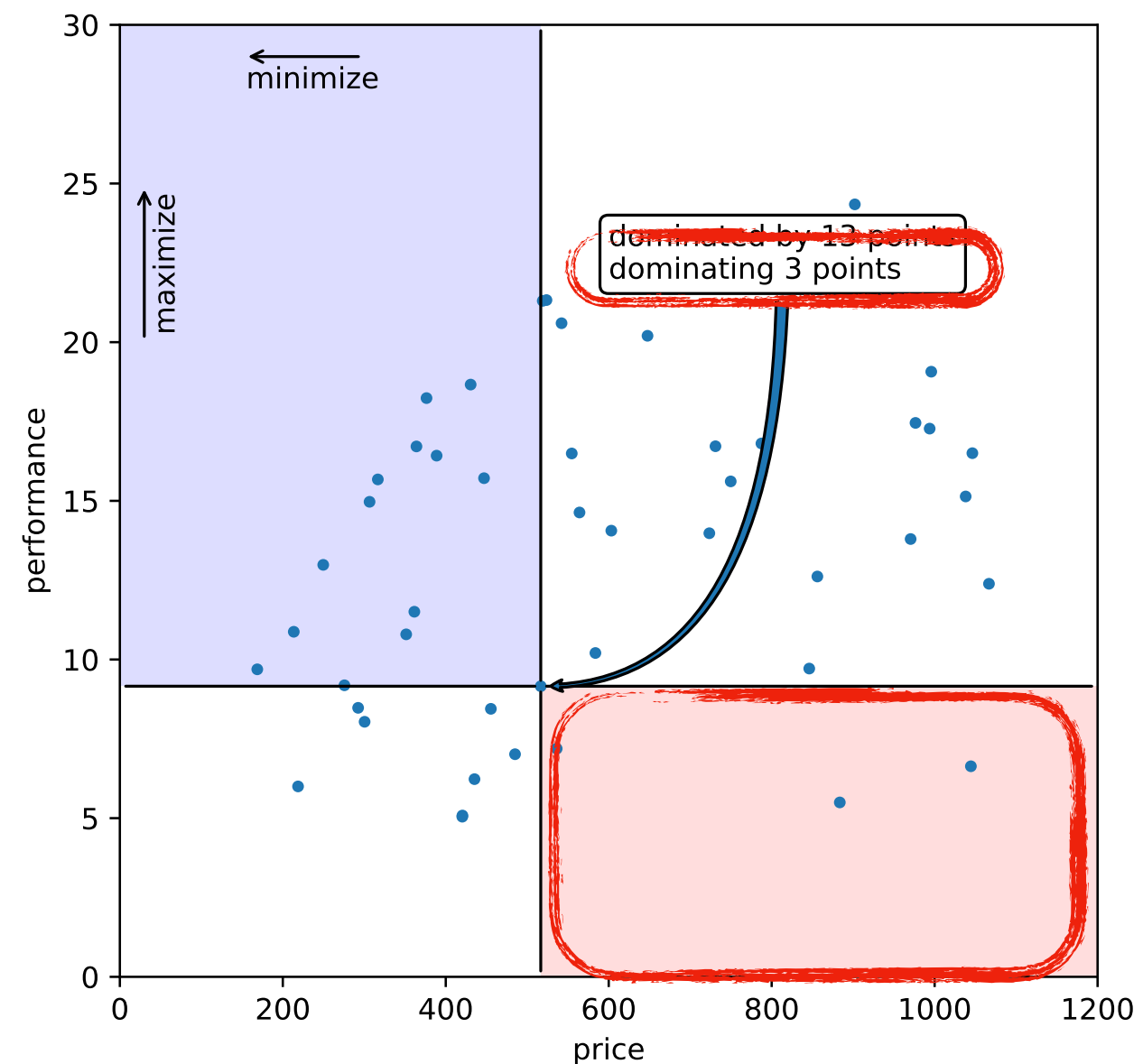


Example: dtlz2



Multi-objective selection

- **Dominance rank:** by how many individuals is an individual dominated ? (MOGA, NPGA)
- **Dominance count:** how many individuals are dominated ? (SPEA, SPEA2)



Strength Pareto Evolutionary Algorithm 2 (SPEA2)

Input: population size N , archive size $\bar{N}$, number of generations G

Output: non-dominated set

Step 1, Initialization:

$P_0 \leftarrow \text{random_init}(N)$

$\bar{P}_0 \leftarrow \emptyset$

$g \leftarrow 0$

Step 2, Fitness assignment:

for $p \in P_g \cup \bar{P}_g$ do

$\text{raw_fitness} \leftarrow \sum_{q \in P_g \cup \bar{P}_g, q \succ p} \text{nb_dominating}(q)$

$\text{density} \leftarrow \frac{1}{d(p, \text{nn}_k) + 2}$

$\# \text{nn}_k = k\text{-th nearest neigh. and } k = \sqrt{N + \bar{N}}$

$\# \text{ fitness (to be minimized):}$

$p.\text{fit} = \text{raw_fitness} + \text{density}$

$\# \text{ Rq: density} < 1$

end for

Step 3, Environment selection:

$\bar{P}_{g+1} \leftarrow \text{non_dominated}(P_g \cup \bar{P}_g)$

if $|\bar{P}_{g+1}| > \bar{N}$ then

$\bar{P}_{g+1} \leftarrow \text{truncates}(\bar{P}_{g+1})$

else if $|\bar{P}_{g+1}| < \bar{N}$ then

$\bar{P}_{g+1} \leftarrow \bar{P}_{g+1} \cup \text{get_best_fit}((P_g \cup \bar{P}_g) \setminus \bar{P}_{g+1})$

end if

Step 4, Termination:

if $g \geq G$ then

 return $\text{non_dominated}(\bar{P}_{g+1})$

end if

Step 5, New population:

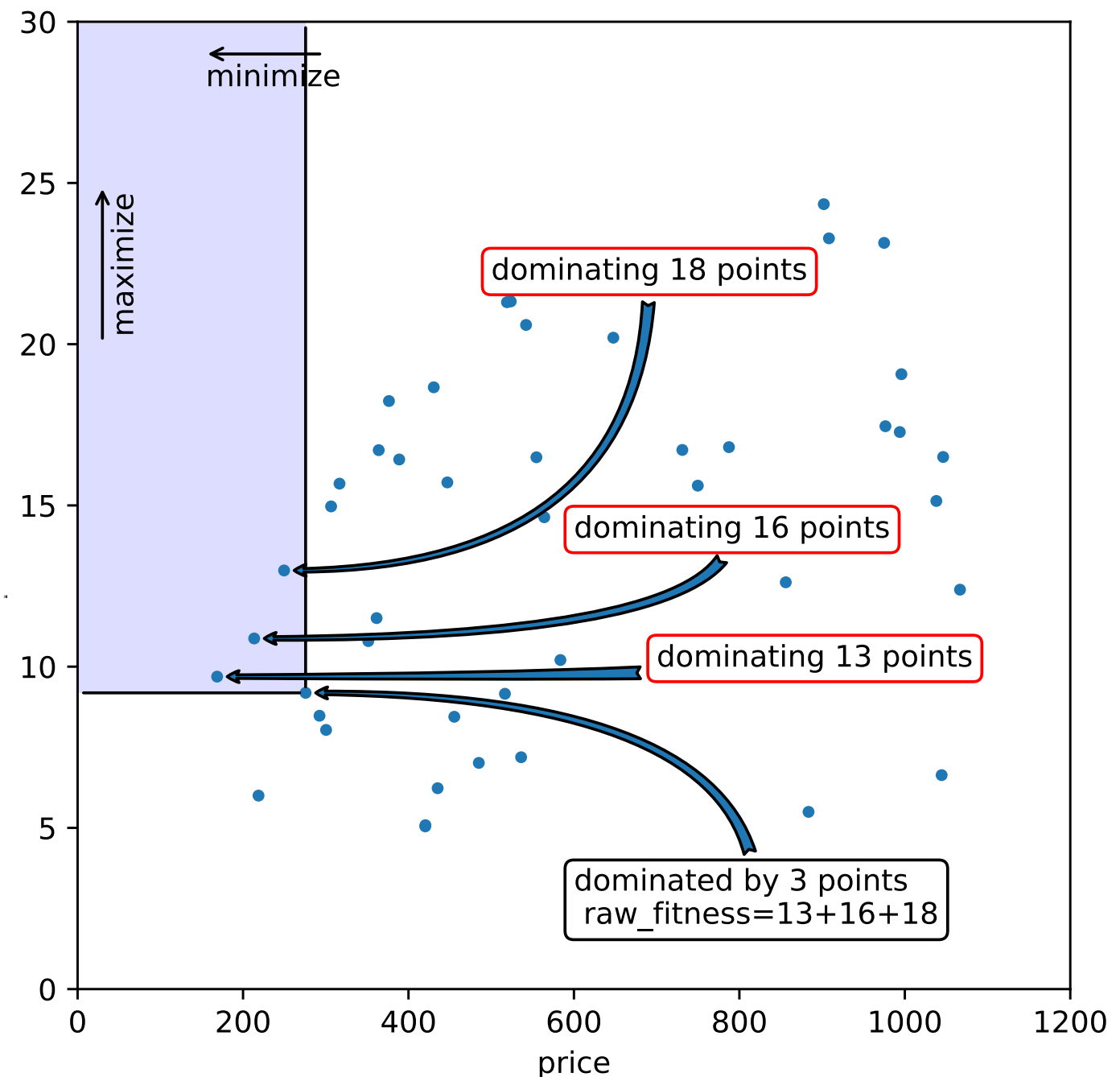
$P_{g+1} \leftarrow \text{binary_tournament_sel}(\bar{P}_{g+1})$

$P_{g+1} \leftarrow \text{crossover}(P_{g+1})$

$P_{g+1} \leftarrow \text{mutate}(P_{g+1})$

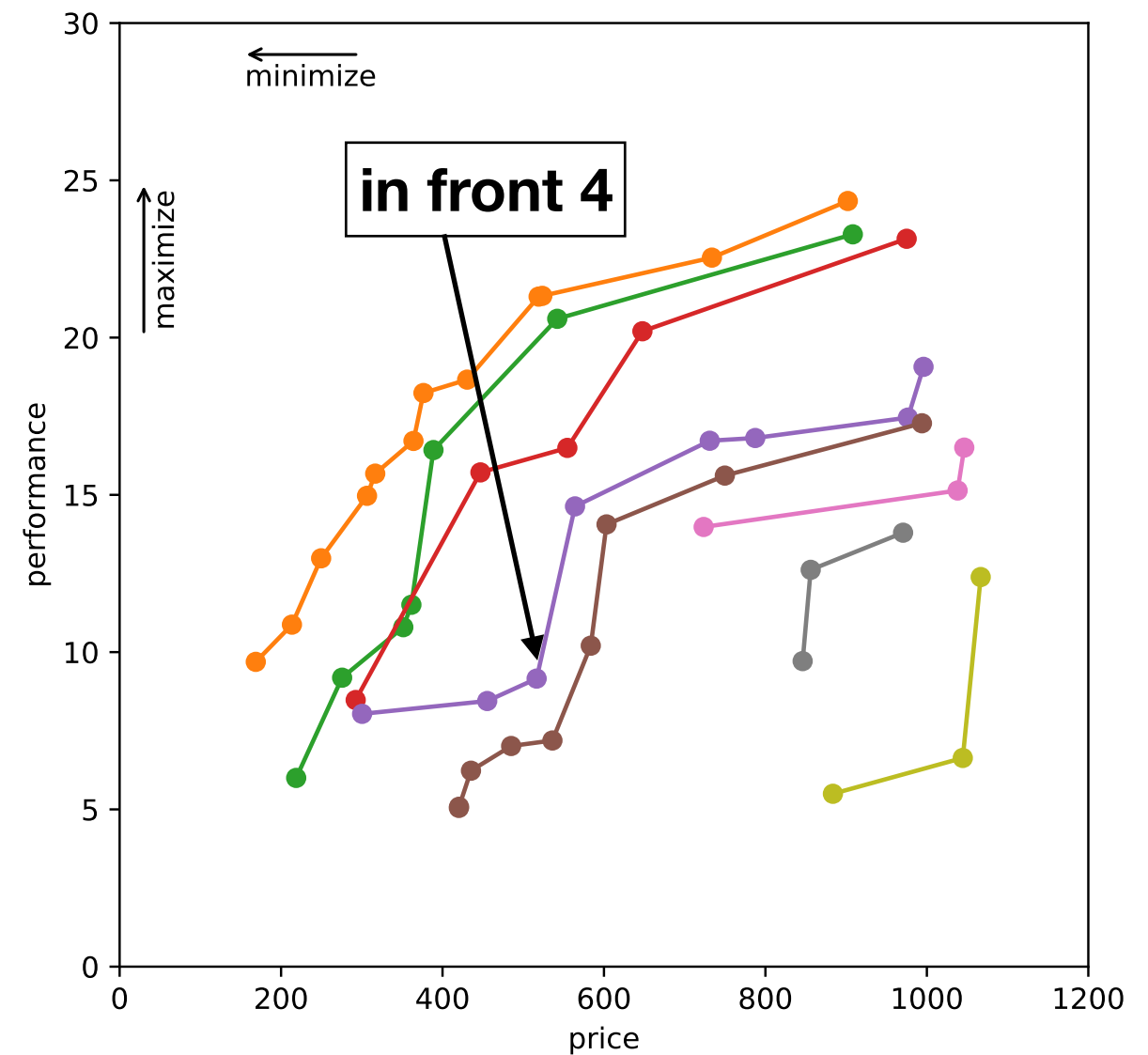
$g \leftarrow g + 1$

back to Step 2



Multi-objective selection

- **Dominance rank:** by how many individuals is an individual dominated ? (MOGA, NPGA)
- **Dominance count:** how many individuals are dominated ? (SPEA, SPEA2)
- **Dominance depth:** on what front is an individual ? (NSGA-II and most recent algorithms)



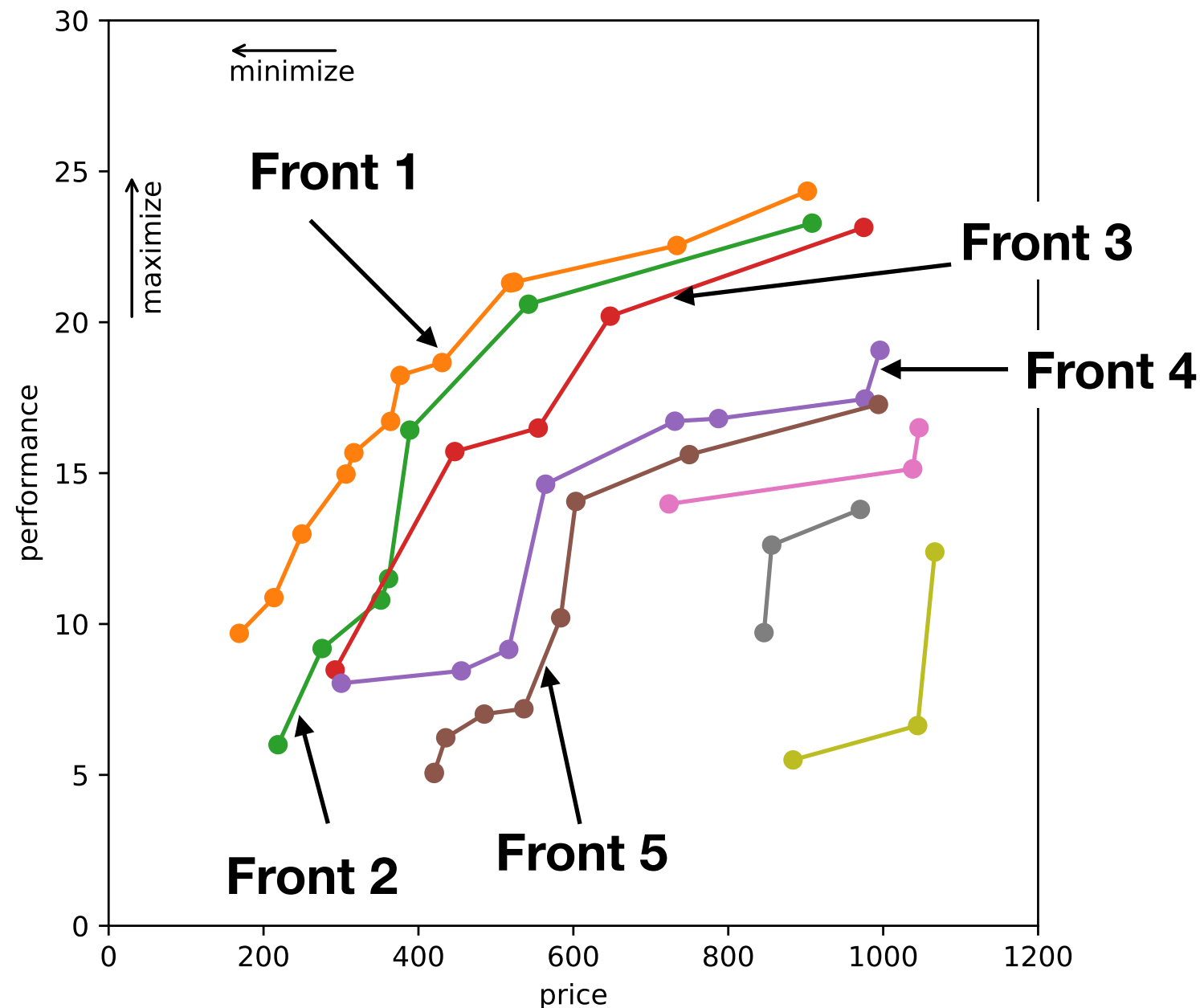
NSGA-2: Non Dominated Sorting Genetic Algorithm - 2

- Algorithm proposed in 2000, but still a reference for multi-objective optimisation
- Main features: selection algorithm
 - **Elitist**
 - **Dominance depth** with a non dominated sorting in $O(MN^2)$
 - Diversity preservation using **crowding distance**

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. A. M. T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation*, 6(2), 182-197.

Deb, K., Agrawal, S., Pratap, A., & Meyarivan, T. (2000, September). A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. In International conference on parallel problem solving from nature (pp. 849-858). Springer, Berlin, Heidelberg.

NSGA-2: non dominated sorting



Deb, K., Agrawal, S., Pratap, A., & Meyarivan, T. (2000). **A Fast Elitist Non-Dominated Sorting Genetic Algorithm for Multi-Objective Optimization: NSGA-II**. Proceedings of the Parallel Problem Solving from Nature VI Conference, 849–858.

How to find the Pareto Front ?

Input: population $\mathcal{P}$

Output: Pareto front $\mathcal{F}$

Parameters: population size N , number of objectives M

$\mathcal{F} \leftarrow \emptyset$

for $i \in \mathcal{P}$ do

 if i is not dominated by any $j \in \mathcal{F}$ then

 for $j \in \mathcal{P}$ do

 if j dominates i then

 continue

 end if

 end for

 if not dominated then

$\mathcal{F} \leftarrow \mathcal{F} \cup \{i\}$

 end if

 end if

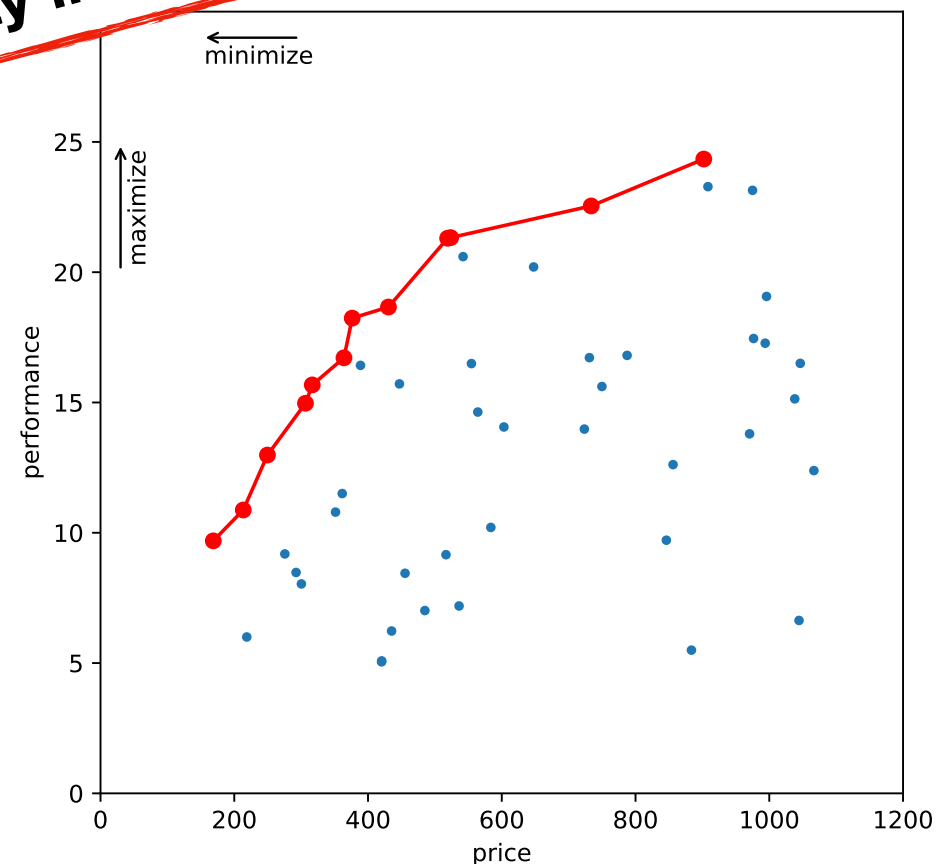
end for

return $\mathcal{F}$

Complexity in $O(MN^2)$

Complexity in $O(MN^2)$ to compute a single Pareto Front

**Computing all fronts:
Worst case: fronts of size 1
Complexity in $O(MN^3)$**



How to find the fronts?

Input: population $\mathcal{P}$

Output: $\mathcal{N} = \{n_i\}$ with n_i = number of solutions dominating i , $\mathcal{S} = \{S_i\}$ with S_i the set of individuals dominated by i

Parameters: population size N , number of objectives M

for $i \in \mathcal{P}$ **do**

$n_i \leftarrow 0$

end for

for $i \in \mathcal{P}$ **do**

$S_i \leftarrow \emptyset$

for $j \in \mathcal{P}, j \neq i$ **do**

if i dominates j **then**

$S_i \leftarrow S_i \cup \{j\}$

$n_j \leftarrow n_j + 1$

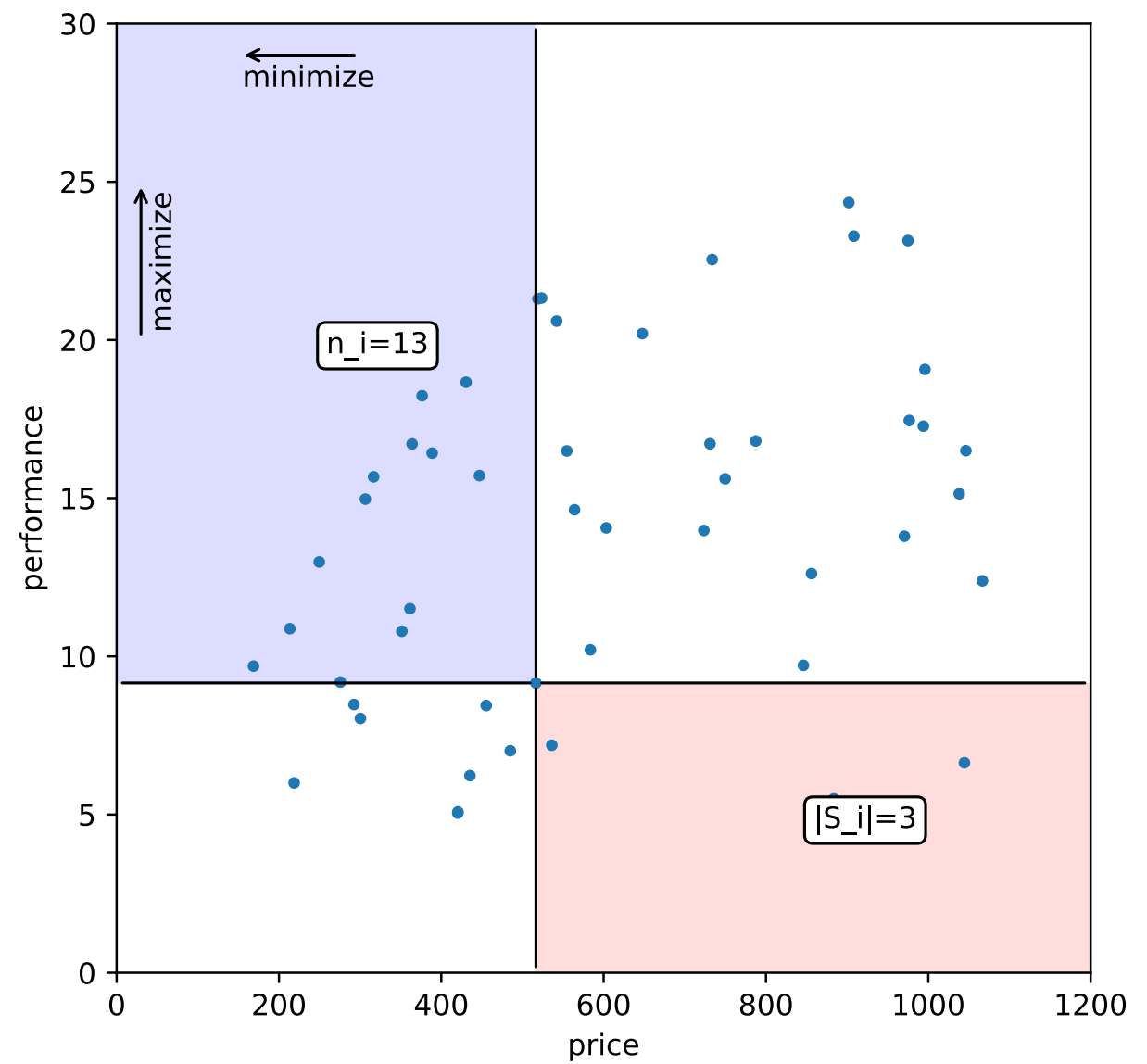
end if

end for

end for

return $\mathcal{N} = \{n_i\}, \mathcal{S} = \{S_i\}$

Complexity in $O(MN^2)$



How to find the fronts?

Input: $\mathcal{N}, \mathcal{S}$

Output: Set of fronts $\{\mathcal{F}_i\} \mid \cup_i \mathcal{F}_i = \mathcal{P}$ $\triangleright \mathcal{F}_0$ being the Pareto front, $\mathcal{F}_1$ the non dominated front once $\mathcal{F}_0$ has been removed, ...

$$i \leftarrow 0$$
$$\mathcal{F}_i \leftarrow \emptyset$$
for $n_j \in \mathcal{N}$ **do**

if $n_j == 0$ then

$$\bar{\mathcal{F}}_i \leftarrow \mathcal{F}_i \cup \{j\}$$

end if

end for

while $\mathcal{F}_i \neq \emptyset$ do
$$\Omega \leftarrow \emptyset$$
for p **in** $\mathcal{F}_i$ **do****for** q in $\mathcal{S}_p$ **do**
$$n_q \leftarrow n_q - 1$$

if $n_q == 0$ then

$$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{q\}$$

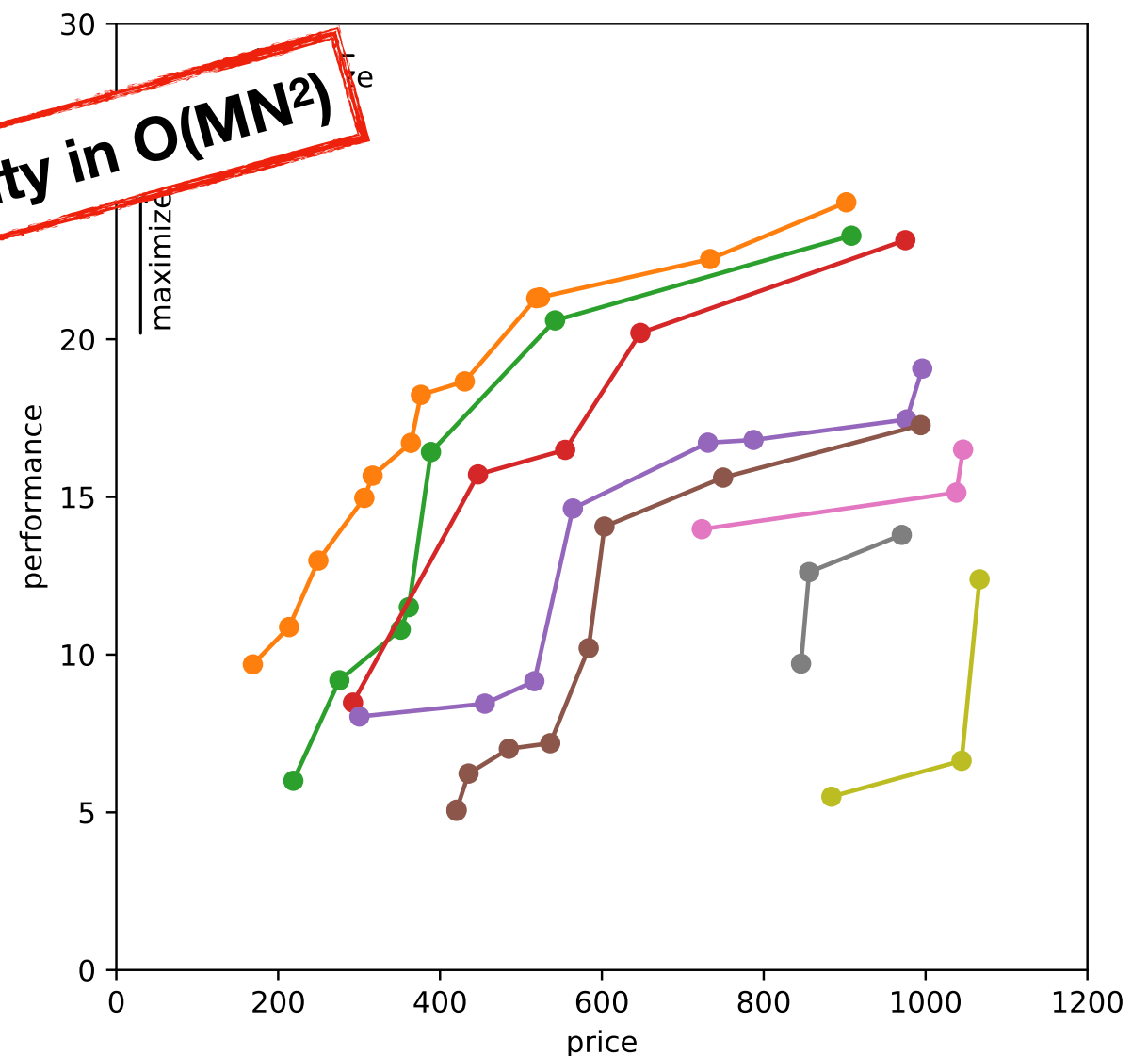
end if

end for

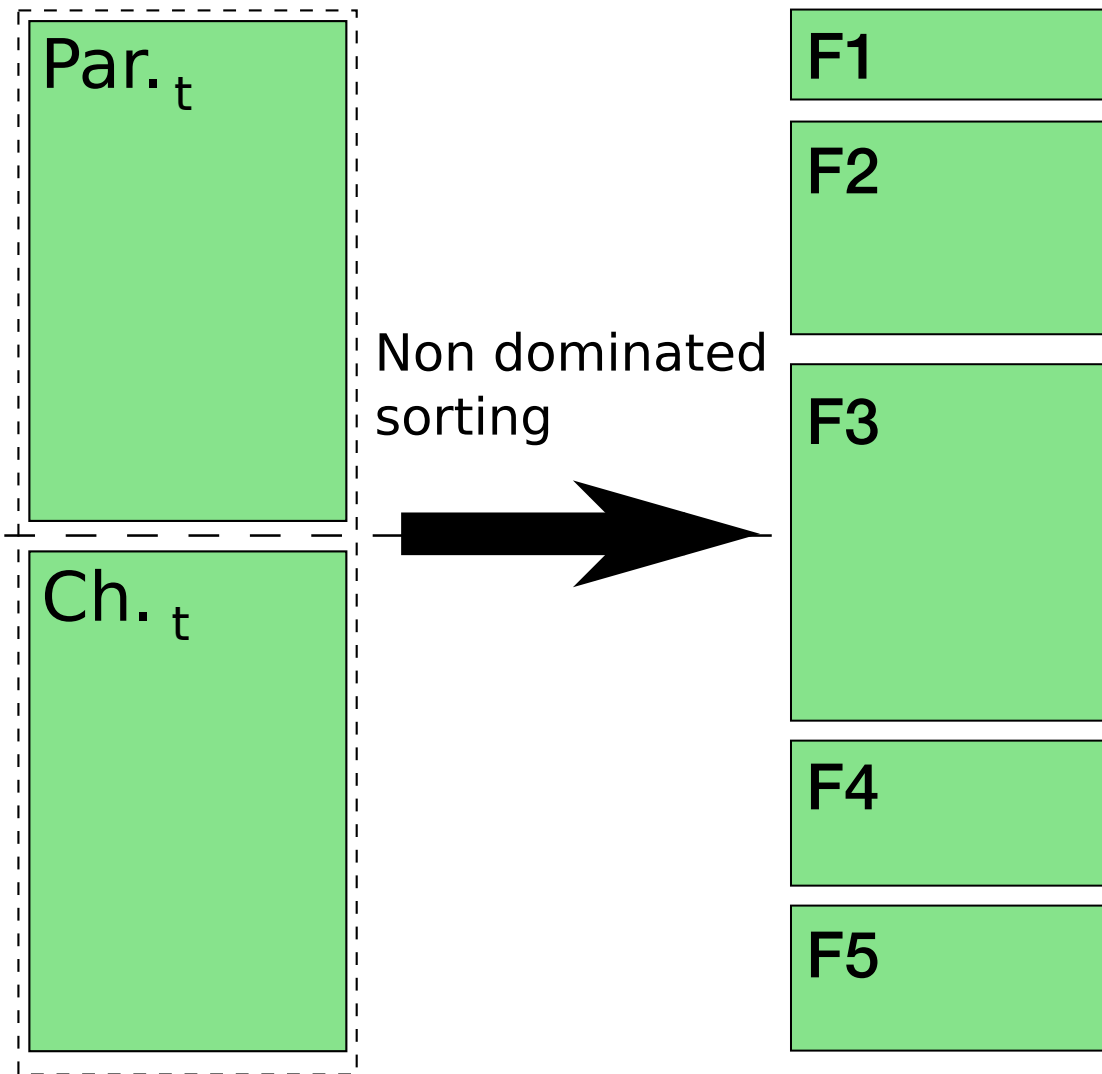
end for

$$i \leftarrow i + 1$$
$$\mathcal{F}_i \leftarrow \mathcal{Q}$$

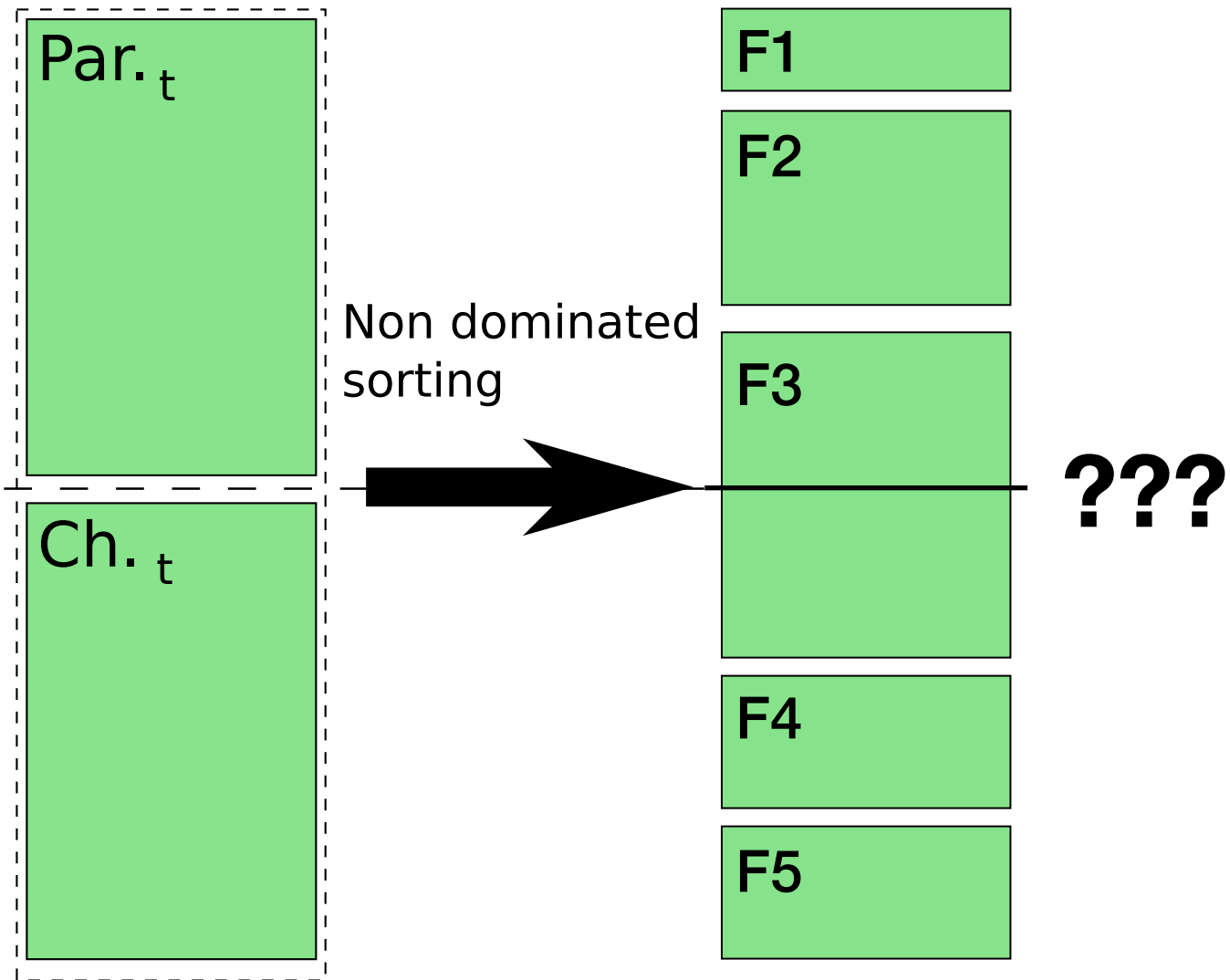
end while

return $\{\mathcal{F}_i\}$ 

NSGA-2



NSGA-2



**How to cut F3
while preserving diversity ?**

Diversity preservation: crowding distance

Input: $\mathcal{F} = \{f\}$, front of non-dominated solutions at a given depth, $f.i$ is the value of the i -th objective

Output: $f.distance$ for each f in $\mathcal{F}$

Parameters: number of objectives M , $\mathcal{F}$ front size N ,

$f_i^{max} \leftarrow \max(\mathcal{F}.i)$

$f_i^{min} \leftarrow \min(\mathcal{F}.i)$

for f in $\mathcal{F}$ **do**

$f.distance \leftarrow 0$

end for

for i in $0 \dots M$ **do**

 # Sort $\mathcal{F}$ according to objective i :

$\mathcal{F} \leftarrow \text{sort}(\mathcal{F}, i)$

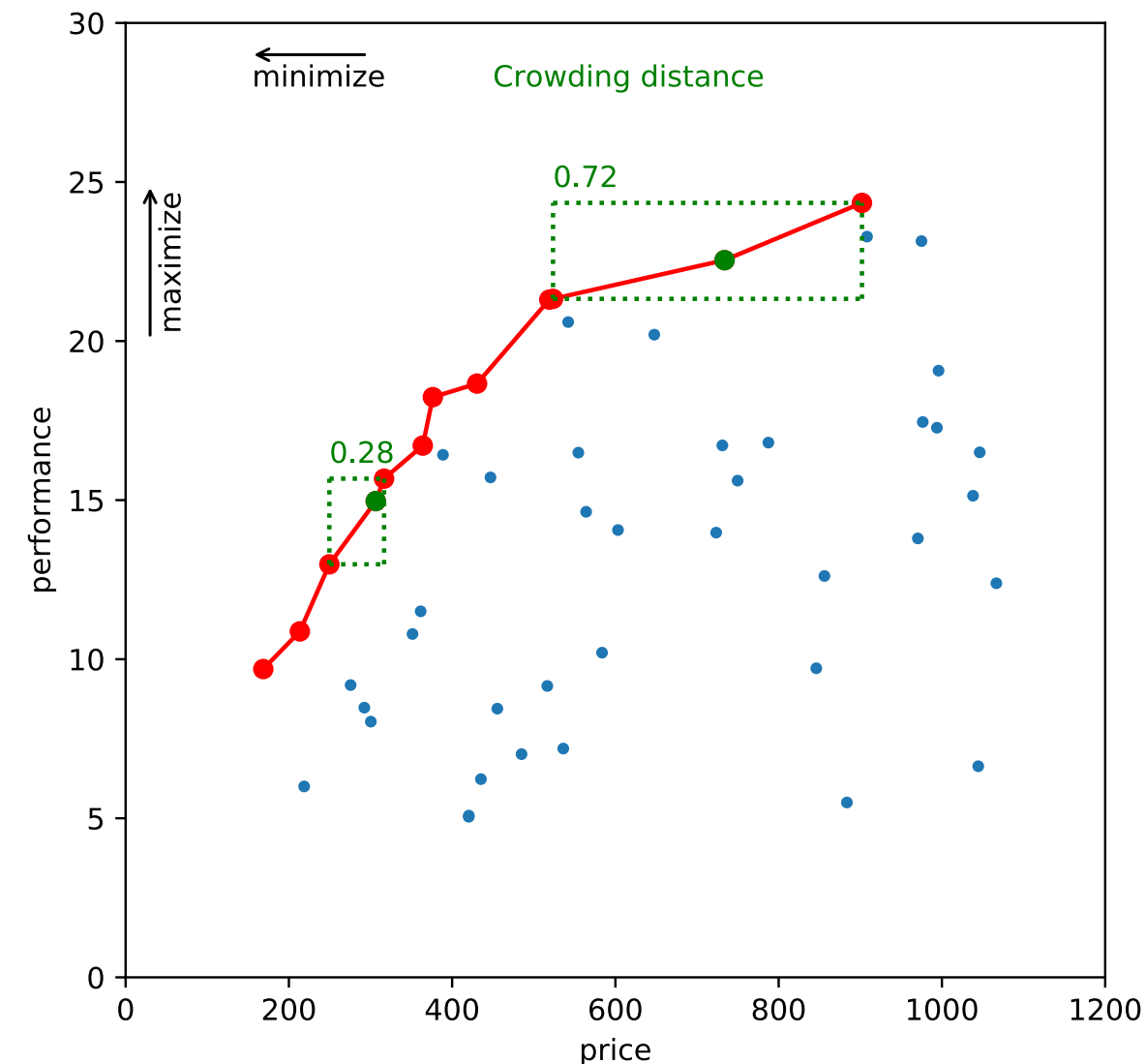
$\mathcal{F}[0] \leftarrow \mathcal{F}[n-1] \leftarrow \infty$

for j in $2 \dots n-2$ **do**

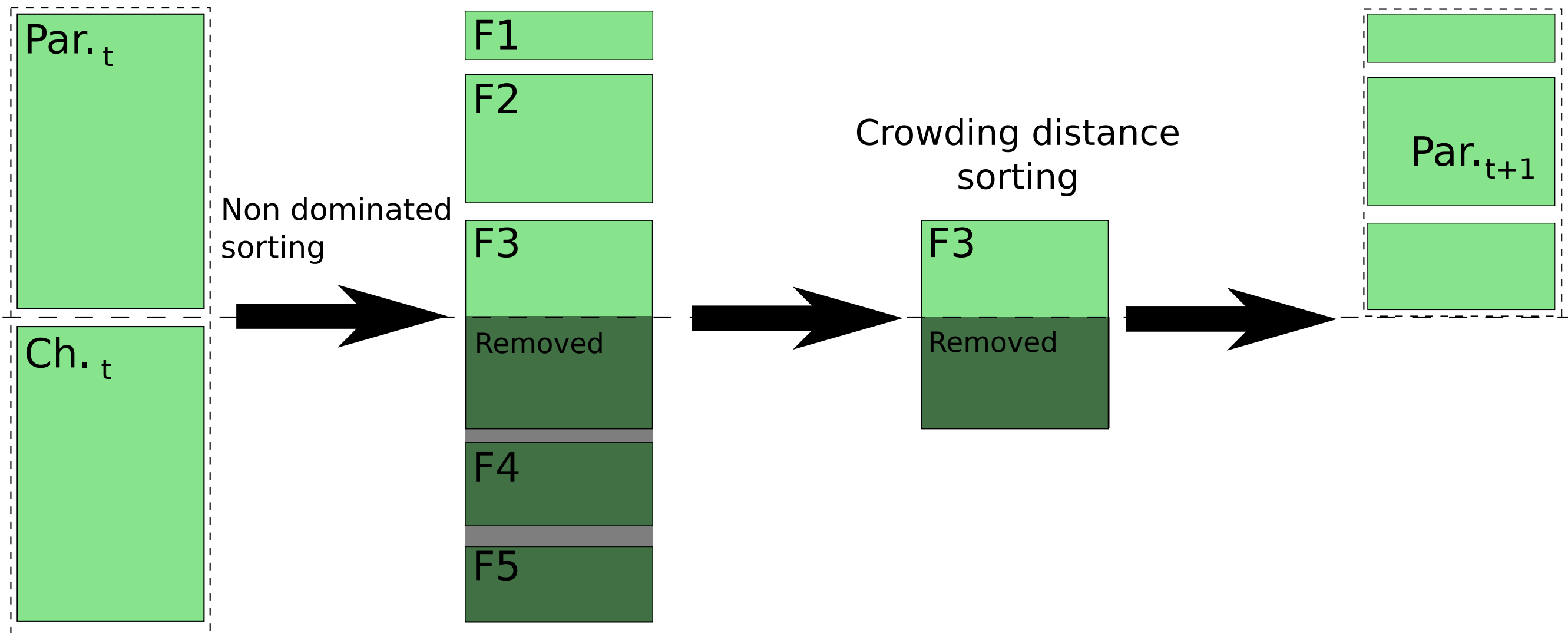
$\mathcal{F}[j].distance \leftarrow \mathcal{F}[j].distance + \frac{\mathcal{F}[j+1].i - \mathcal{F}[j-1].i}{f_i^{max} - f_i^{min}}$

end for

end for



NSGA-2 selection process



NSGA-2: others

- Parent selection scheme: TournamentDCD, tournament based on domination or (if non dominated) crowding distance
- Typical mutation operators:
 - Binary: Bitwise mutation $(0010110100) \longrightarrow (00101101\textcolor{red}{1}0)$
 - Real: Polynomial mutation (see next slide)
- Typical crossover operators:
 - Binary: single point crossover
$$\begin{array}{ccc} (\textcolor{red}{0}0101|\textcolor{red}{1}0110) & \longrightarrow & (\textcolor{red}{0}0101|\textcolor{green}{0}1101) \\ (\textcolor{green}{1}0100|\textcolor{green}{0}1101) & & (\textcolor{green}{1}0100|\textcolor{red}{1}0110) \end{array}$$
 - Real: simulated binary crossover (see next slide)

NSGA-2: algorithm

Input: population size N , number of generations G

Output: non-dominated set

$P_0 \leftarrow \text{random_init}(N)$

$g \leftarrow 0$

for $p \in P_0$ **do**

$p.\text{fitness} \leftarrow \text{evaluate}(p)$

end for

while $g < G$ **do**

$O \leftarrow \emptyset$

while $\text{len}(O) < N$ **do**

$\text{ind}_1, \text{ind}_2 \leftarrow \text{randomSelection}(P_g)$

$\text{offspring} \leftarrow \text{BinaryDCDTournament}(\text{ind}_1, \text{ind}_2)$

$O \leftarrow O \cup \{\text{offspring}\}$

end while

for $\text{off}_1, \text{off}_2 \in O$ **do**

if $\text{random.random} < CXPB$ **then**

$\text{off}_1, \text{off}_2 \leftarrow \text{crossover}(\text{off}_1, \text{off}_2)$

end if

end for

for $\text{off} \in O$ **do**

if $\text{random.random} < MUTPB$ **then**

$\text{off} \leftarrow \text{mutate}(\text{off})$

end if

end for

for $\text{off} \in O$ **do**

$\text{off.fitness} \leftarrow \text{evaluate}(\text{off})$

end for

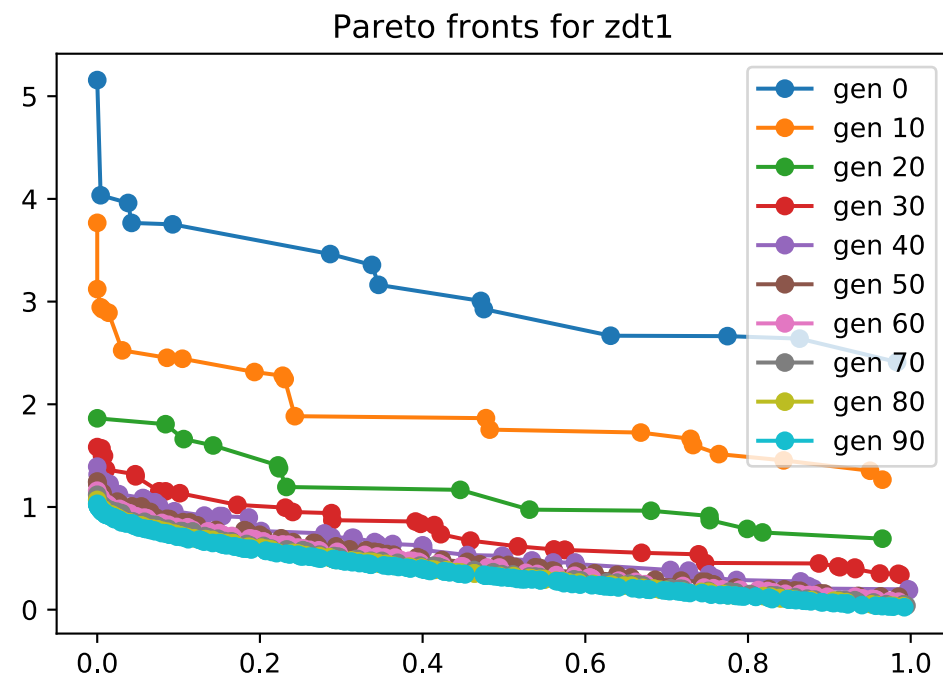
$P_{g+1} \leftarrow \text{FrontSelection}(P_g \cup O)$

end while

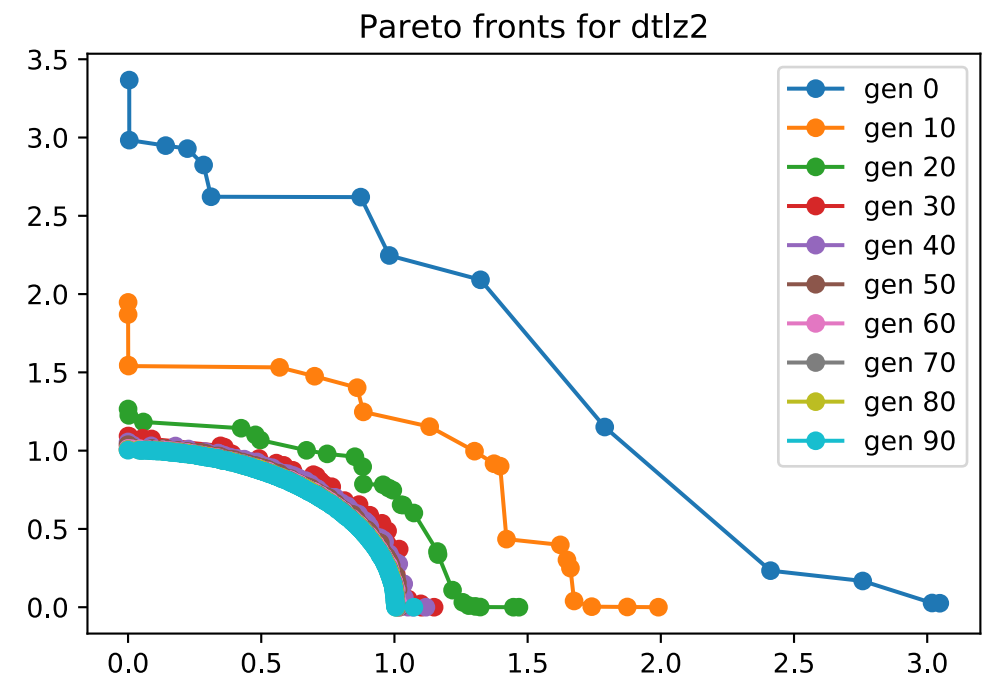
Binary Tournament
based on (1) Dominance
and, if none dominates
(2) Crowding distance

Examples

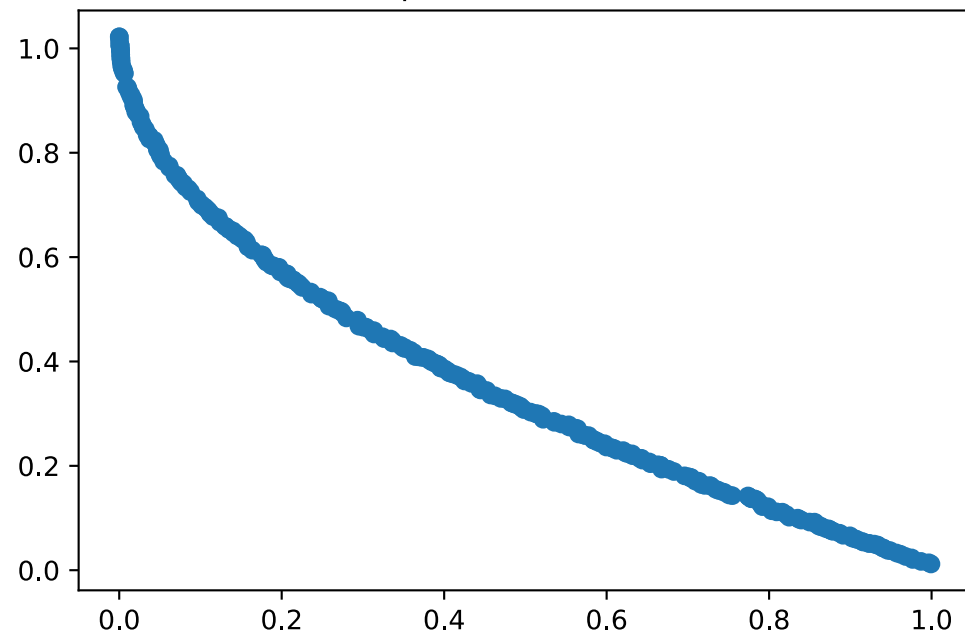
ZDT1



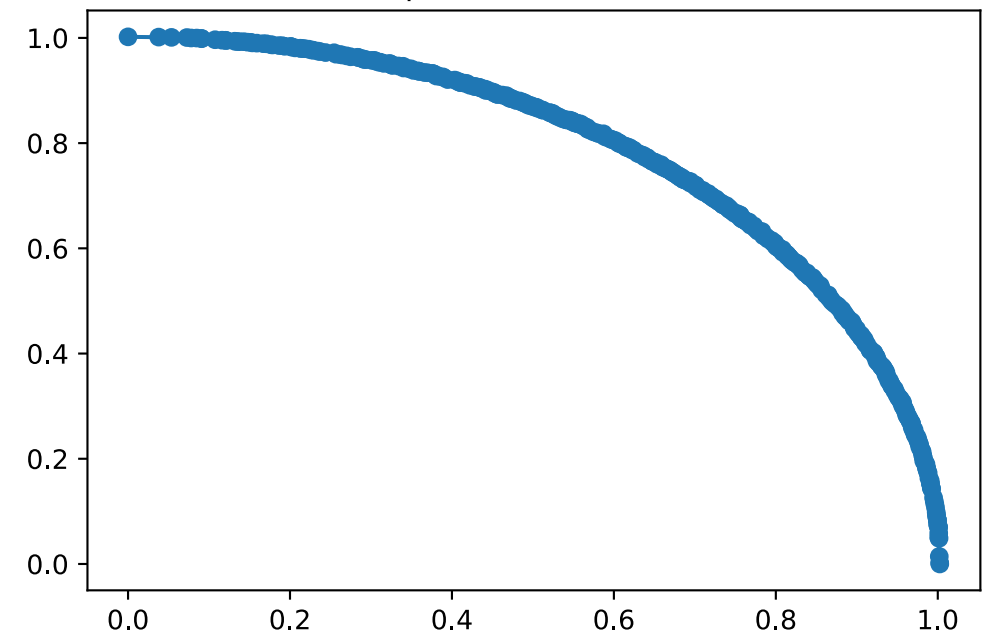
DTLZ2



Last pareto front for zdt1



Last pareto front for dtlz2



Polynomial mutation

Input: genotype g , of size n , g_i being the i -th gene value, g_{min}, g_{max} the min and max values of g

Parameters: probability of mutation $indpb$, mutation parameter η

for g_i in g **do**

if $random(0, 1) < indpb$ **then**

$$d1 \leftarrow \frac{g_i - g_{min}}{g_{max} - g_{min}}$$

$$d2 \leftarrow \frac{g_{max} - g_i}{g_{max} - g_{min}}$$

$$mut_{pow} \leftarrow \frac{1}{\eta + 1}$$

$$rand \leftarrow random(0, 1)$$

if $rand < 0.5$ **then**

$$xy \leftarrow 1 - d1$$

$$val \leftarrow 2 * rand + (1 - 2 * rand) * xy^{\eta+1}$$

$$dq = val^{mut_{pow}} - 1$$

else

$$xy \leftarrow 1 - d2$$

$$val \leftarrow 2 * (1 - rand) + 2 * (rand - 0.5) * xy^{\eta+1}$$

$$dq = 1 - val^{mut_{pow}}$$

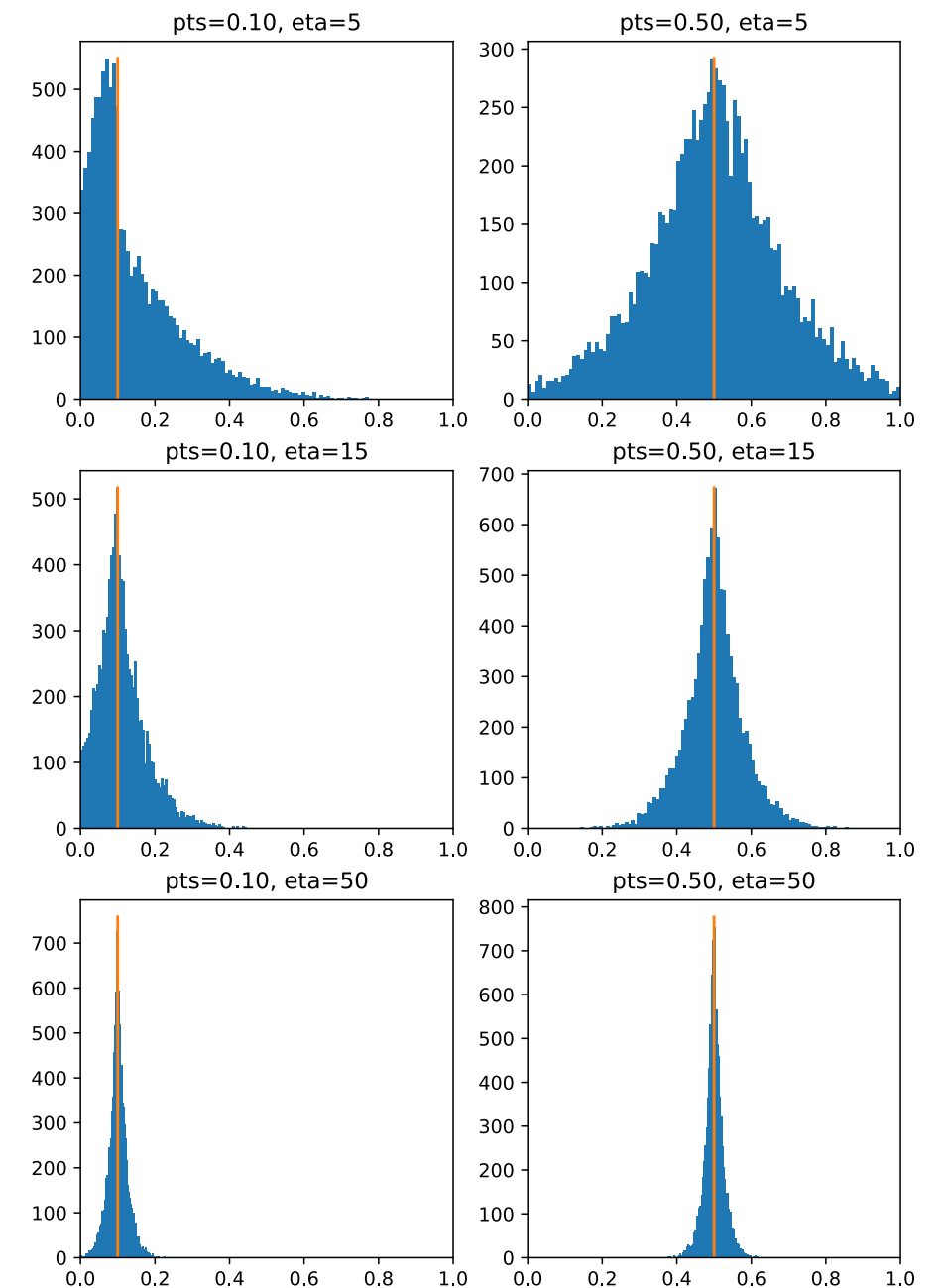
end if

$$g_i \leftarrow g_i + dq * (g_{max} - g_{min})$$

$$g_i \leftarrow \min(\max(g_i, g_{min}), g_{max})$$

end if

end for



Simulated binary crossover

- Properties:
 - Keep the same average value
 - Spread factor $\beta \approx 1$

Parents: p_1, p_2 (suppose $p_2 > p_1$)

$$\bar{x} = \frac{1}{2}(p_1 + p_2)$$

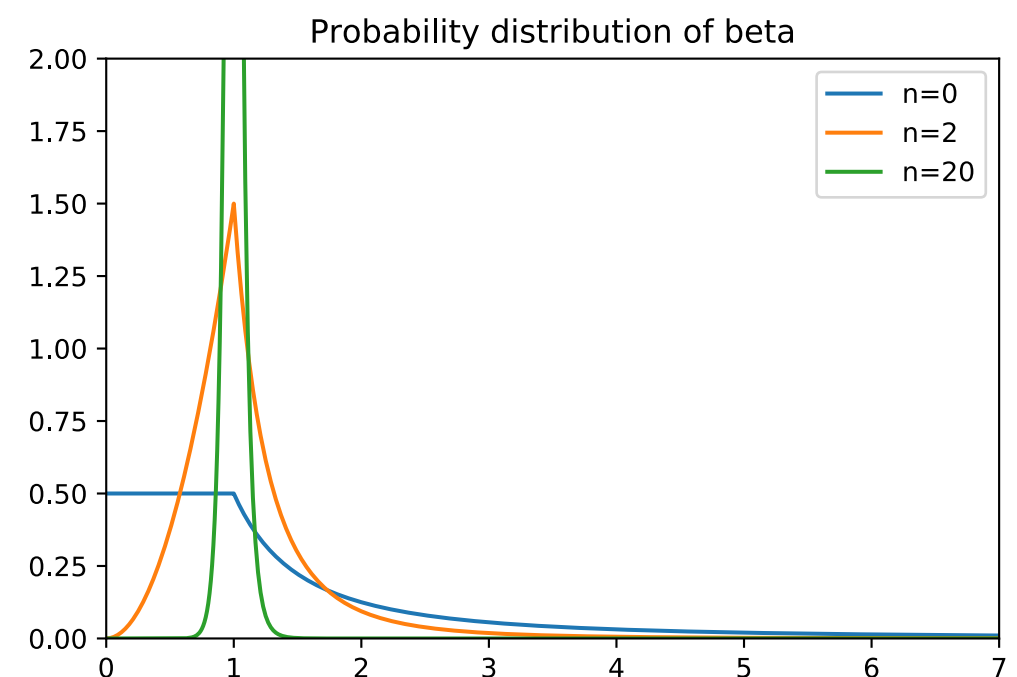
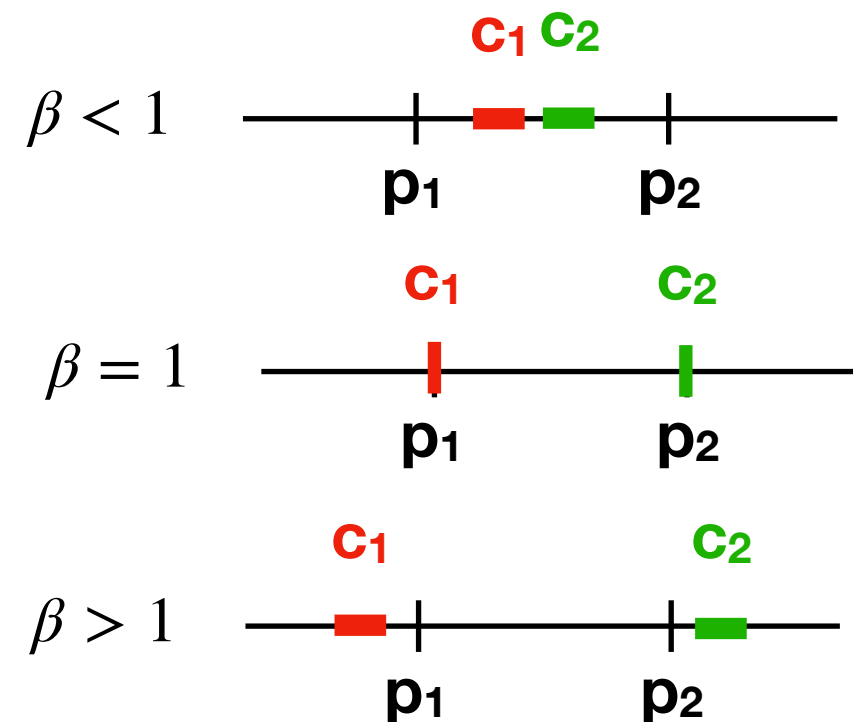
Offspring:

$$c_1 = \bar{x} - \frac{1}{2}\beta(p_2 - p_1)$$

$$c_2 = \bar{x} + \frac{1}{2}\beta(p_2 - p_1)$$

with β following the distribution:

$$\begin{cases} c(\beta) = 0.5(n+1)\beta^n, \beta \leq 1 \\ c(\beta) = 0.5(n+1)\frac{1}{\beta^{n+2}}, \beta > 1 \end{cases}$$



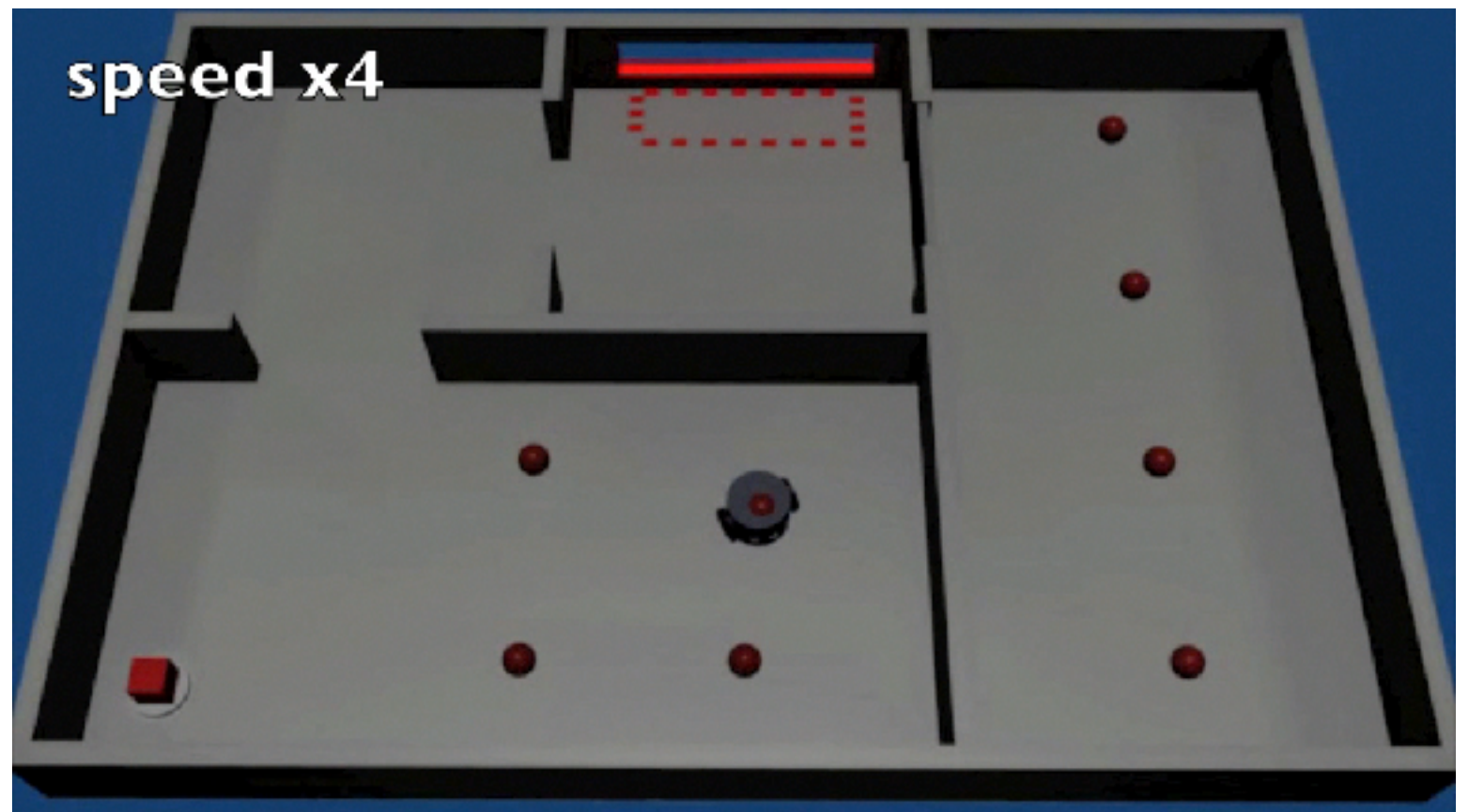
Collect ball experiment

Fitness objectives:

1. nb_{ball}
2. Behavior diversity

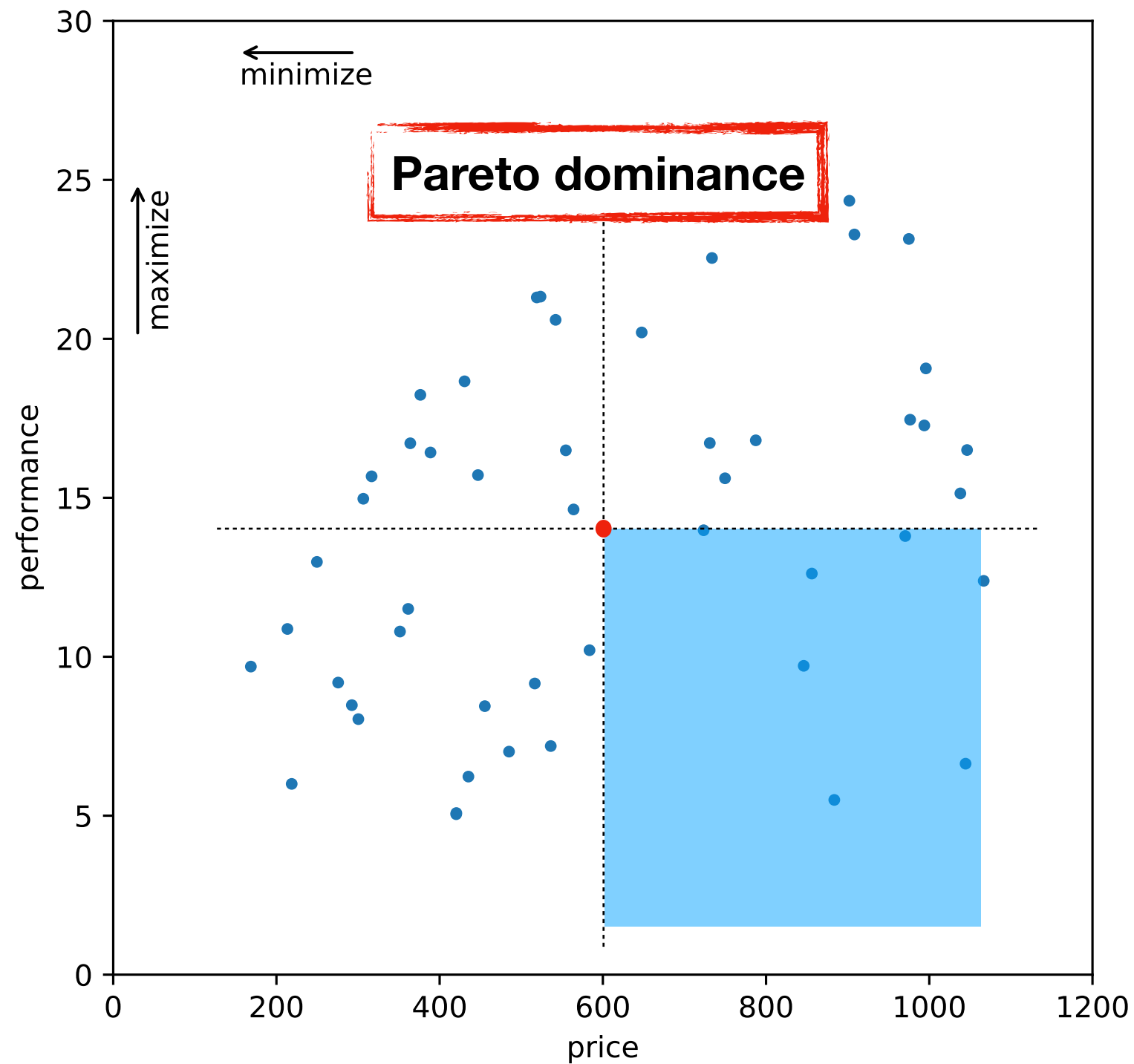
Multi-objective EA:
NSGA-II

Neuroevolution

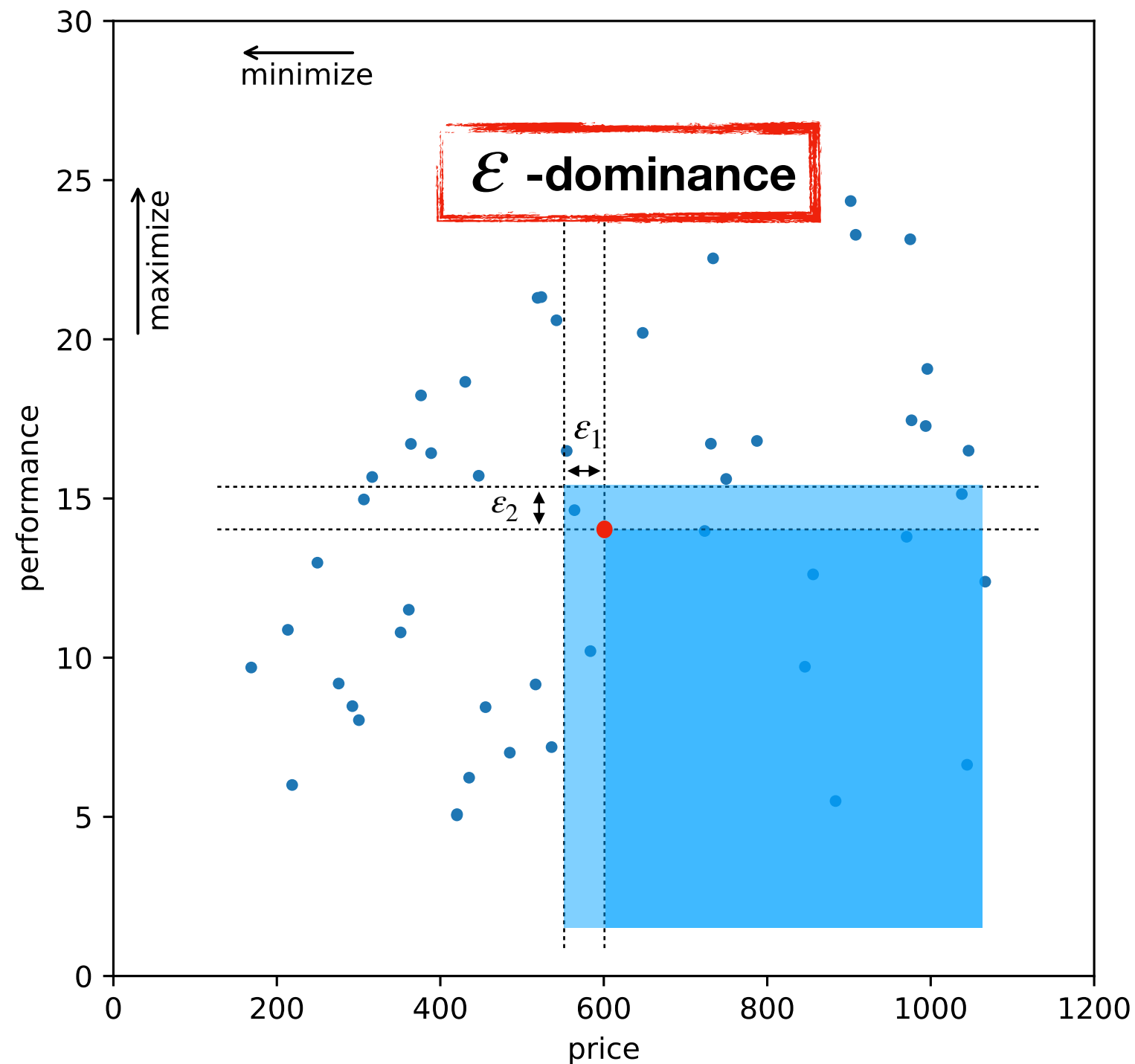


<https://github.com/doncieux/collectball>

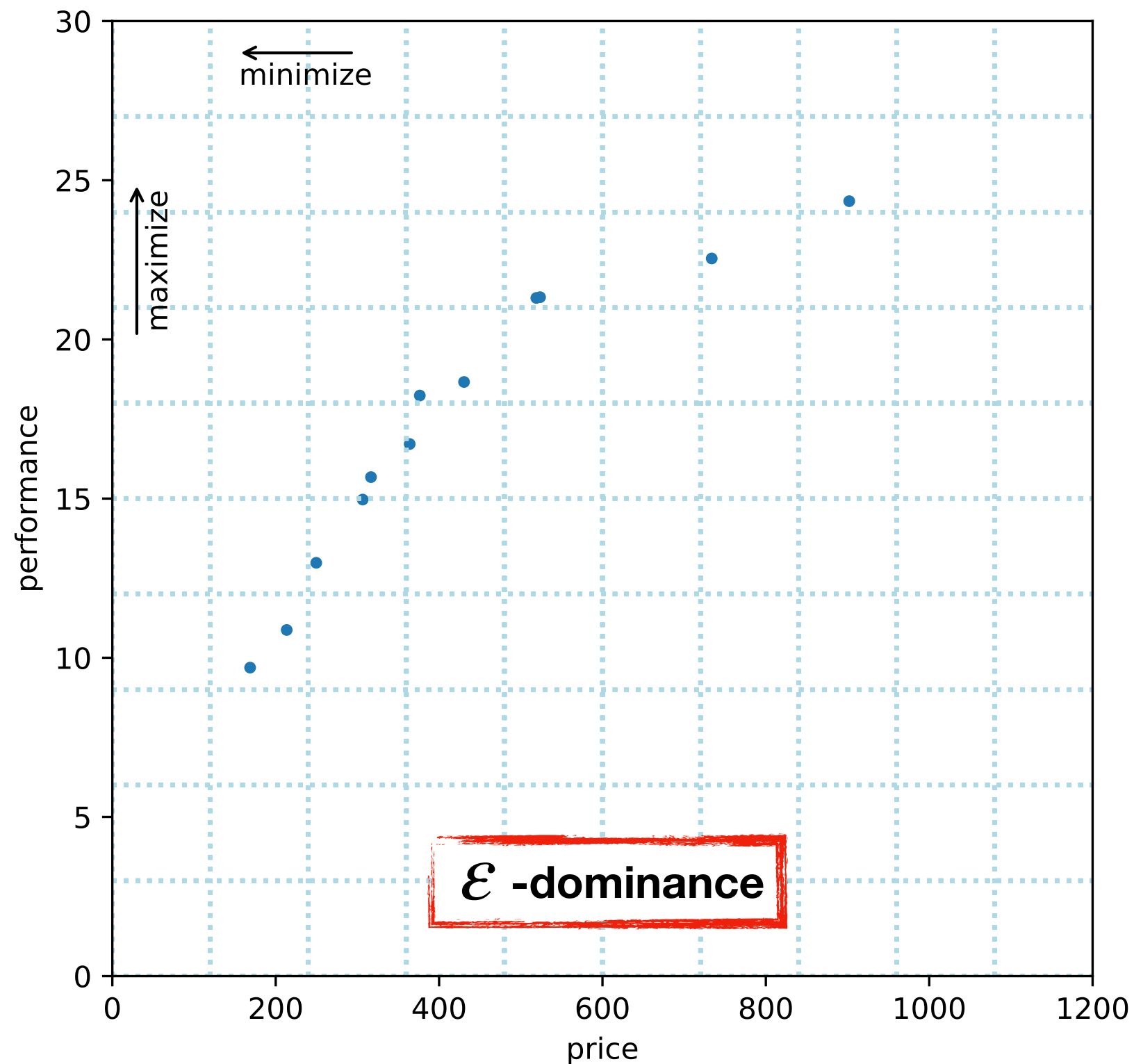
Different dominance relations



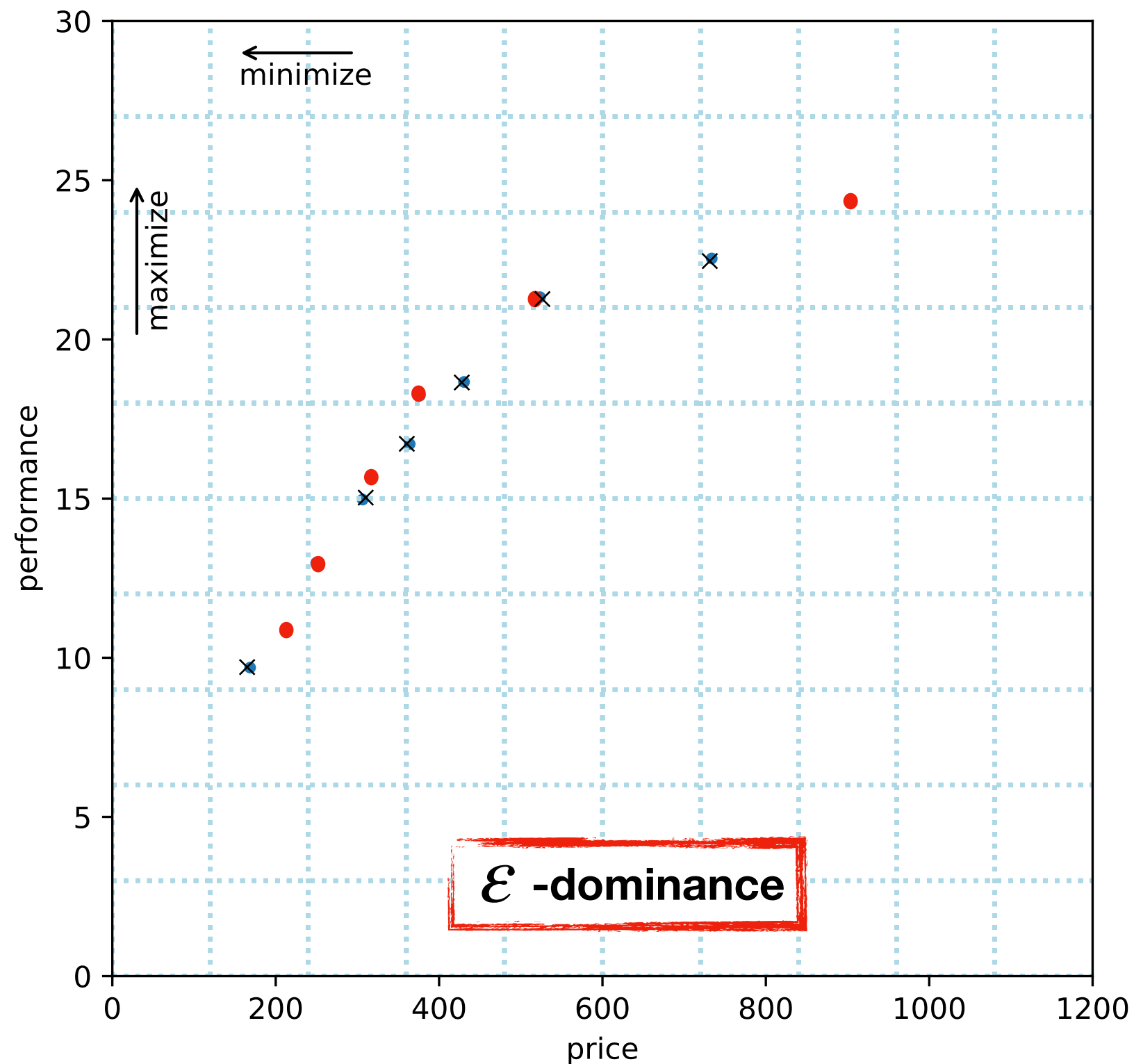
Different dominance relations



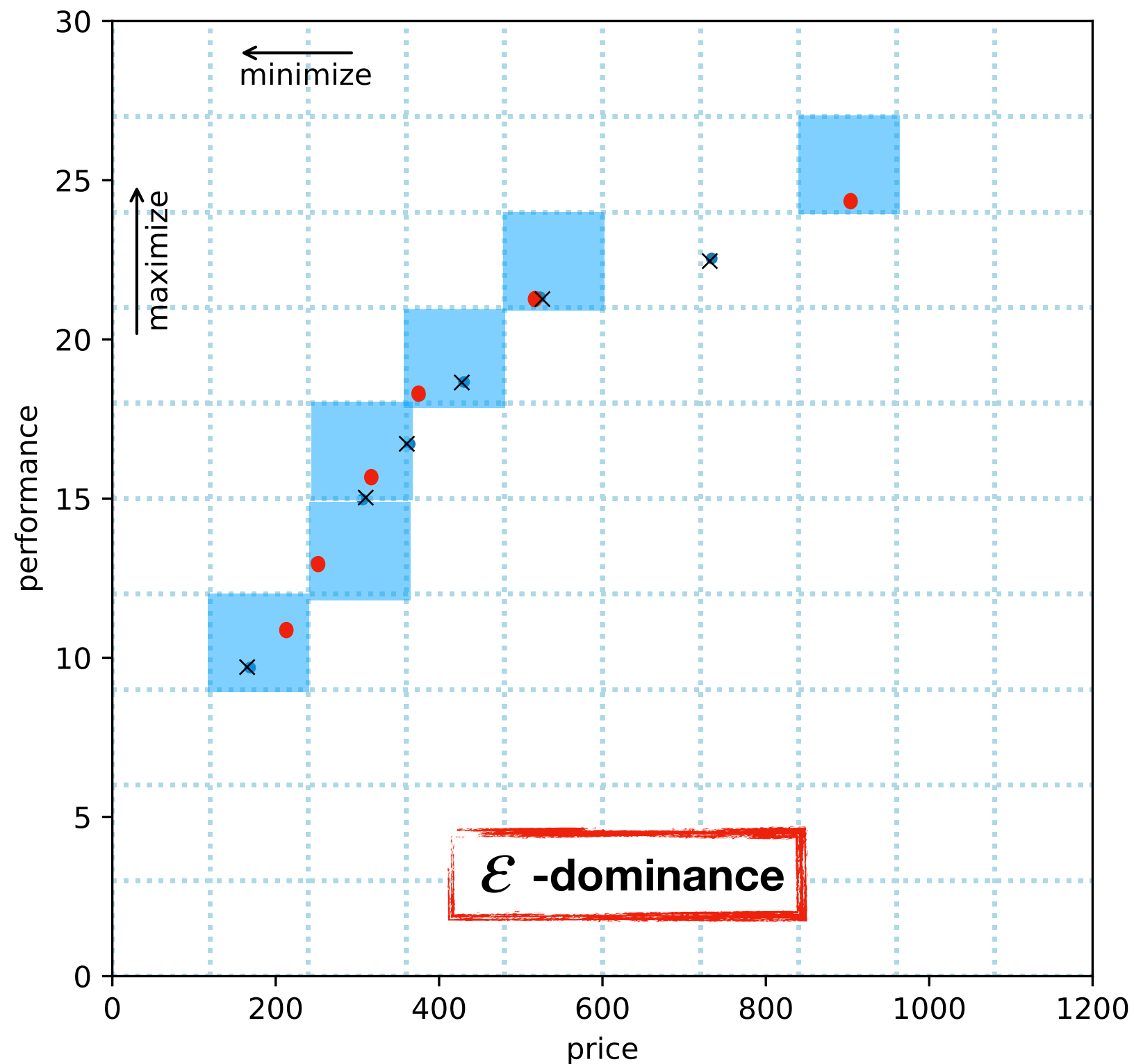
Different dominance relations



Different dominance relations

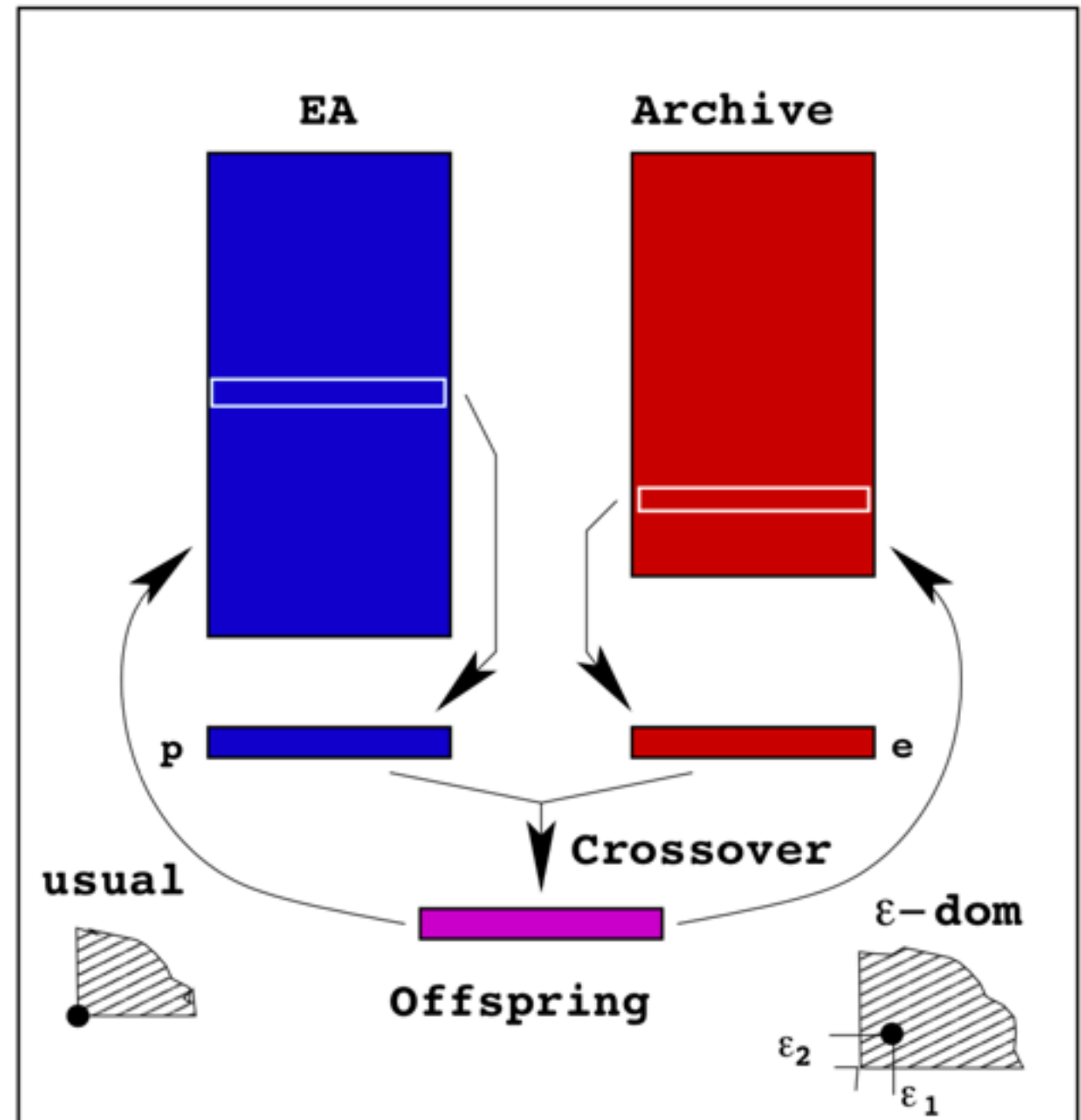


Different dominance relations



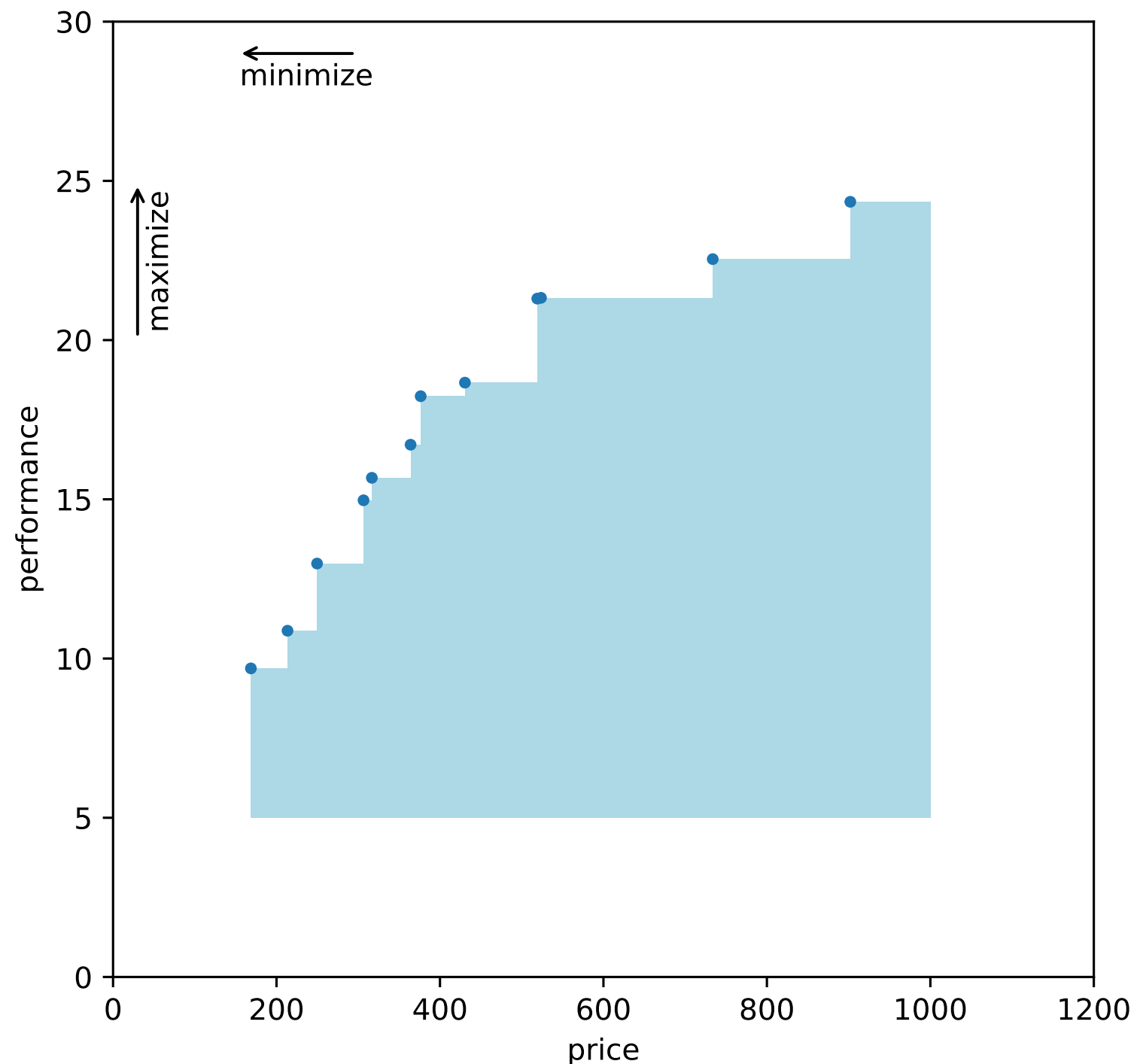
Epsilon-MOEA

- Steady-state EA
- Population + Archive
- Selection of parents:
 - Parent 1: Tournament from pop (pareto dom.)
 - Parent 2: random from archive
- Offspring inclusion:
 - to archive: epsilon-dominance
 - to pop: pareto dominance



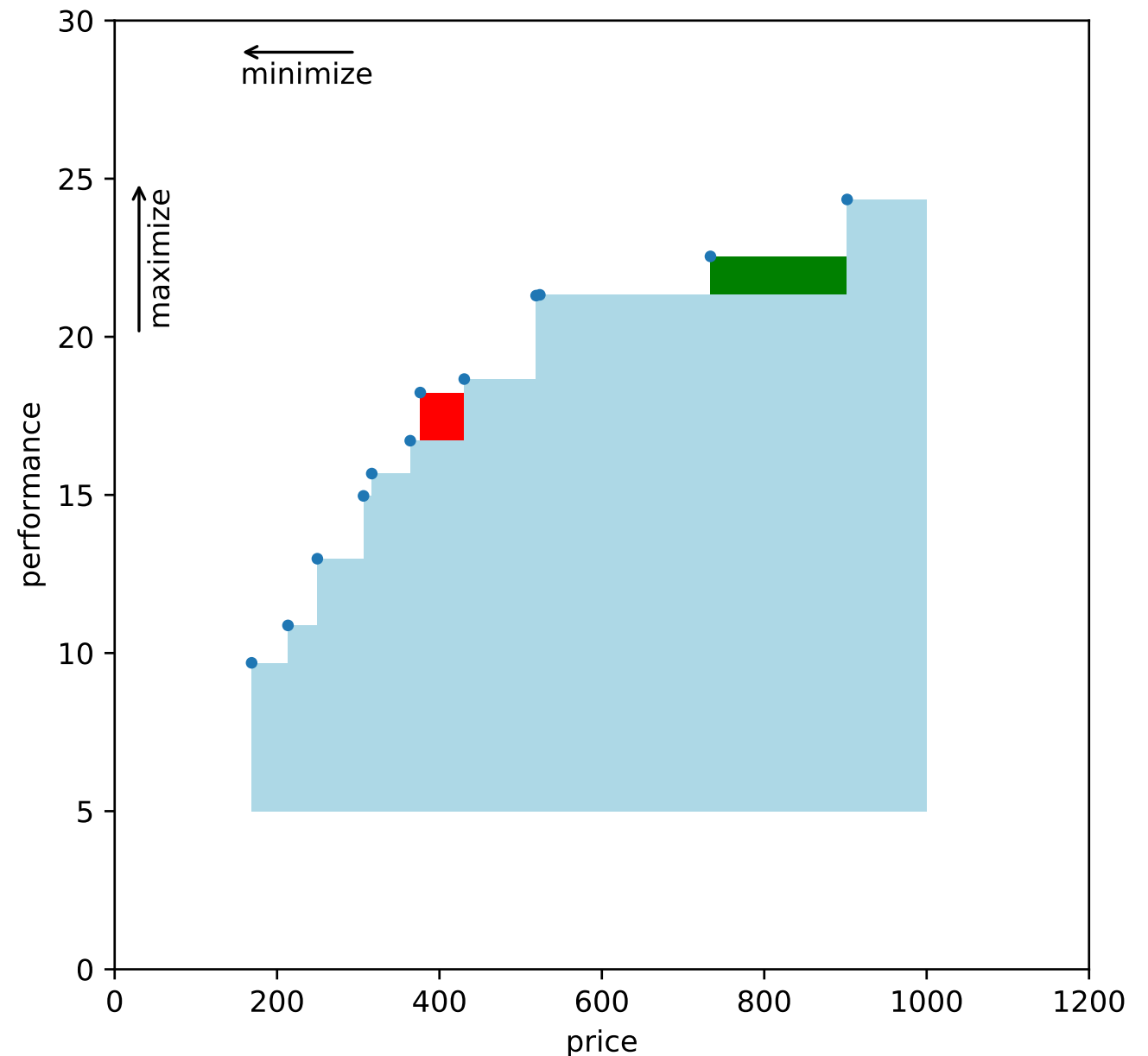
Indicator-based selection

- Keep multiple objectives, but determines a (unary) quality-based indicator.
- Example: **Hypervolume**
- Some interesting properties:
 - allows to compare set of points with a unique scalar value
 - ranking of solutions invariant to linear scaling



SMS-EMOA

- Steady state algorithm: 1 individual at a time...
- NSGA-II non dominated sort
- The individual from the last front with the smallest contribution to the hyper volume is removed



Overview



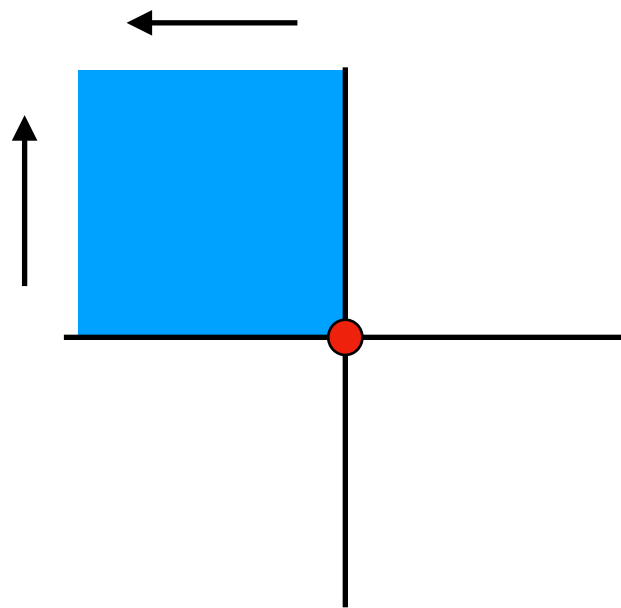
- Introduction
- Back to single-objective
- Multi-objective optimization
- Many-objective optimization

From multi- to many-

- Multi-objective optimisation: 2 to 3 objectives
- Many-objective optimisation: >3 objectives

Challenges of many-objective optimisation

- 1. Limitation of Pareto dominance relation

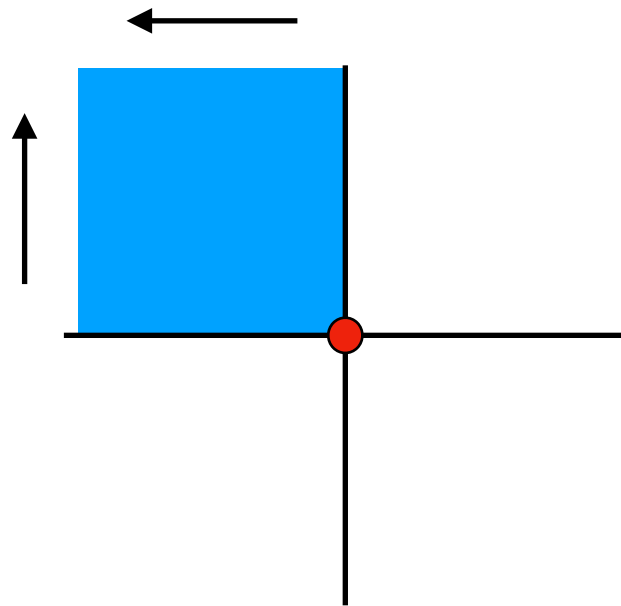


Size of the better region with Pareto dominance relation

2 objectives

Challenges of many-objective optimisation

- 1. Limitation of Pareto dominance relation

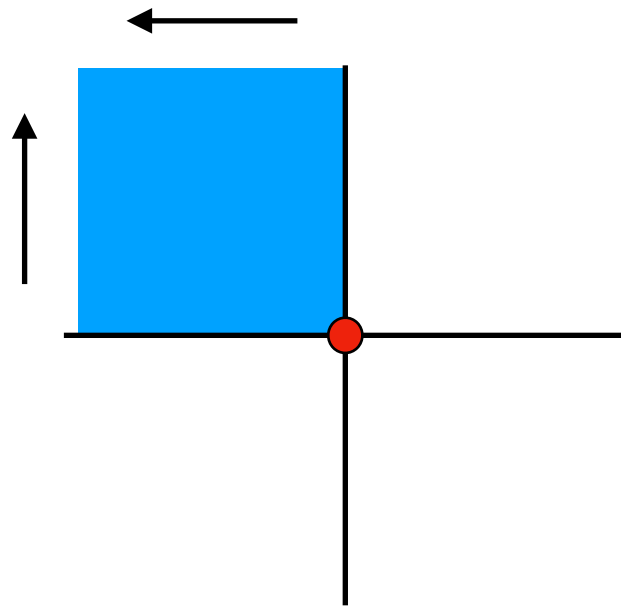


Size of the better region with Pareto dominance relation

2 objectives	1/4	25 %
3 objectives		

Challenges of many-objective optimisation

- 1. Limitation of Pareto dominance relation

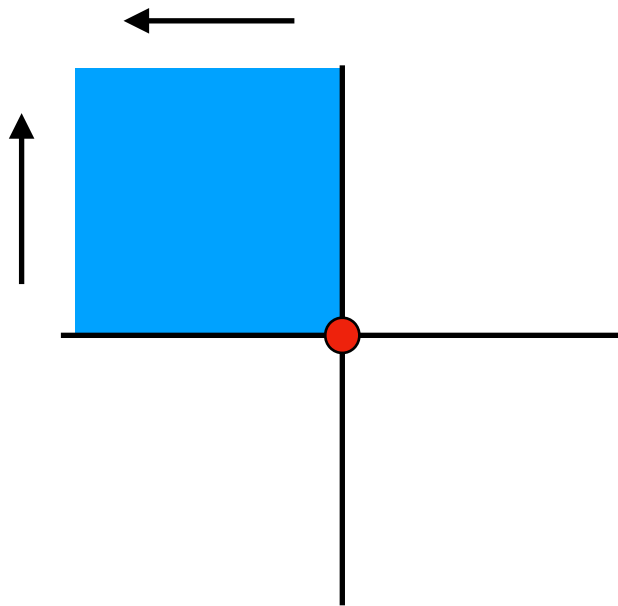


Size of the better region with Pareto dominance relation

2 objectives	$1/4$	25 %
3 objectives	$1/8$	13 %
4 objectives		

Challenges of many-objective optimisation

- 1. Limitation of Pareto dominance relation



Size of the better region with Pareto dominance relation		
2 objectives	$1/4$	25 %
3 objectives	$1/8$	13 %
4 objectives	$1/16$	6 %
5 objectives	$1/32$	3 %
10 objectives	$1/1024$	0,1 %
15 objectives	$1/32768$	0,003%
20 objectives	$1/1048576$	0,0001 %

Challenges of many-objective optimisation

- 2. Difficulty to get a good sampling of the Pareto front: too many points are needed !
 - Let's use 5 points to represent the Pareto front in 2D
 - With the same precision, how many points in 3D ?

Challenges of many-objective optimisation

- 2. Difficulty to get a good sampling of the Pareto front: too many points are needed !
 - Let's use 5 points to represent the Pareto front in 2D
 - In 3D: 25 points
 - In 4D: ... ?

Challenges of many-objective optimisation

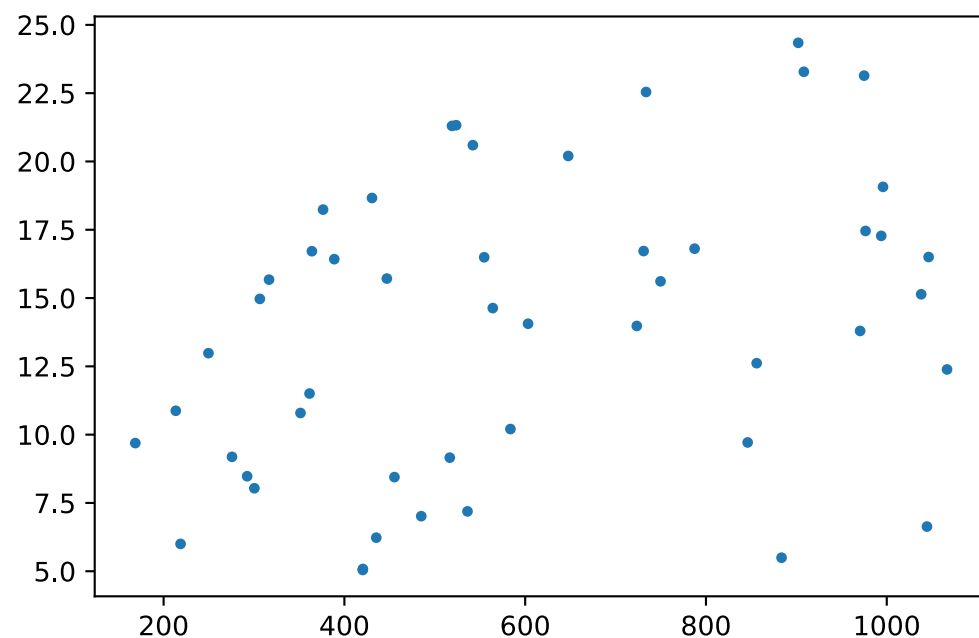
- 2. Difficulty to get a good sampling of the Pareto front: too many points are needed !
 - Let's use 5 points to represent the Pareto front in 2D
 - In 3D: 25 points
 - In 4D: 125 points
 - In kD: ...?

Challenges of many-objective optimisation

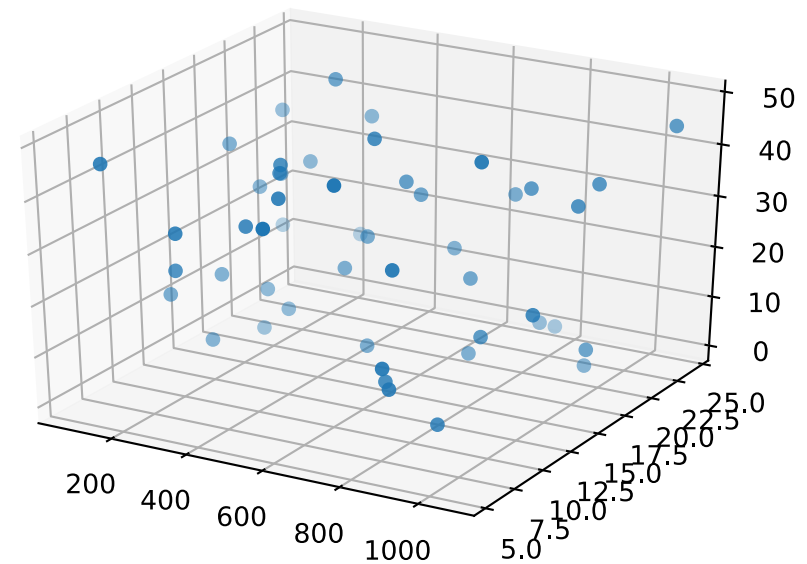
- 2. Difficulty to get a good sampling of the Pareto front: too many points are needed !
 - Let's use 5 points to represent the Pareto front in 2D
 - In 3D: 25 points
 - In 4D: 125 points
 - In kD: 5^{k-1}
 - In 10D: 10 million points !

The challenges of many-objective optimisation

- 3. Visualising solutions in dimension $m > 3$?



2D

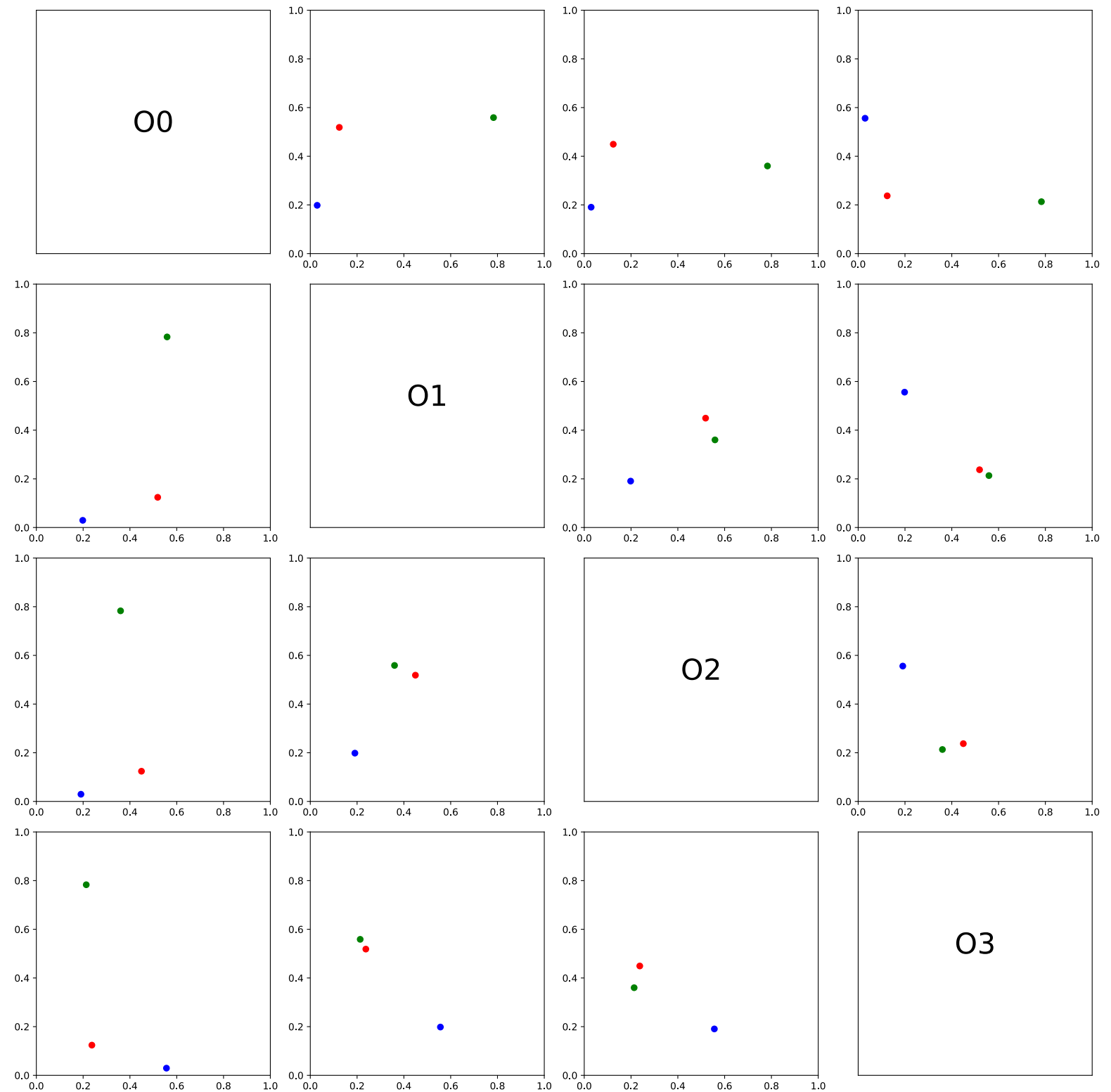


3D

>3D ?

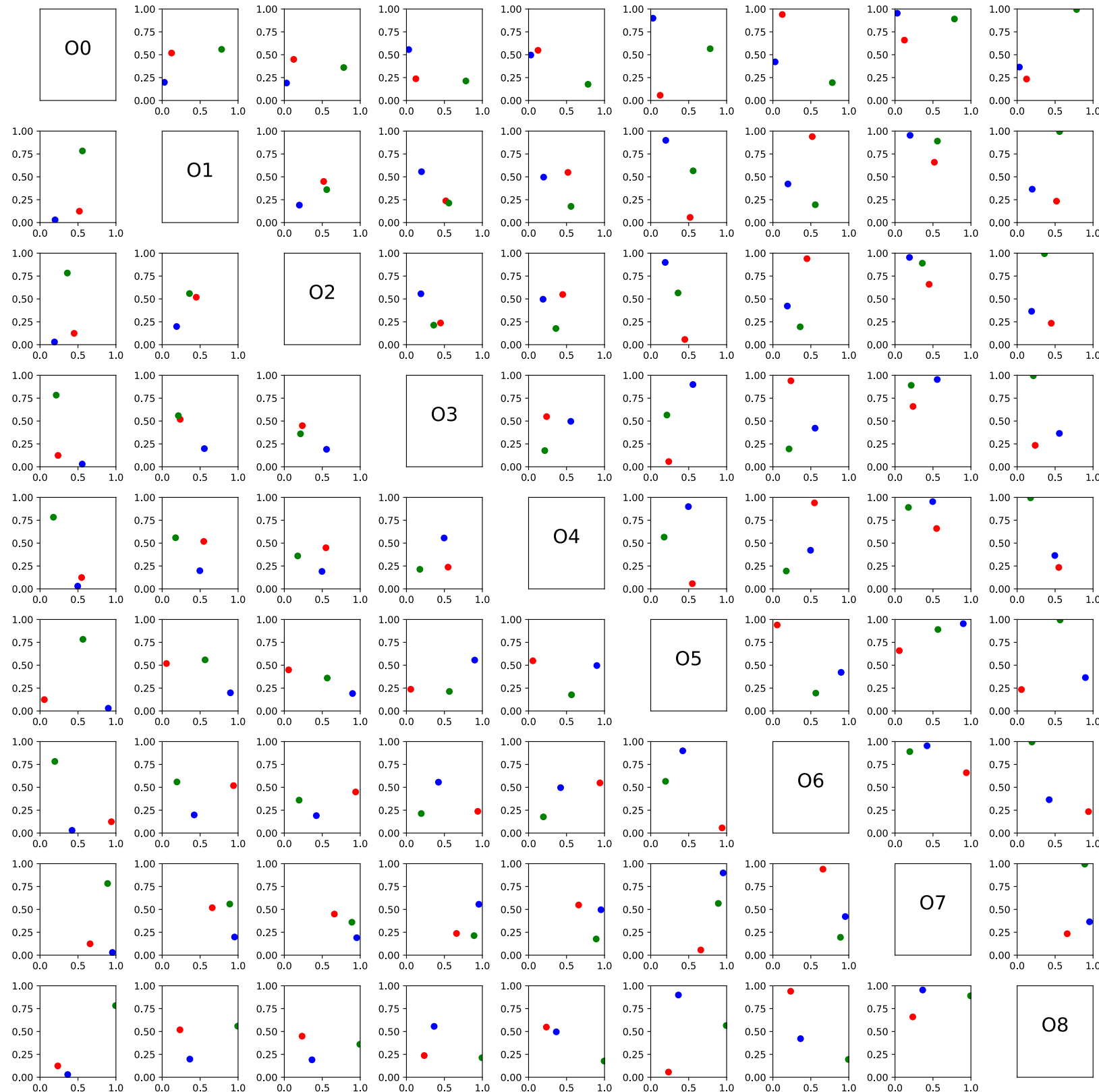
The challenges of many-objective optimisation

- 3. Visualising solutions in dimension $m > 3$? **Scatter plots**



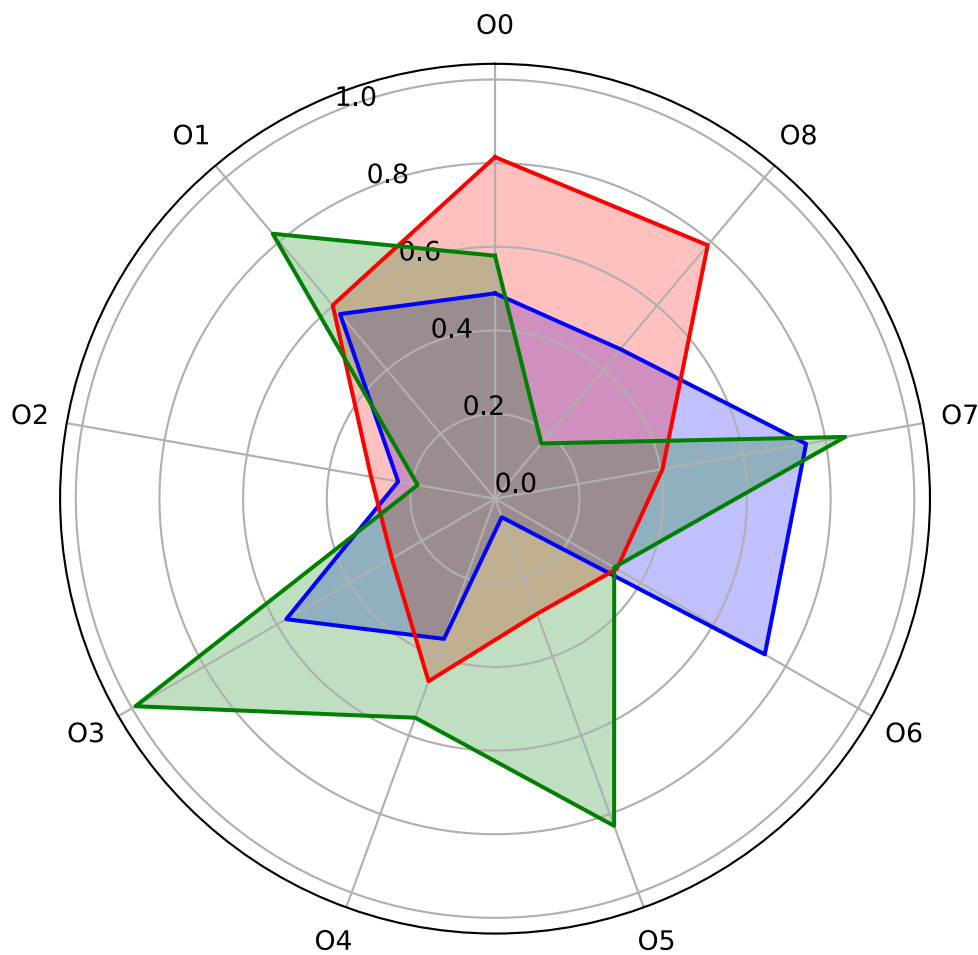
The challenges of many-objective optimisation

- 3. Visualising solutions in dimension $m > 3$? **Scatter plots**

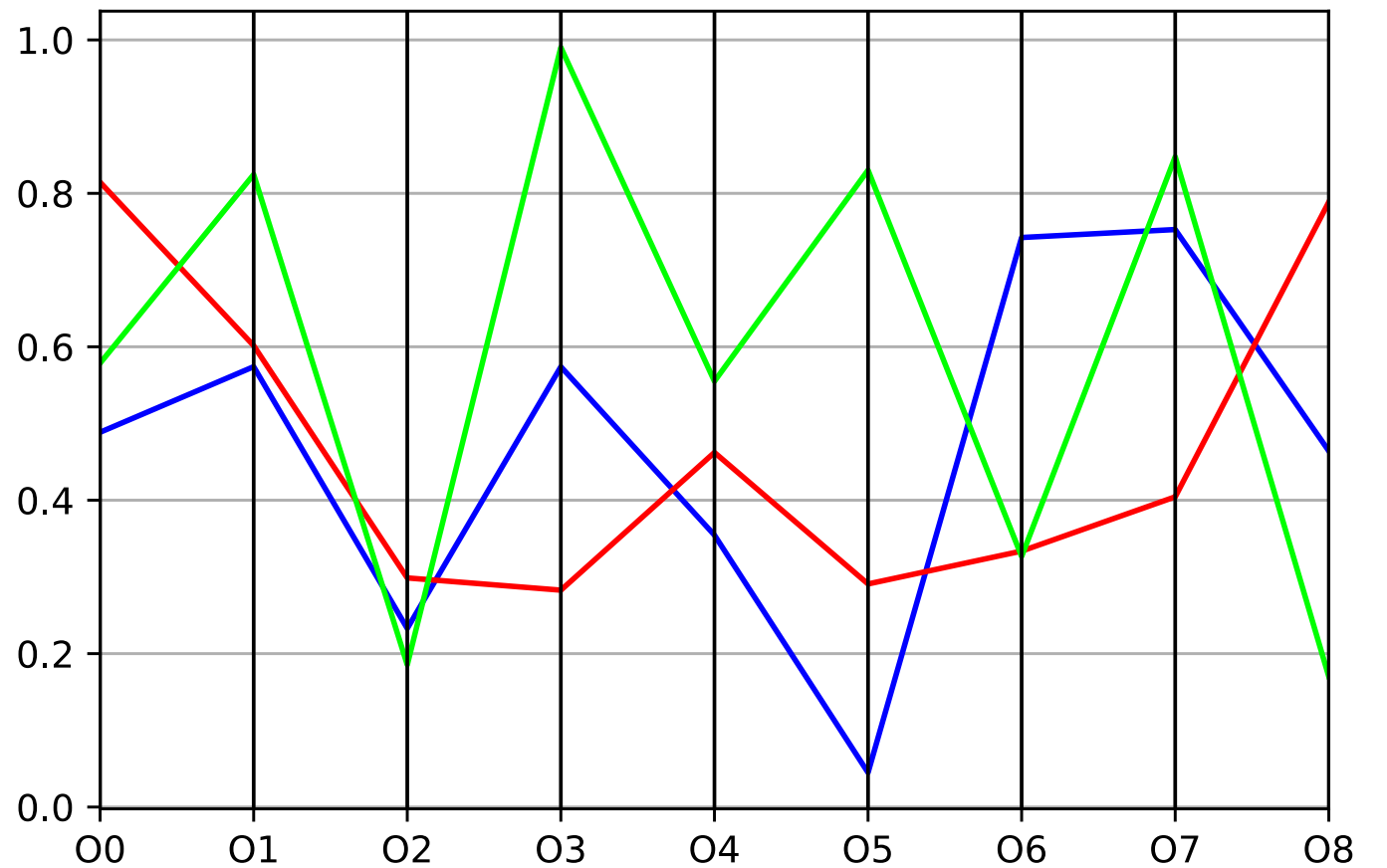


The challenges of many-objective optimisation

- 3. Visualising solutions in dimension $m > 3$?



Radar chart



Parallel plot

Solving many-objective problems

- Can't rely on Pareto dominance relation
 - Hard to keep a good sampling of the Pareto front
- ➡ How to proceed ?

Solving many-objective problems

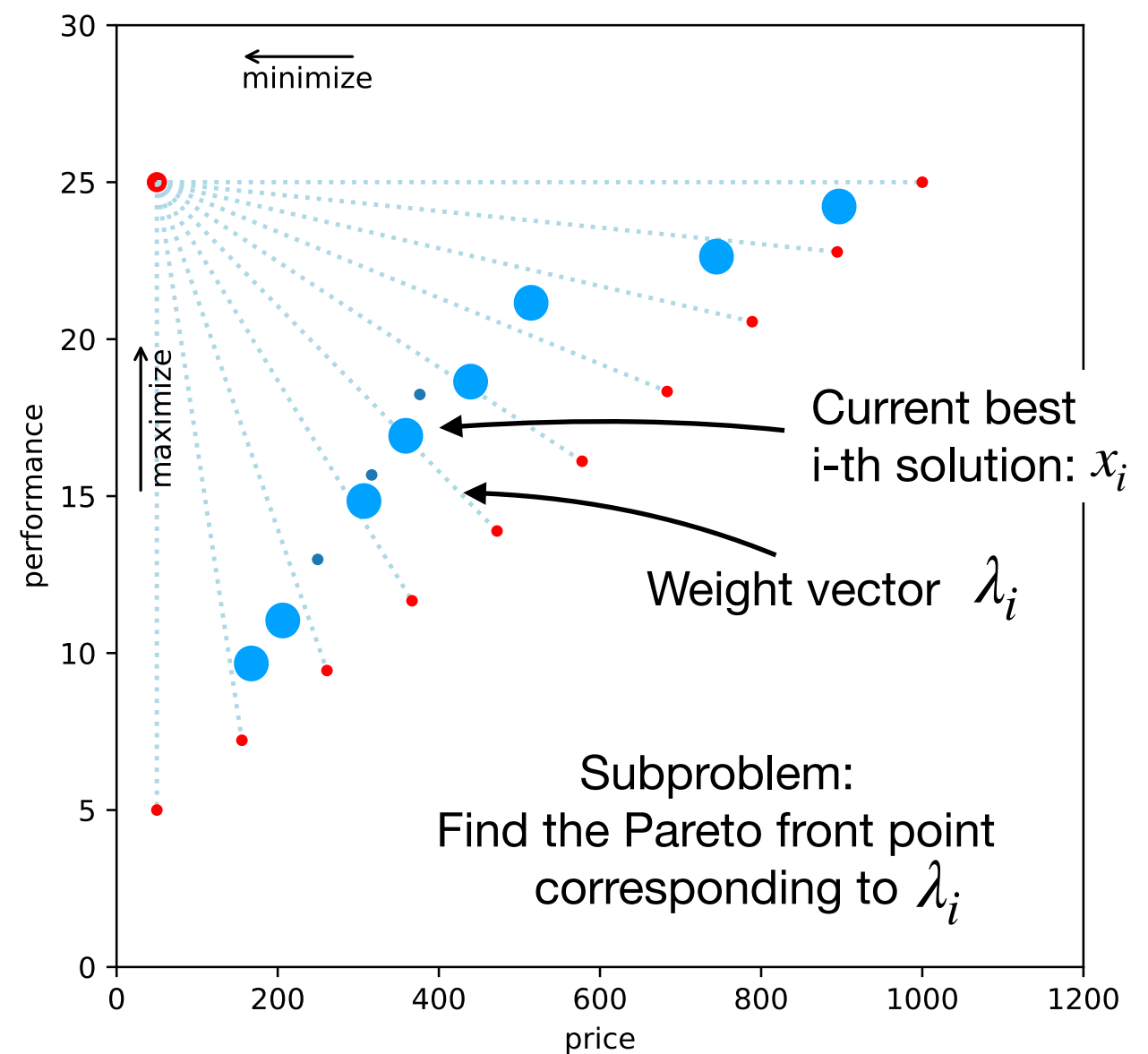
- Can't rely on Pareto dominance relation
- Hard to keep a good sampling of the Pareto front

➡ How to proceed ?

Decomposition of the problem into several problems

Decomposition approaches

- Basic idea:
 - **Decomposition:** N subtasks (mono- or multi-objective) targeting a particular area of the Pareto front
 - **Collaboration:**
 - N agents for N subproblems
 - Subproblems not independents, agents collaborate



MOEA/D

Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$

▷ Set of the best-so-far solutions

$\mathcal{B} \leftarrow \text{neighbourhood_init}(\lambda)$ ▷ $B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i

$P \leftarrow \text{rand_init}()$ ▷ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

for $i = 0, \dots, N$ do

$p_1, p_2 \leftarrow \text{parent_selection}(B_i)$

$q \leftarrow \text{offspring}(p_1, p_2)$

 for $j \in B(i)$ do

 if $f(q|\lambda_j) > f(x_j|\lambda_j)$ then

$x_j \leftarrow q$

 end if

 end for

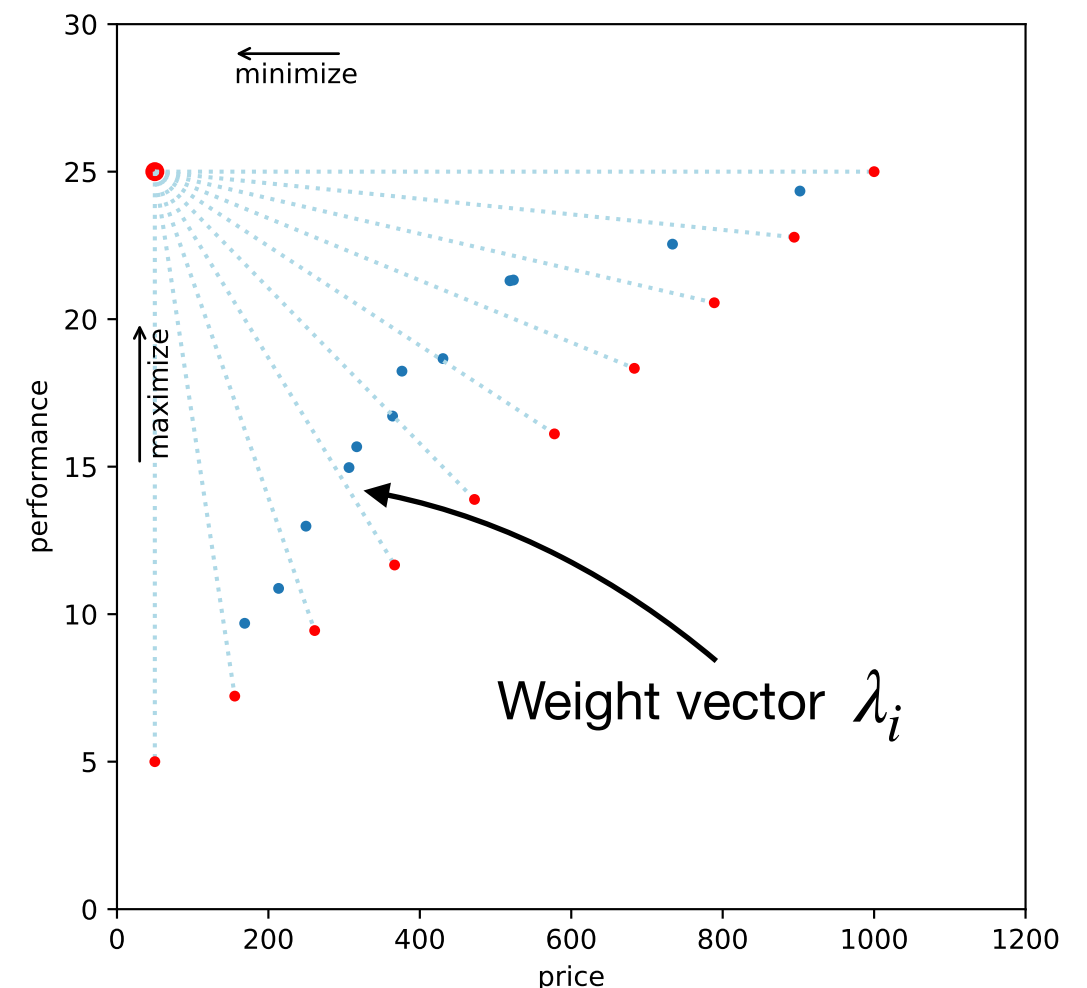
$EP \leftarrow \text{update_front}(q, EP)$

end for

Step 3, Stopping Criteria::

 If stopping criteria not met, go back to step 2

return EP



Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$ $\triangleright$ Set of the best-so-far solutions
 $B \leftarrow \text{neighbourhood_init}(\lambda)$ $\triangleright B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i
 $P \leftarrow \text{rand_init}()$ $\triangleright$ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

```

for  $i = 0, \dots, N$  do
   $p_1, p_2 \leftarrow \text{parent\_selection}(B_i)$ 
   $q \leftarrow \text{offspring}(p_1, p_2)$ 
  for  $j \in B(i)$  do
    if  $f(q|\lambda_j) > f(x_j|\lambda_j)$  then
       $x_j \leftarrow q$ 
    end if
  end for
   $EP \leftarrow \text{update\_front}(q, EP)$ 
end for

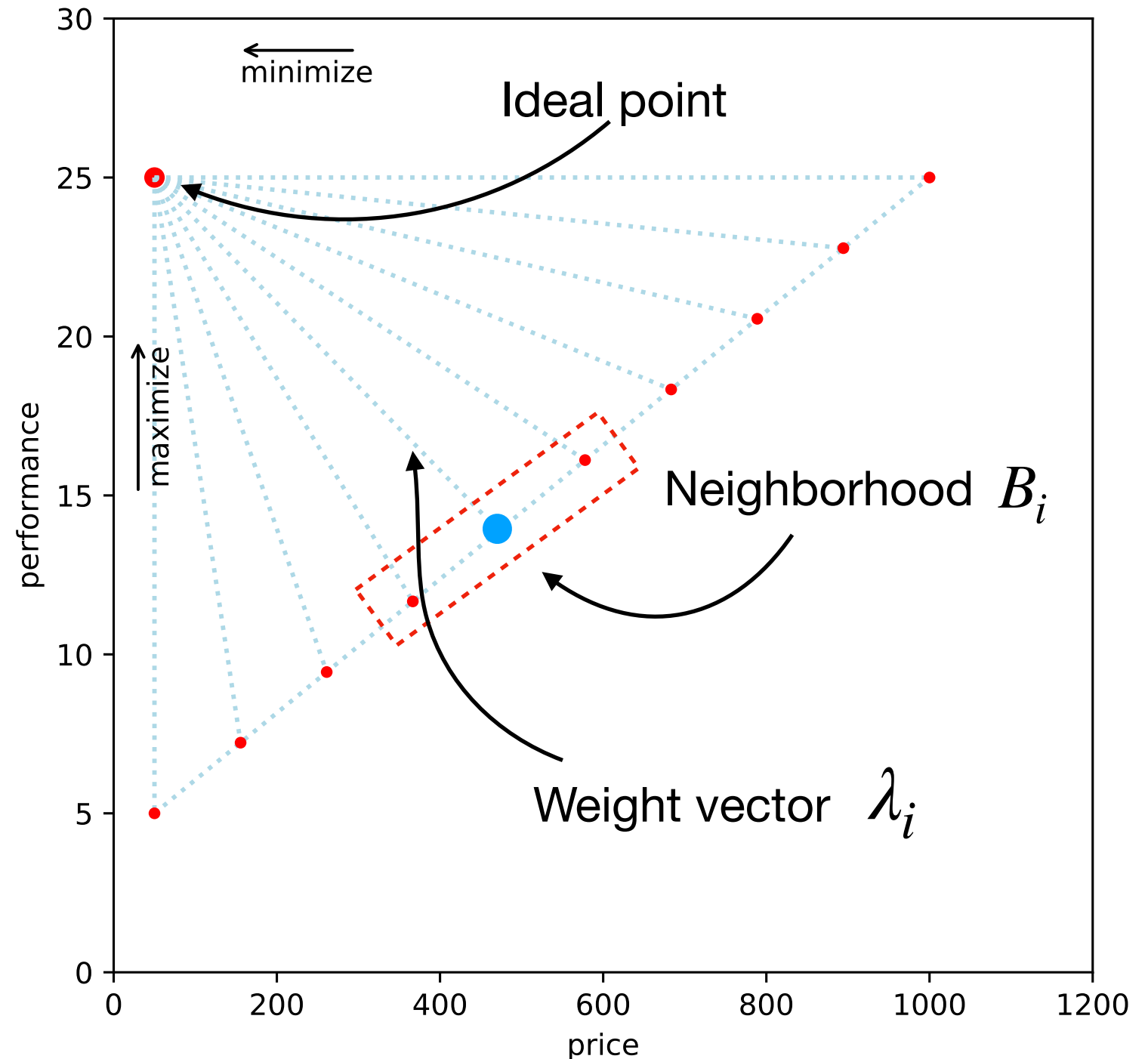
```

Step 3, Stopping Criteria::

If stopping criteria not met, go back to step 2

return EP

MOEA/D



Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$ $\triangleright$ Set of the best-so-far solutions
 $\mathcal{B} \leftarrow \text{neighbourhood_init}(\lambda)$ $\triangleright B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i
 $P \leftarrow \text{rand_init}()$ $\triangleright$ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

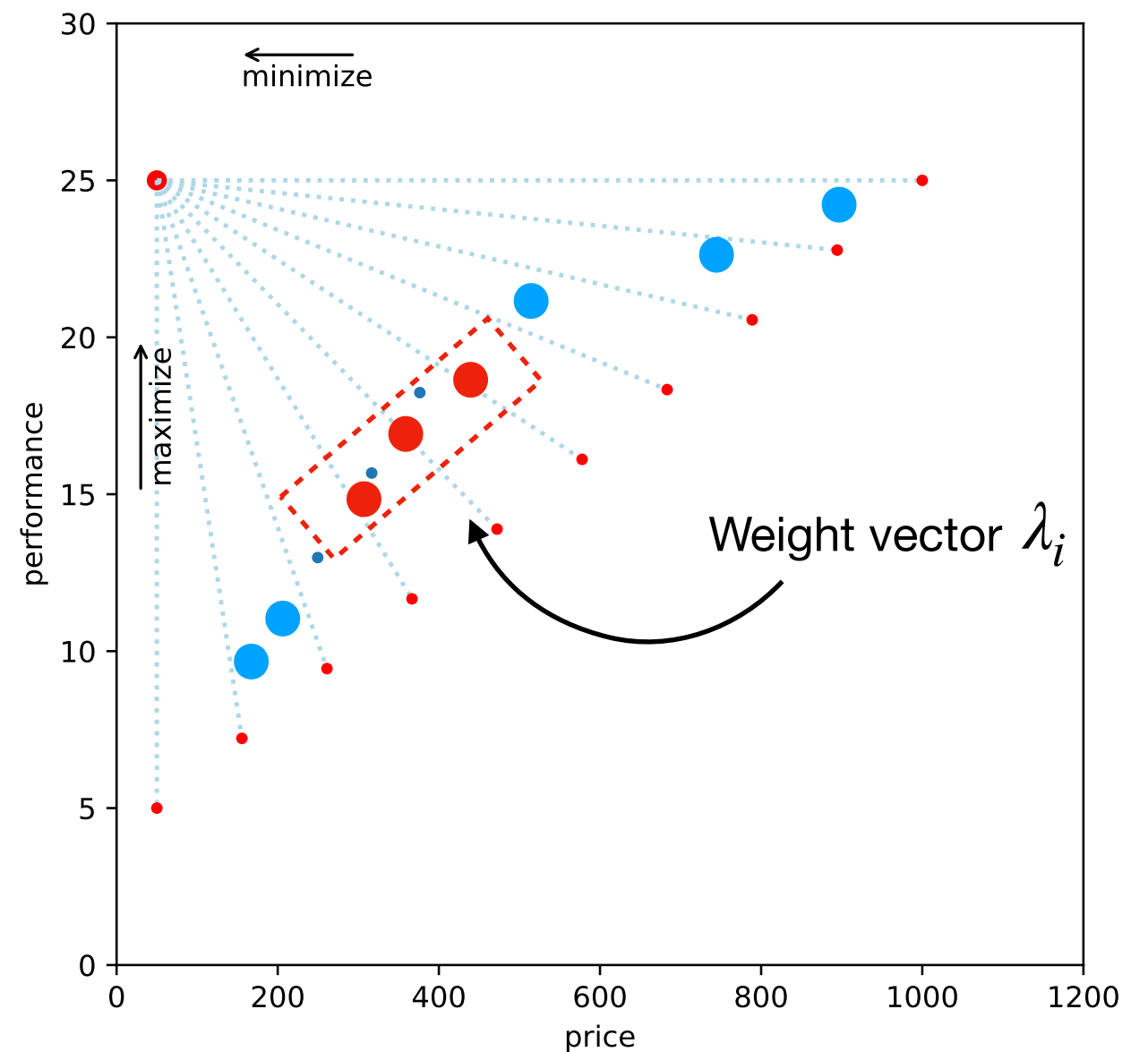
```
for  $i = 0, \dots, N$  do
   $p_1, p_2 \leftarrow \text{parent\_selection}(B_i)$ 
   $q \leftarrow \text{offspring}(p_1, p_2)$ 
  for  $j \in B(i)$  do
    if  $f(q|\lambda_j) > f(x_j|\lambda_j)$  then
       $x_j \leftarrow q$ 
    end if
  end for
   $EP \leftarrow \text{update\_front}(q, EP)$ 
end for
```

Step 3, Stopping Criteria::

If stopping criteria not met, go back to step 2

return EP

MOEA/D



Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$ ▷ Set of the best-so-far solutions
 $B \leftarrow neighbourhood_init(\lambda)$ ▷ $B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i
 $P \leftarrow rand_init()$ ▷ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

```

for  $i = 0, \dots, N$  do
   $p_1, p_2 \leftarrow parent\_selection(B_i)$ 
   $q \leftarrow offspring(p_1, p_2)$ 
  for  $j \in B(i)$  do
    if  $f(q|\lambda_j) > f(x_j|\lambda_j)$  then
       $x_j \leftarrow q$ 
    end if
  end for
   $EP \leftarrow update\_front(q, EP)$ 
end for

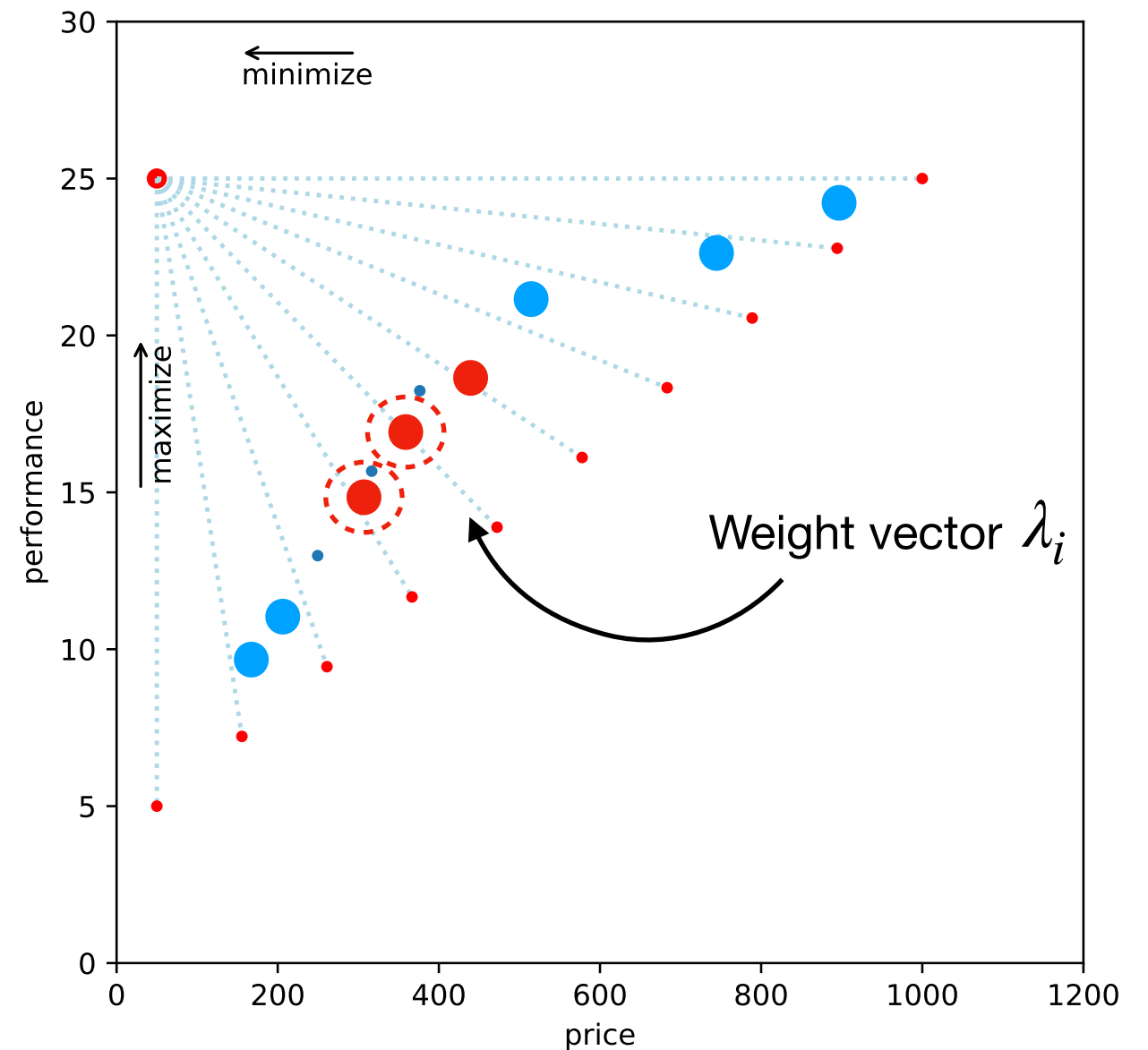
```

Step 3, Stopping Criteria::

If stopping criteria not met, go back to step 2

return EP

MOEA/D



MOEA/D

Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$ ▷ Set of the best-so-far solutions
 $B \leftarrow \text{neighbourhood_init}(\lambda)$ ▷ $B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i
 $P \leftarrow \text{rand_init}()$ ▷ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

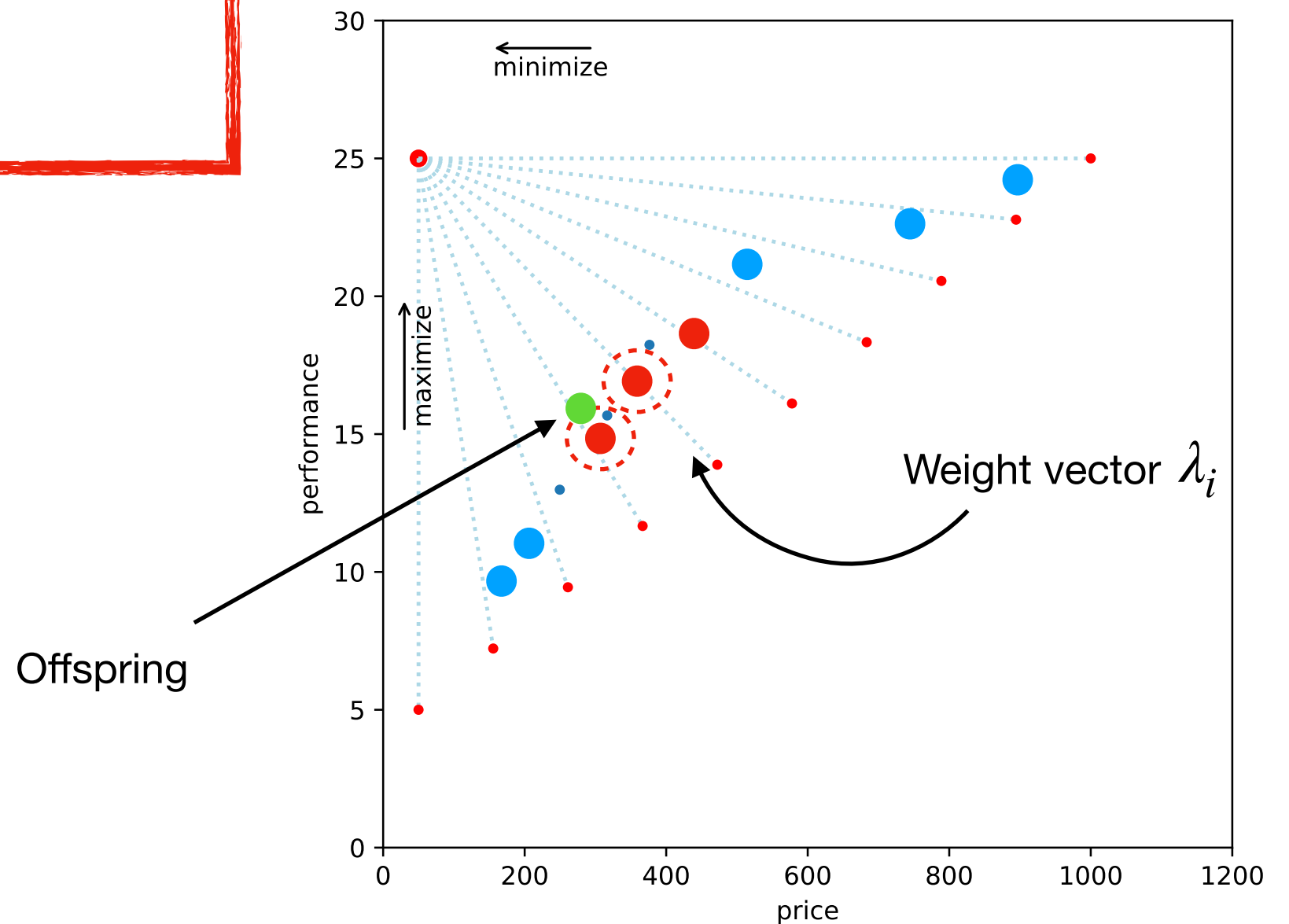
```

for  $i = 0, \dots, N$  do
   $p_1, p_2 \leftarrow \text{parent\_selection}(B_i)$ 
   $q \leftarrow \text{offspring}(p_1, p_2)$ 
  for  $j \in B(i)$  do
    if  $f(q|\lambda_j) > f(x_j|\lambda_j)$  then
       $x_j \leftarrow q$ 
    end if
  end for
   $EP \leftarrow \text{update\_front}(q, EP)$ 
end for
    
```

Step 3, Stopping Criteria::

If stopping criteria not met, go back to step 2

return EP



MOEA/D

Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$ ▷ Set of the best-so-far solutions
 $B \leftarrow \text{neighbourhood_init}(\lambda)$ ▷ $B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i
 $P \leftarrow \text{rand_init}()$ ▷ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

```

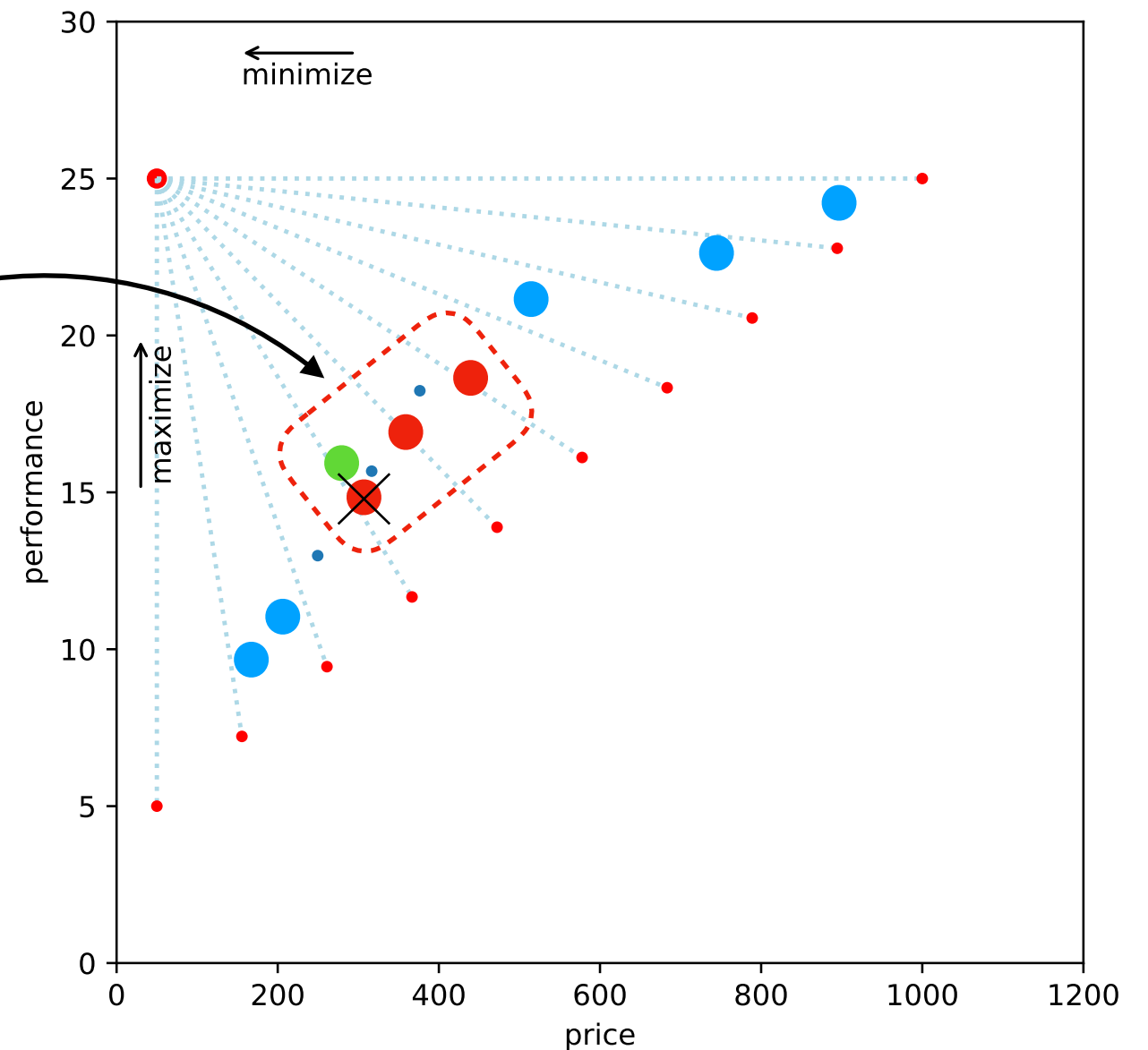
for  $i = 0, \dots, N$  do
   $p_1, p_2 \leftarrow \text{parent\_selection}(B_i)$ 
   $q \leftarrow \text{offspring}(p_1, p_2)$ 
  for  $j \in B(i)$  do
    if  $f(q|\lambda_j) > f(x_j|\lambda_j)$  then
       $x_j \leftarrow q$ 
    end if
  end for
   $EP \leftarrow \text{update\_front}(q, EP)$ 
end for
  
```

Step 3, Stopping Criteria::

If stopping criteria not met, go back to step 2

return EP

Comparison of the offspring with each current solution in B_i (weighted sum or Tchebytcheff) and replacement if better



Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$ ▷ Set of the best-so-far solutions
 $B \leftarrow neighbourhood_init(\lambda)$ ▷ $B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i
 $P \leftarrow rand_init()$ ▷ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

```
for  $i = 0, \dots, N$  do
   $p_1, p_2 \leftarrow parent\_selection(B_i)$ 
   $q \leftarrow offspring(p_1, p_2)$ 
  for  $j \in B(i)$  do
    if  $f(q|\lambda_j) > f(x_j|\lambda_j)$  then
       $x_j \leftarrow q$ 
    end if
  end for
   $EP \leftarrow update\_front(q, EP)$ 
end for
```

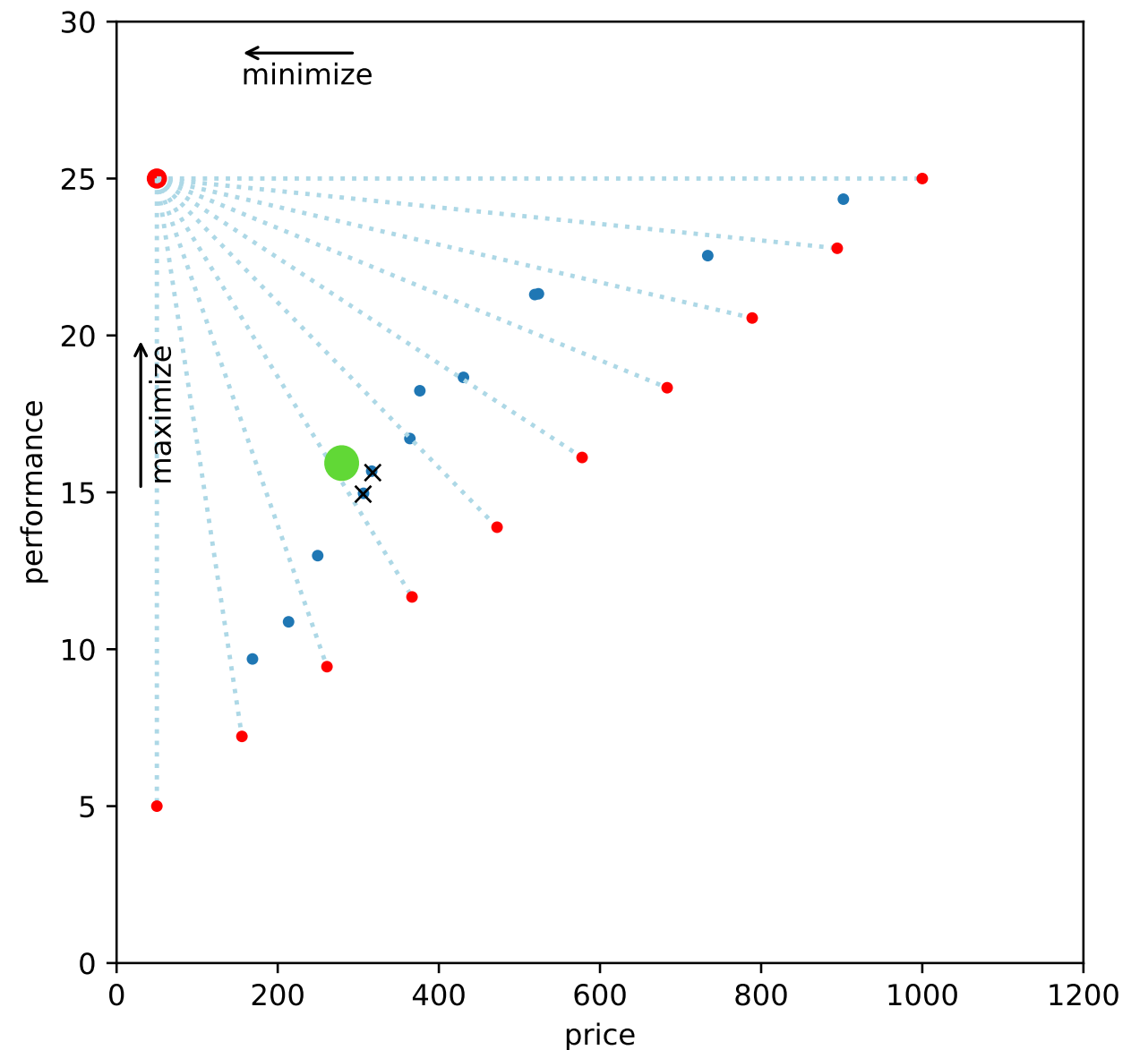
Step 3, Stopping Criteria::

If stopping criteria not met, go back to step 2

return EP

Update of EP, the current estimate of the Pareto Front

MOEA/D



MOEA/D

Input: number of subproblems N , size of the neighbourhood T , $\lambda = \{\lambda_i\}$ weight vectors defining the subproblems

Output: set of non-dominated solutions EP

Step 1, Initialization:

$EP \leftarrow \emptyset$ ▷ Set of the best-so-far solutions
 $B \leftarrow neighbourhood_init(\lambda)$ ▷ $B_i = \{i_1, \dots, i_T\}$ where $\lambda_{i_1}, \dots, \lambda_{i_T}$ are the T -closest weight vectors to λ_i
 $P \leftarrow rand_init()$ ▷ Initialization of $P = \{x_i\}_{i=0}^{N-1}$ the set of current solutions to the N subproblems

Step 2, Update:

```

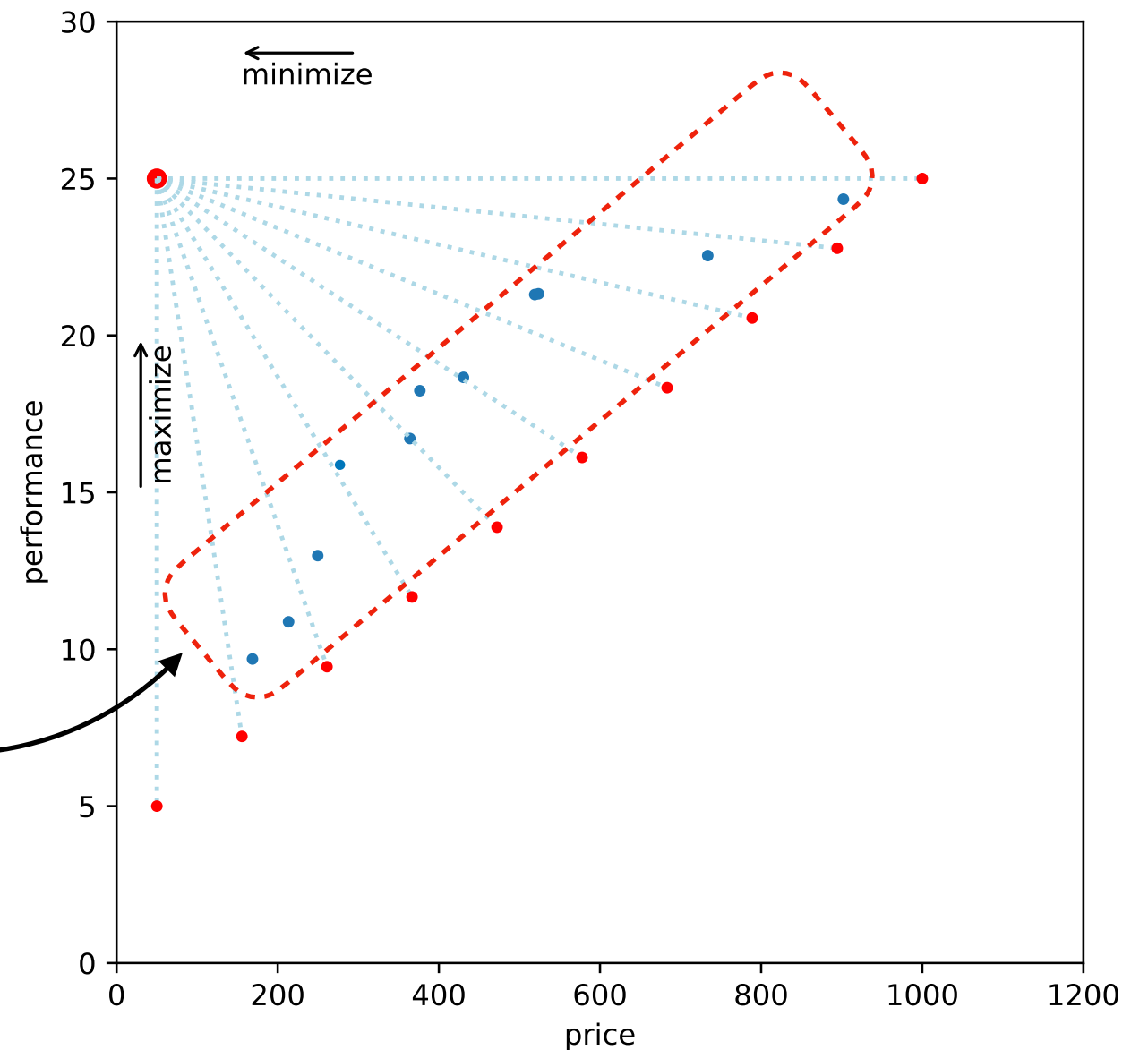
for  $i = 0, \dots, N$  do
   $p_1, p_2 \leftarrow parent\_selection(B_i)$ 
   $q \leftarrow offspring(p_1, p_2)$ 
  for  $j \in B(i)$  do
    if  $f(q|\lambda_j) > f(x_j|\lambda_j)$  then
       $x_j \leftarrow q$ 
    end if
  end for
   $EP \leftarrow update\_front(q, EP)$ 
end for
  
```

Step 3, Stopping Criteria::

If stopping criteria not met, go back to step 2

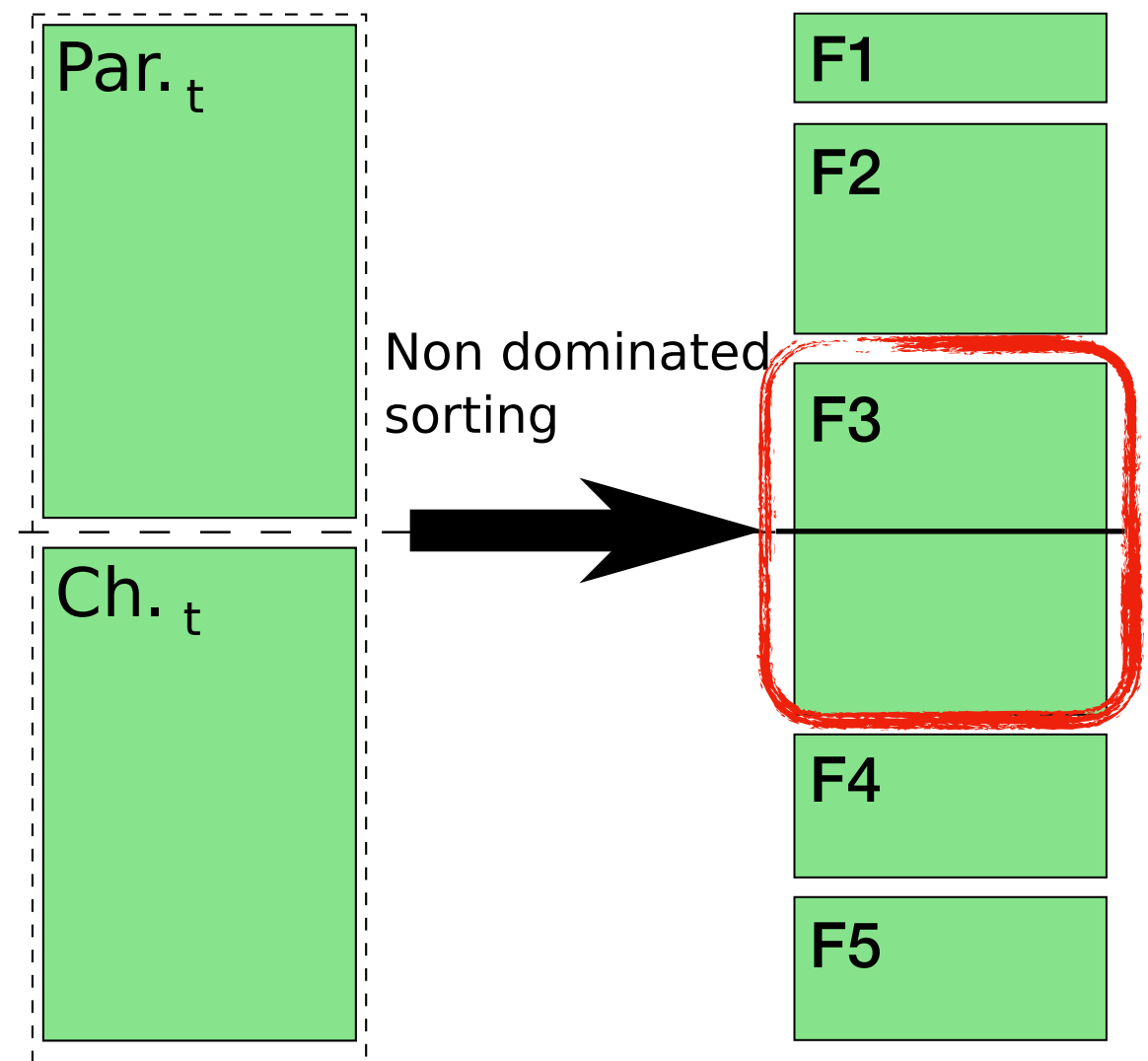
return EP

EP



NSGA-III: adaptation of NSGA-II for many objectives

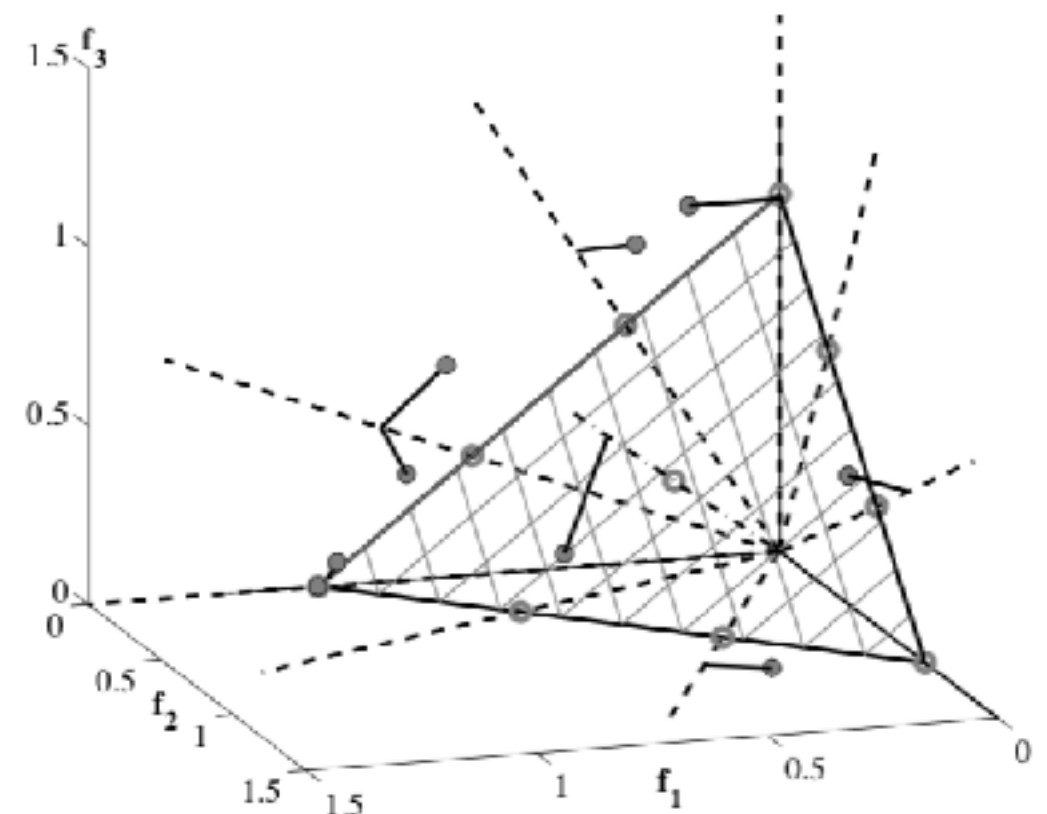
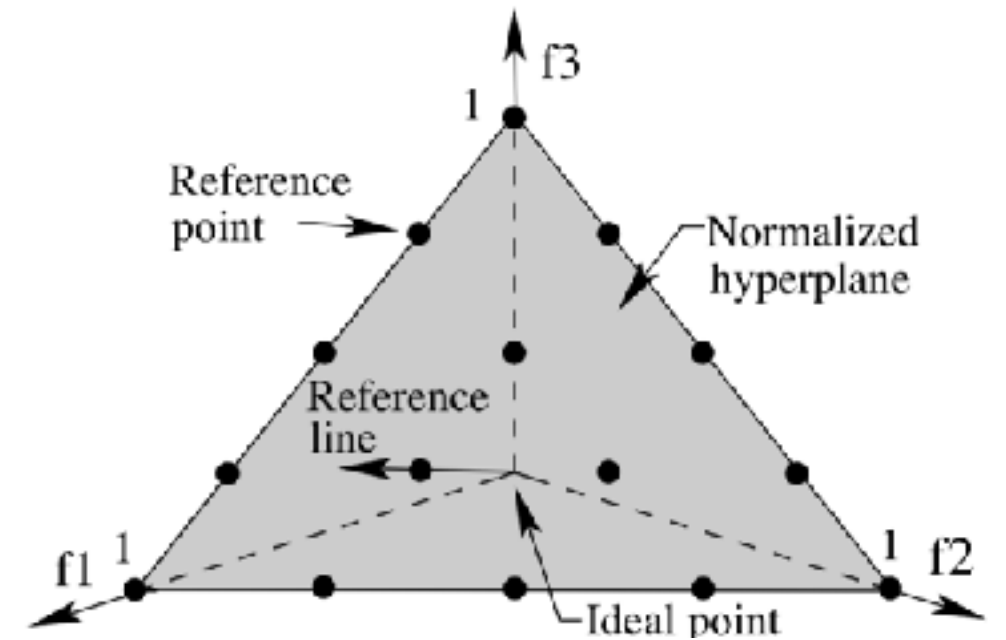
- Same principle as NSGA-II
- Modified selection of points from a front



Deb, K., & Jain, H. (2013). An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. *IEEE transactions on evolutionary computation*, 18(4), 577-601.

NSGA-III: adaptation of NSGA-II for many objectives

- Goal: complete P_{t+1} with points from front F_l
- Each point of P_{t+1} is associated to a single reference point
- Selected points are those associated to reference points having the smallest niche count.



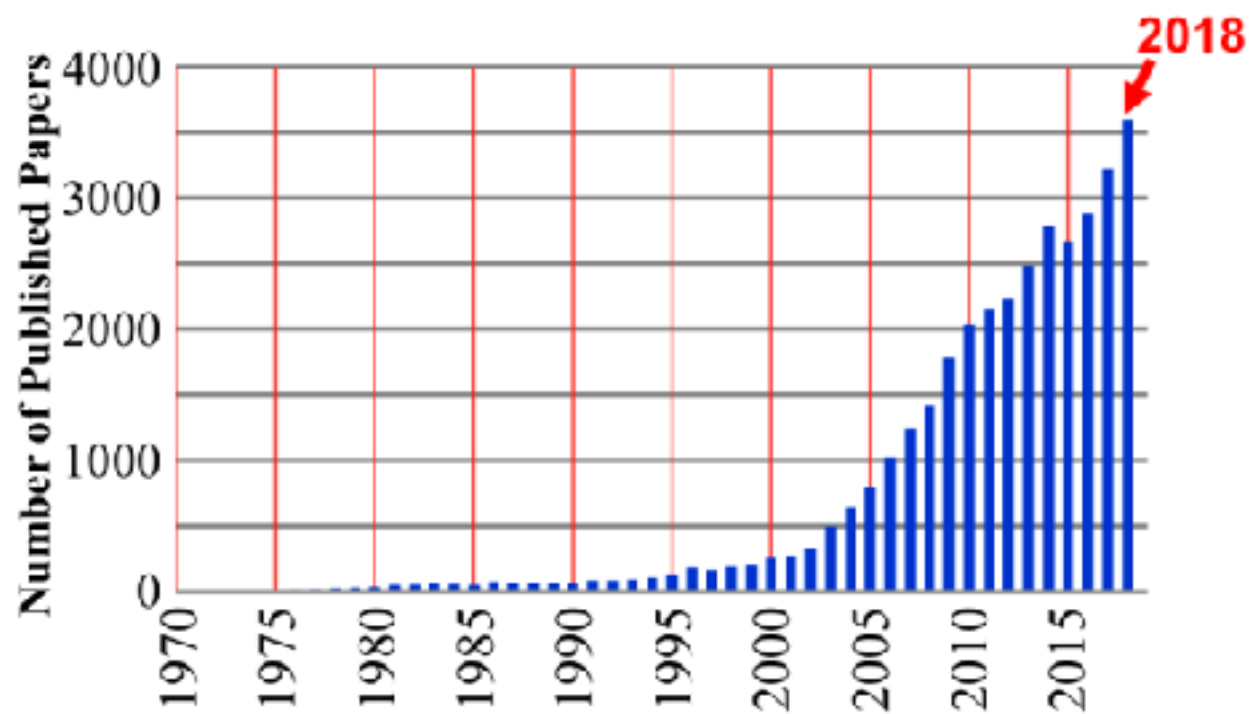
Exemple NSGA-III

- see 4_EA_Many-Obj.ipynb, test on ZDT2
- nb objectifs:9, pop size: 100, nbgen: 400
- Pareto front size: NSGA2 6965, NSGA3 11919
- IGD (inverted generational distance, i.e. sum of the distances from ref points to the closest point in the pareto front, closer to 0 = better):
 - NSGA2-ref: 0.533484
 - NSGA3-ref: 0.230875
- NSGA2 dominated points: 6391 (on 6965)
- NSGA3 dominated points: 8 (on 11919)

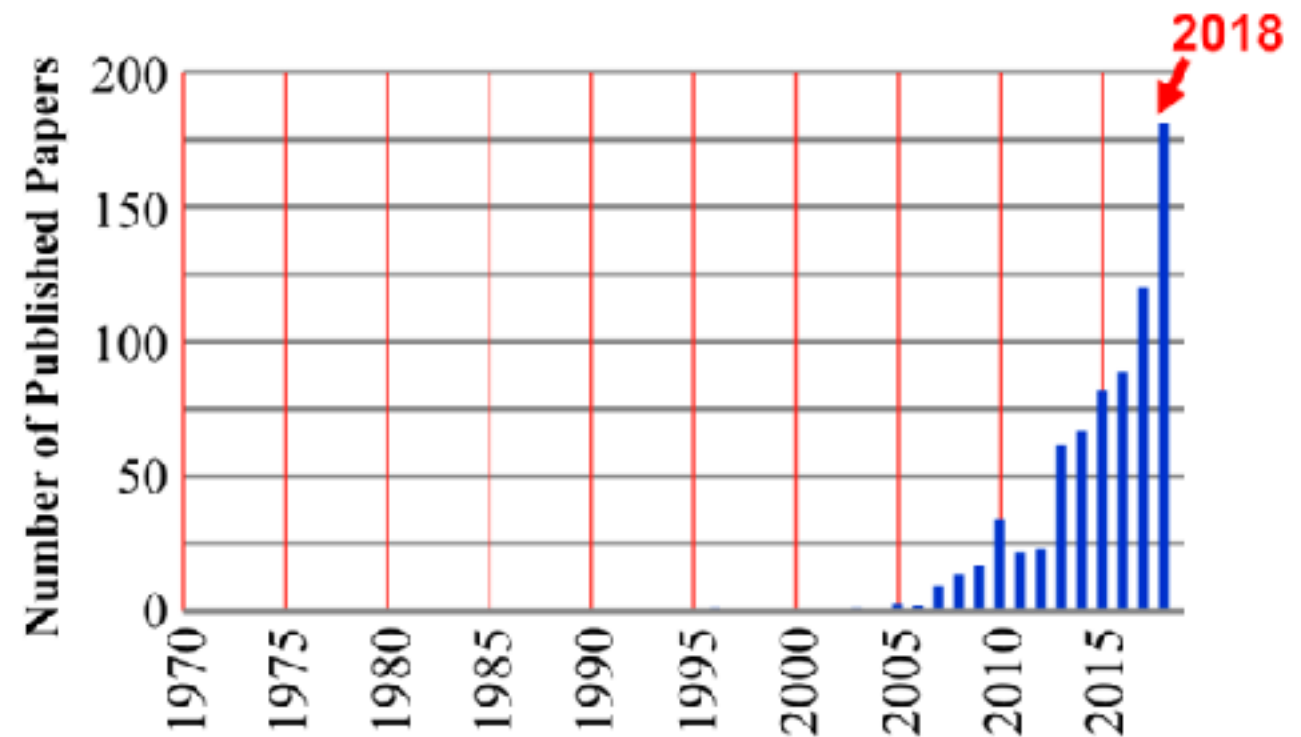


Conclusion

An evolving field...



Articles with « Multi-objective » in the title



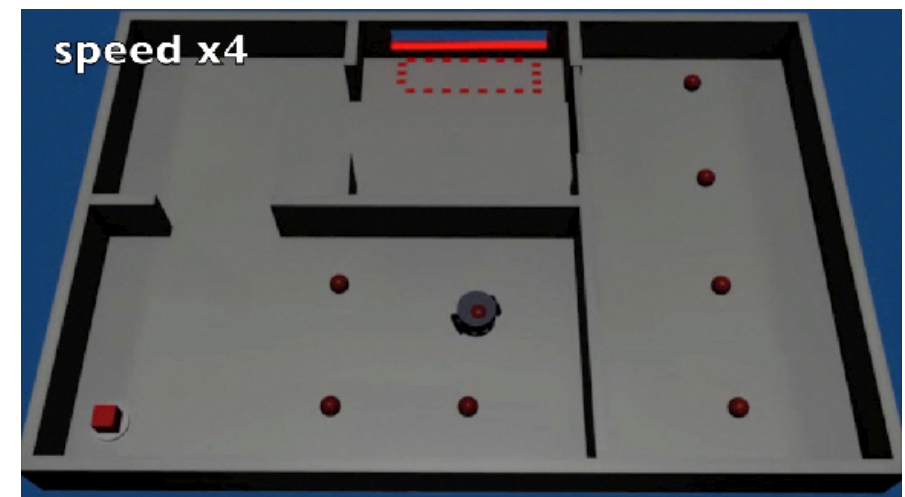
Articles with « Many-objective » in the title

- Some relevant conferences:
 - GECCO: Genetic and Evolutionary Computation Conference (look at its tutorials !)
 - EMO: Evolutionary Multiobjective Optimization

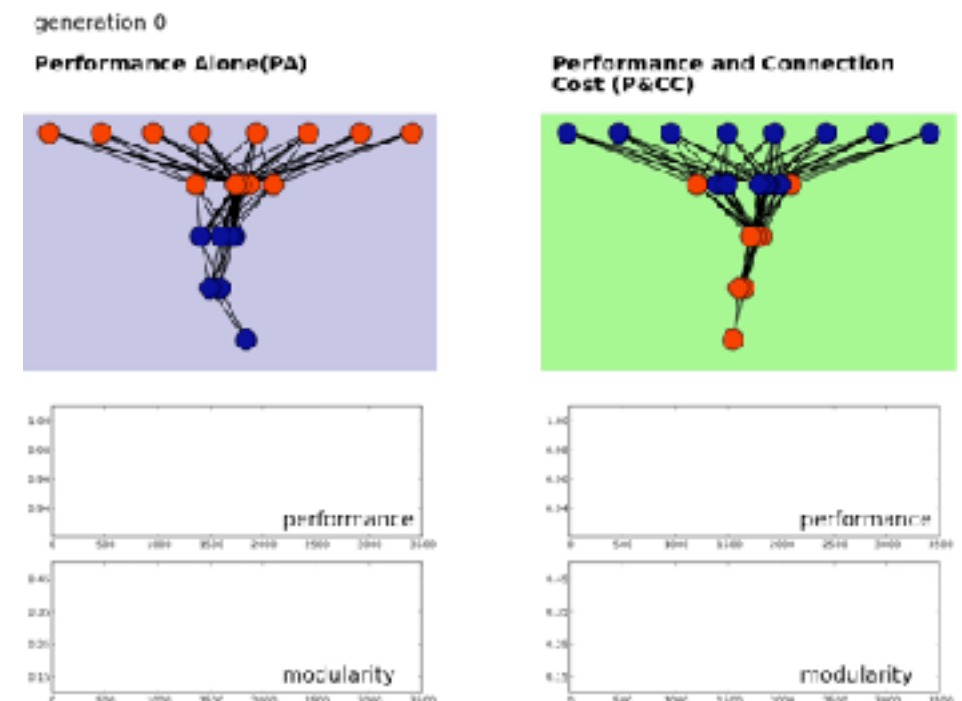
Back to learning & robotics: what is it useful for?



Crossing the reality gap



Avoiding premature convergence



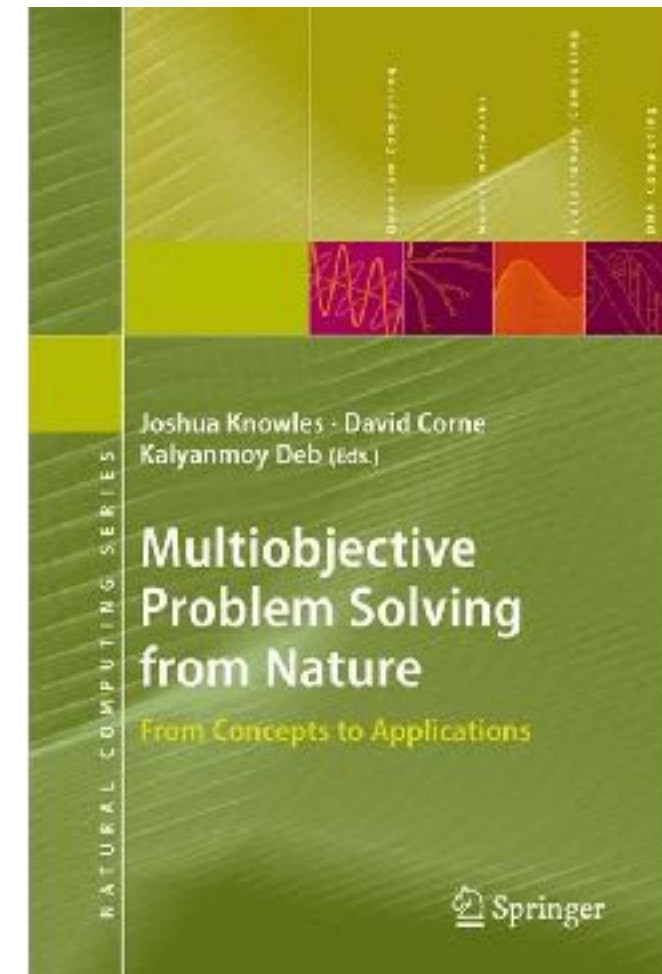
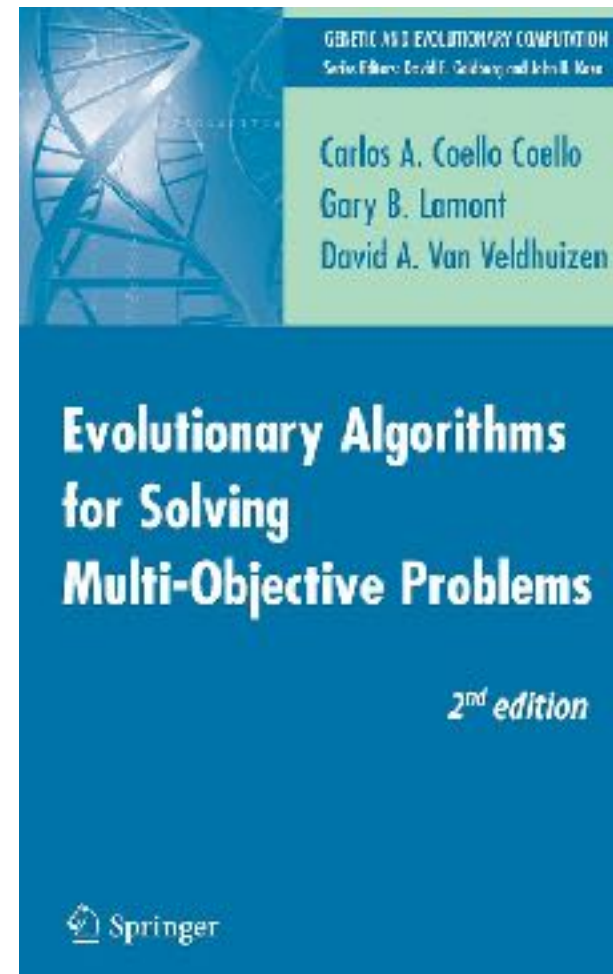
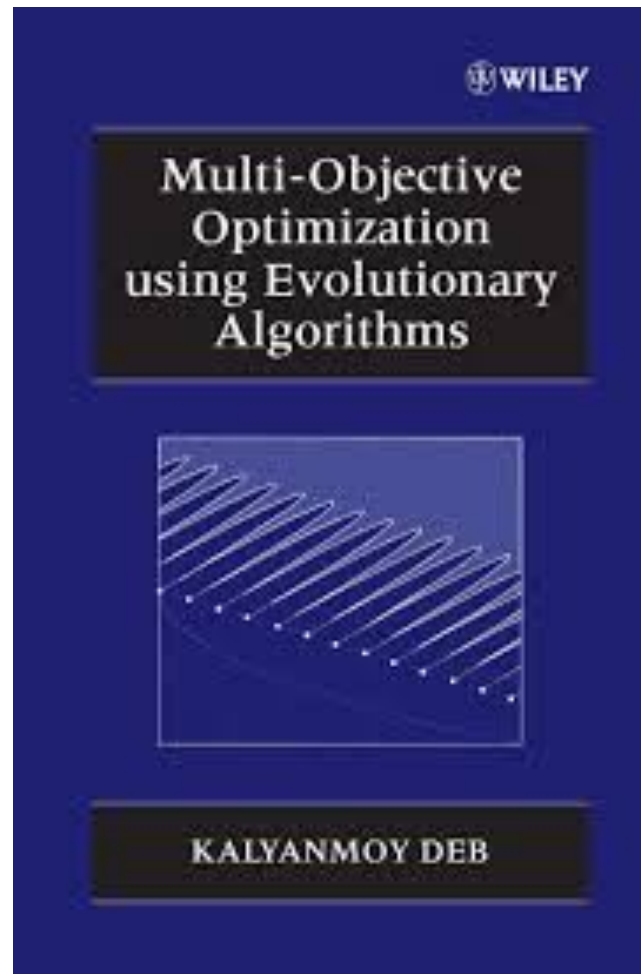
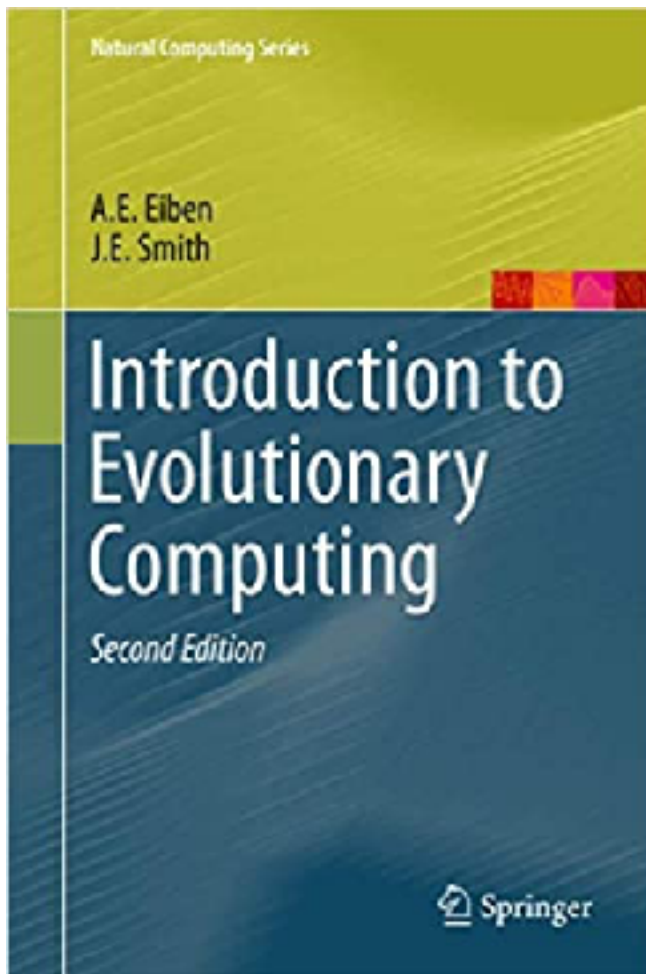
Generate modular NN

Take home messages

- Weighted sum is not a panacea
- Turning a single-objective problem into a multi-objective one may help, in particular in robotics [1]
- Optimisation vs decision: getting a set of Pareto optimal points is precious to understand what is possible and find relations between parameters
- Evolutionary algorithms are particularly adapted for multi- and many- objective optimisation

[1] Doncieux, S., & Mouret, J. B. (2014). Beyond black-box optimization: a review of selective pressures for evolutionary robotics. *Evolutionary Intelligence*, 7(2), 71-93.

To go further...



Thank you for listening !



stephane.doncieux@sorbonne-universite.fr



<https://twitter.com/SDoncieux>



<http://people.isir.upmc.fr/doncieux>

